

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC QUY NHƠN



KHÓA LUẬN TỐT NGHIỆP ĐẠI HỌC

NGÀNH: CÔNG NGHỆ PHẦN MỀM

**Tên đề tài: ỨNG DỤNG KIẾN TRÚC HYBRID (EDGE-CLOUD) VÀ
DEEP LEARNING TRONG BÀI TOÁN NHẬN DIỆN HÀNH VI TÀI XẾ
XE KHÁCH**

Người hướng dẫn : TS. Nguyễn Thanh Bình

Sinh viên thực hiện : La Đại Lộc

Mã số sinh viên : 4551050116

Lớp : Công nghệ thông tin 45B

Gia Lai, tháng 6 năm 2026

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC QUY NHƠN



KHÓA LUẬN TỐT NGHIỆP ĐẠI HỌC

NGÀNH: CÔNG NGHỆ PHẦN MỀM

**Tên đề tài: ỨNG DỤNG KIẾN TRÚC HYBRID (EDGE-CLOUD) VÀ
DEEP LEARNING TRONG BÀI TOÁN NHẬN DIỆN HÀNH VI TÀI XẾ
XE KHÁCH**

Người hướng dẫn : TS. Nguyễn Thanh Bình

Sinh viên thực hiện : La Đại Lộc

Mã số sinh viên : 4551050116

Lớp : Công nghệ thông tin 45B

Gia Lai, tháng 6 năm 2026

LỜI CẢM ƠN

Trước hết, em xin bày tỏ lòng biết ơn sâu sắc đến Ban Giám hiệu Trường Đại học Quy Nhơn, quý thầy cô Khoa Công nghệ Thông tin đã tận tình giảng dạy, truyền đạt cho em những kiến thức nền tảng và chuyên môn quý báu trong suốt quá trình học tập tại trường. Những kiến thức, kỹ năng và kinh nghiệm mà em được tiếp thu chính là hành trang quan trọng giúp em có thể thực hiện và hoàn thành khóa luận tốt nghiệp này.

Em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến thầy TS. Nguyễn Thanh Bình, giảng viên hướng dẫn, người đã tận tình định hướng, chỉ bảo và góp ý cho em trong suốt quá trình thực hiện đề tài. Những nhận xét, góp ý chuyên môn và sự hỗ trợ của thầy đã giúp em từng bước hoàn thiện nội dung nghiên cứu, từ việc xác định hướng tiếp cận, xây dựng cơ sở lý thuyết, thiết kế hệ thống cho đến quá trình trình bày và hoàn thiện báo cáo.

Em cũng xin chân thành cảm ơn anh Đỗ Minh Đức, đồng hướng dẫn, đã dành thời gian hỗ trợ, góp ý và giúp em tháo gỡ những khó khăn trong quá trình nghiên cứu và triển khai đề tài. Sự hướng dẫn tận tình của anh là nguồn động viên lớn giúp em có thêm sự tự tin để hoàn thành khóa luận.

Mặc dù em đã cố gắng nghiên cứu và hoàn thiện báo cáo một cách nghiêm túc, song do kiến thức, kinh nghiệm thực tế và thời gian thực hiện còn hạn chế, khóa luận chắc chắn không tránh khỏi những thiếu sót. Em rất mong nhận được sự góp ý, nhận xét của quý thầy cô để đề tài được hoàn thiện hơn, đồng thời giúp em tích lũy thêm kinh nghiệm cho quá trình học tập và làm việc sau này.

Em xin chân thành cảm ơn!

MỤC LỤC

DANH MỤC HÌNH ẢNH.....	1
DANH MỤC BẢNG BIỂU	2
DANH MỤC CÁC TỪ VIẾT TẮT.....	3
MỞ ĐẦU	4
CHƯƠNG 1: TỔNG QUAN VỀ BÀI TOÁN VÀ CÔNG NGHỆ.....	5
1.1. Tổng quan hệ thống giám sát hành vi tài xế (DMS)	5
1.1.1. Sự cần thiết của hệ thống DMS.....	5
1.1.2. Hạn chế của các phương pháp truyền thống	6
1.1.3. Thách thức trong triển khai thực tế	7
1.2. Edge AI Computing.....	8
1.2.1. Khái niệm Edge AI.....	8
1.2.2. So sánh Edge AI và Cloud AI	9
1.2.3. Ưu điểm của Edge AI trong bài toán DMS	11
1.3. Kiến trúc Hybrid (Edge–Cloud)	12
1.3.1. Triết lý mô hình Hybrid.....	12
1.3.2. Vai trò và luồng xử lý tại Edge Node.....	13
1.3.3. Vai trò của Cloud Node.....	15
1.3.4. Lợi thế và giá trị của kiến trúc Hybrid	16
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ.....	17
2.1. Bài toán phát hiện đối tượng và mô hình YOLO	17
2.1.1. Tổng quan bài toán Object Detection.....	17
2.1.2. Kiến trúc mô hình YOLO.....	19
2.1.3. Ứng dụng YOLO trong bài toán DMS	22
2.2. Ước lượng tư thế và MediaPipe Pose	24
2.2.1. Tổng quan bài toán Pose Estimation	24
2.2.2. Framework MediaPipe Pose.....	26
2.2.3. Ứng dụng MediaPipe Pose trong phân tích hành vi tài xế	29
2.3. Nhận diện hành động (Action Recognition).....	30
2.3.1. Tổng quan bài toán Action Recognition	30
2.3.2. Mô hình SlowFast	32

2.3.3.	Đặc trưng không gian – thời gian.....	34
2.4.	Tối ưu hóa mô hình cho thiết bị Edge	35
2.4.1.	Quantization (Lượng tử hóa).....	35
2.4.2.	TensorRT	36
2.4.3.	Tối ưu hiệu năng trên thiết bị nhúng	38
CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG.....		40
3.1.	Kiến trúc tổng thể hệ thống	40
3.1.1.	Mô hình hệ thống tổng thể	40
3.1.2.	Luồng dữ liệu và giao thức giao tiếp.....	40
3.2.	Thiết kế phân hệ Edge	44
3.2.1.	Pipeline xử lý tại Edge	44
3.2.2.	Kết hợp YOLO và MediaPipe Pose	47
3.2.3.	Cơ chế phát hiện và cảnh báo.....	50
3.3.	Thiết kế phân hệ Cloud.....	50
3.3.1.	Kiến trúc Backend	50
3.3.2.	Cơ chế nhận và xử lý dữ liệu.....	52
3.3.3.	Xác thực hành vi và giám cảnh báo sai	54
3.4.	Thiết kế cơ sở dữ liệu, API và giao diện giám sát.....	56
3.4.1.	Thiết kế cơ sở dữ liệu	56
3.4.2.	Giao diện giám sát (Alert Center)	57
CHƯƠNG 4: XÂY DỰNG VÀ THỰC NGHIỆM.....		60
4.1.	Môi trường và công cụ triển khai	60
4.2.	Thu thập, gán nhãn và tiền xử lý dữ liệu	61
4.3.	Huấn luyện và tối ưu mô hình	63
4.3.1.	Tối ưu hóa bằng chiến lược trích mẫu thời gian (Frame Skipping) ..	64
4.3.2.	Tối ưu hóa bằng chuẩn hóa độ phân giải không gian.....	64
4.3.3.	Biên dịch và gia tốc phần cứng với TensorRT.....	65
4.4.	Triển khai hệ thống và kiểm thử.....	65
4.4.1.	Triển khai phân hệ Edge (Thiết bị biên)	65
4.4.2.	Triển khai phân hệ Cloud (Đám mây trung tâm)	66
4.4.3.	Thẩm định và kiểm thử kịch bản (System Validation)	67
4.5.	Đánh giá hiệu năng và kết quả	68

4.5.1.	Đánh giá hiệu năng xử lý (FPS)	68
4.5.2.	Đánh giá độ chính xác thực tế	70
4.5.3.	Phân tích các trường hợp lỗi (Error Analysis)	72
4.5.4.	Đánh giá cơ chế lưu trữ và truy vết bằng chứng	72
CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN		75
5.1.	Kết quả đạt được	75
5.2.	Hạn chế của hệ thống	76
5.3.	Hướng phát triển trong tương lai	77
TÀI LIỆU THAM KHẢO		79

DANH MỤC HÌNH ẢNH

Hình 2.1.2. Kiến trúc mô hình YOLO gồm Backbone, Neck và Head.....	21
Hình 2.2.2. Minh họa 13 landmarks được sử dụng trong hệ thống	28
Hình 2.3.2. Kiến trúc SlowFast 2-pathway gồm Slow Pathway, Fast Pathway và lateral connections	33
Hình 3.1.1. Kiến trúc tổng thể hệ thống Hybrid Edge–Cloud	40
Hình 3.1.2. Luồng xử lý cục bộ tại Edge trước khi đồng bộ lên Cloud	41
Hình 3.1.3. Luồng đồng bộ dữ liệu từ Edge lên Cloud.....	42
Hình 3.1.4. Pipeline xử lý dữ liệu tại Cloud	43
Hình 3.1.5. Sequence diagram luồng xử lý và đồng bộ cảnh báo vi phạm	44
Hình 3.2.1. Pipeline xử lý tại Edge.....	45
Hình 3.2.2. Cơ chế kết hợp YOLO và MediaPipe Pose trong suy luận hành vi	48
Hình 3.2.3. Minh họa Driver ROI và Chest ROI trong khoang lái	49
Hình 3.3.1. Kiến trúc phân lớp của Backend Cloud.....	51
Hình 3.3.2. Quy trình tiếp nhận và xử lý dữ liệu cảnh báo tại Cloud.....	53
Hình 3.3.3. Cơ chế xác thực hành vi tại Cloud	55
Hình 3.4.1. Cấu trúc cơ sở dữ liệu bảng Alerts	56
Hình 3.4.2. Dashboard tổng quan hệ thống giám sát	58
Hình 3.4.3. Alerts Center quản lý danh sách cảnh báo.....	59
Hình 3.4.4. Evidence Modal dùng để xem và xác thực bằng chứng cảnh báo	59
Hình 4.4.1. Pipeline Edge đang chạy với detection, pose và FPS.....	66
Hình 4.4.3.1. Ví dụ kiểm thử phát hiện hút thuốc	67
Hình 4.4.3.2. Ví dụ kiểm thử phát hiện sử dụng điện thoại	68
Hình 4.5.1. Ma trận nhầm lẫn của mô hình nhận diện hành vi trên tập kiểm thử ...	71

DANH MỤC BẢNG BIỂU

Bảng 1.1. So sánh Edge AI và Cloud AI	10
Bảng 2.1. Danh sách 13 điểm mốc MediaPipe Pose sử dụng trong hệ thống.....	27
Bảng 3.1. Cấu hình ngưỡng xác nhận vi phạm theo từng loại hành vi	46
Bảng 3.2. Các tham số cấu hình chính của YOLO và Behavior Rules Engine.....	50
Bảng 3.3. Quy trình tiếp nhận và xử lý dữ liệu cảnh báo tại Cloud	54
Bảng 3.4. Cấu trúc các trường dữ liệu của bảng Alerts	57
Bảng 4.1. Thông lượng xử lý (FPS) trung bình theo các kịch bản cấu hình	69
Bảng 4.2. Kết quả đánh giá mô hình YOLO trên tập kiểm thử	70
Bảng 4.3. Hiệu suất nhận diện hành vi trên tập kiểm thử	71

DANH MỤC CÁC TỪ VIẾT TẮT

STT	Từ viết tắt	Giải thích
1	AI	Artificial Intelligence
2	API	Application Programming Interface
3	CNN	Convolutional Neural Network
4	CPU	Central Processing Unit
5	CSV	Comma-Separated Values
6	DMS	Driver Monitoring System
7	Edge AI	Edge Artificial Intelligence
8	FPS	Frames Per Second
9	GPU	Graphics Processing Unit
10	HTTP	HyperText Transfer Protocol
11	IoU	Intersection over Union
12	mAP	mean Average Precision
13	NMS	Non-Maximum Suppression
14	NPU	Neural Processing Unit
15	REST	Representational State Transfer
16	ROI	Region of Interest
17	SPA	Single Page Application
18	TPU	Tensor Processing Unit
19	YOLO	You Only Look Once

MỞ ĐẦU

Đề tài "Ứng dụng kiến trúc Hybrid (Edge-Cloud) và Deep Learning trong bài toán nhận diện hành vi tài xế xe khách" hướng tới việc xây dựng hệ thống giám sát hành vi tài xế (Driver Monitoring System - DMS) có khả năng phát hiện và cảnh báo thời gian thực các hành vi nguy hiểm trong buồng lái xe khách, bao gồm: sử dụng điện thoại, hút thuốc và không thắt dây an toàn.

Hệ thống được thiết kế theo kiến trúc phân tán Hybrid Edge–Cloud. Trong đó, phân hệ Edge triển khai trên thiết bị nhúng NVIDIA Jetson để xử lý luồng video tại chỗ bằng cách kết hợp YOLO và MediaPipe Pose. Phân hệ Cloud sử dụng FastAPI, SQLite và giao diện Dashboard bằng React để tiếp nhận, lưu trữ và hỗ trợ quản lý các cảnh báo vi phạm. Cách tổ chức này giúp hệ thống vừa đáp ứng yêu cầu phản hồi gần thời gian thực tại phương tiện, vừa hỗ trợ truy vết và giám sát tập trung ở phía quản trị.

Kết quả thực nghiệm cho thấy hệ thống đạt tốc độ xử lý FPS đáp ứng tiêu chuẩn thời gian thực trên thiết bị Edge, đồng thời có khả năng lọc cảnh báo sai thông qua cơ chế xác thực ngữ cảnh hai tầng (Edge + Cloud).

CHƯƠNG 1: TỔNG QUAN VỀ BÀI TOÁN VÀ CÔNG NGHỆ

1.1. Tổng quan hệ thống giám sát hành vi tài xế (DMS)

1.1.1. Sự cần thiết của hệ thống DMS

Trong bối cảnh giao thông hiện đại, sự phát triển bùng nổ của hạ tầng và sự gia tăng nhanh chóng của các phương tiện cơ giới đã kéo theo những hệ lụy nghiêm trọng về an toàn giao thông. Tai nạn giao thông hiện không chỉ là bài toán về an sinh xã hội mà còn là nguyên nhân hàng đầu gây ra những thiệt hại kinh tế lớn trên quy mô toàn cầu. Theo các báo cáo thống kê từ Tổ chức Y tế Thế giới (WHO) và các cơ quan quản lý an toàn giao thông quốc tế, phần lớn các vụ va chạm nghiêm trọng không xuất phát từ lỗi kỹ thuật của phương tiện, mà bắt nguồn từ yếu tố chủ quan của con người. Trong đó, sự mất tập trung (Distracted Driving) và việc coi thường các quy tắc an toàn cơ bản chiếm tỷ trọng cao nhất trong chuỗi nguyên nhân trực tiếp dẫn đến tai nạn.

Nhìn nhận dưới góc độ hành vi học, sự vi phạm của người điều khiển phương tiện thường được phân ly thành hai nhóm đặc trưng. Nhóm thứ nhất là sự suy giảm nhận thức không gian và thao tác vật lý, điển hình là các hành vi rời mắt khỏi làn đường hoặc buông tay khỏi vô lăng để sử dụng điện thoại di động, hút thuốc. Hành động này cắt đứt luồng thông tin thị giác liên tục mà não bộ cần để xử lý tình huống. Nhóm thứ hai là việc vi phạm các quy tắc an toàn thụ động, phổ biến nhất là không thắt dây an toàn. Mặc dù không trực tiếp gây ra tai nạn, hành vi này tước đi lớp bảo vệ cơ học quan trọng nhất, làm khuếch đại mức độ nghiêm trọng của chấn thương khi sự cố vật lý xảy ra. Những tác nhân này cộng hưởng lại làm tăng đáng kể độ trễ phản xạ sinh học, tước đi "thời gian vàng" tính bằng mili-giây để tài xế có thể phản ứng khẩn cấp hay đánh lái tránh chướng ngại vật.

Trước thực trạng cấp bách đó, việc nghiên cứu và triển khai hệ thống giám sát hành vi tài xế (Driver Monitoring System - DMS) không còn là một tính năng phụ trợ mà đã trở thành một yêu cầu công nghệ thiết yếu. Khác biệt cốt lõi của hệ thống DMS hiện đại nằm ở khả năng theo dõi trạng thái sinh trắc học và hành vi của người lái theo thời gian thực (real-time). Ngay tại khoảnh khắc thuật toán phát hiện chuỗi hành vi sai lệch (như đưa điện thoại lên tai hoặc không nhận diện được dải dây an toàn), hệ thống sẽ lập tức kích hoạt chuỗi cảnh báo bằng âm thanh và hình ảnh. Sự can thiệp tức thời này đóng vai trò như một cơ chế đánh thức sự tập trung, giúp tài xế

tái thiết lập quyền kiểm soát phương tiện trước khi vượt qua điểm tới hạn của vụ va chạm.

Bên cạnh giá trị nhân văn cốt lõi là bảo vệ tính mạng con người, việc ứng dụng DMS còn mang lại lợi ích kinh tế chiến lược cho các doanh nghiệp vận tải hành khách và logistics. Nó đánh dấu bước chuyển mình từ mô hình quản lý rủi ro "thụ động" (xử lý hậu quả sau tai nạn) sang mô hình quản lý "chủ động" (phòng ngừa từ gốc rễ). Thông qua dữ liệu vi phạm được số hóa và đẩy về trung tâm điều hành, các nhà quản lý có thể đánh giá minh bạch ý thức tuân thủ của từng cá nhân, từ đó tối ưu hóa quy trình đào tạo, giảm thiểu chi phí bồi thường bảo hiểm và xây dựng một hệ sinh thái giao thông thông minh, chuyên nghiệp.

1.1.2. Hạn chế của các phương pháp truyền thống

Trước khi các công nghệ Trí tuệ nhân tạo (AI) học sâu được ứng dụng rộng rãi, các hệ thống giám sát trên phương tiện thương mại chủ yếu dựa vào camera hành trình truyền thống (dashcam) và một số cảm biến cơ học nội bộ. Tuy nhiên, rào cản kỹ thuật lớn nhất của các phương pháp này nằm ở cơ chế vận hành mang tính thụ động cao. Camera hành trình về bản chất chỉ đóng vai trò như một thiết bị lưu trữ cục bộ (một dạng "hộp đen" quang học), ghi lại chuỗi sự kiện một cách máy móc. Việc phát hiện vi phạm thường phải thực hiện thủ công thông qua quy trình trích xuất và phân tích hậu kiểm (post-incident analysis) bởi người quản lý đội xe sau khi sự cố đã xảy ra. Sự trễ pha này khiến hệ thống hầu như không có khả năng trong việc can thiệp, nhắc nhở hoặc ngăn chặn rủi ro ngay tại thời điểm tài xế đang thực hiện hành vi nguy hiểm.

Nhằm khắc phục tính bị động, một thế hệ hệ thống giám sát trung gian đã ra đời bằng cách tích hợp các thuật toán Thị giác máy tính (Computer Vision) cơ bản để giải quyết bài toán phát hiện đối tượng (Object Detection). Mặc dù có khả năng nhận diện các vật thể như điện thoại hay điều thuốc trong khung hình, điểm yếu của cách tiếp cận này là sự thiếu hụt khả năng nhận thức ngữ cảnh (contextual awareness). Nếu chỉ dừng lại ở việc xác định sự hiện diện của vật thể, hệ thống sẽ rơi vào trạng thái "mù không gian", dẫn đến tỷ lệ cảnh báo sai (False Positive) tăng vọt. Một ví dụ điển hình là hệ thống có thể nhận diện một chiếc điện thoại thông minh trong khung hình, nhưng thiếu cơ sở hình học để phân định đó là thiết bị do tài xế đang thao tác hay chỉ là điện thoại của hành khách ngồi ở ghế phụ.

Trong môi trường vận hành thực tế, tần suất xuất hiện các báo động giả (false alarms) liên tục không chỉ gây phiền toái, làm phân tâm người lái mà còn tạo ra hội chứng "mệt mỏi vì cảnh báo" (alert fatigue). Hậu quả tất yếu là sự xói mòn lòng tin của tài xế vào năng lực của công nghệ, dẫn đến việc họ có xu hướng phớt lờ, hoặc thậm chí tìm cách vô hiệu hóa hệ thống giám sát. Chính những giới hạn khắt khe về độ trễ thời gian và năng lực thấu hiểu ngữ cảnh này đã đặt ra yêu cầu cấp thiết về việc nghiên cứu một kiến trúc phần mềm thế hệ mới. Trong đó, hệ thống không chỉ dừng lại ở mức độ "nhìn thấy" vật thể rời rạc, mà phải có khả năng thiết lập các mối liên hệ không gian để thực sự "hiểu" được hành vi, đồng thời phải đưa ra được các phản hồi theo thời gian thực.

1.1.3. Thách thức trong triển khai thực tế

Để khắc phục những khiếm khuyết của các phương pháp truyền thống, các giải pháp DMS hiện đại đã tiên phong ứng dụng công nghệ Học sâu (Deep Learning) nhằm trực tiếp phân tích hình ảnh không gian buồng lái. Tuy nhiên, việc chuyển giao các mô hình AI từ môi trường phòng thí nghiệm (nơi dữ liệu được kiểm soát lý tưởng) ra môi trường vận hành thực tế trên phương tiện giao thông phải đối mặt với hàng loạt thách thức kỹ thuật.

Thách thức đầu tiên và thường trực nhất đến từ sự biến thiên phức tạp của môi trường ánh sáng. Cabin xe là một không gian quang học có dải nhạy sáng (dynamic range) biến đổi liên tục, phụ thuộc nhiều vào thời gian trong ngày, điều kiện thời tiết, hoặc các tình huống thay đổi độ sáng đột ngột khi phương tiện di chuyển qua hầm chui. Các hiện tượng lóa sáng (glare) do ánh nắng mặt trời chiếu trực tiếp, ngược sáng (backlight) từ đèn pha xe đi ngược chiều, hay tình trạng thiếu sáng nghiêm trọng vào ban đêm (buộc hệ thống phải kích hoạt chế độ camera hồng ngoại) rất dễ làm suy giảm hoặc phá hủy các đặc trưng quang học của vật thể. Điều này đòi hỏi các kiến trúc mạng nơ-ron không chỉ phải được huấn luyện trên các tập dữ liệu vô cùng đa dạng, mà còn phải sở hữu năng lực biểu diễn đủ sâu để chống chịu được nhiễu thị giác cực đoan.

Khó khăn thứ hai xuất phát từ đặc tính động học của môi trường xe và bài toán che khuất cục bộ (partial occlusion). Sự rung lắc liên tục của phương tiện khi di chuyển trên các điều kiện địa hình khác nhau thường xuyên gây ra hiện tượng nhòe chuyển động (motion blur) trên khung hình, làm mờ đi các chi tiết quan trọng như đầu lọc thuốc hay viên điện thoại di động. Thêm vào đó, đặc thù không gian hẹp

của vị trí ghế lái khiến một phần cơ thể tài xế thường xuyên bị lấp khuất bởi vô lăng, cánh tay hoặc các thiết bị nội thất. Trong tình trạng dữ liệu hình ảnh bị khuyết thiếu (missing data) như vậy, việc chỉ sử dụng duy nhất một mô hình nhận diện vật thể (Object Detection) là kém hiệu quả. Hệ thống cần phải tích hợp thêm các thuật toán đánh giá đa tầng, kết hợp việc phát hiện vật thể với thuật toán ước lượng cấu trúc giải phẫu cơ thể (Pose Estimation) nhằm nội suy ra hành vi vi phạm thông qua các mối liên kết không gian hình học.

Thách thức thứ ba liên quan đến giới hạn tài nguyên tính toán trên thiết bị đầu cuối (Edge Devices). Khác với các máy chủ đám mây có năng lực xử lý lớn và khả năng mở rộng linh hoạt, phần cứng triển khai trực tiếp trên phương tiện thường bị ràng buộc bởi nhiều yếu tố như kích thước nhỏ gọn, khả năng chống rung, khả năng chịu nhiệt và mức tiêu thụ điện năng. Trong khi đó, các mô hình học sâu thường có số lượng tham số lớn, yêu cầu tài nguyên tính toán và băng thông bộ nhớ đáng kể. Do đó, việc triển khai các mô hình AI trên bo mạch nhúng như NVIDIA Jetson không chỉ phụ thuộc vào độ chính xác của mô hình, mà còn phụ thuộc vào khả năng tối ưu hóa để duy trì tốc độ xử lý khung hình phù hợp với yêu cầu cảnh báo thời gian thực. Bài toán này đòi hỏi sự kết hợp giữa lựa chọn kiến trúc mô hình, giảm độ phức tạp tính toán, tối ưu kích thước đầu vào và tận dụng các công cụ gia tốc phần cứng như GPU hoặc TensorRT.

1.2. Edge AI Computing

1.2.1. Khái niệm Edge AI

Edge AI (Trí tuệ nhân tạo tại biên) không chỉ là một thuật ngữ hẹp mà đại diện cho một sự chuyển dịch mô hình (paradigm shift) thể hiện một sự thay đổi đáng chú ý trong kiến trúc hệ thống phân tán. Trong mô hình điện toán đám mây truyền thống, thiết bị đầu cuối chỉ làm nhiệm vụ thu thập và đẩy dữ liệu lên máy chủ. Ngược lại, Edge AI là triết lý thiết kế trong đó năng lực tính toán và các quá trình suy luận mạng nơ-ron (Deep Learning Inference) được đẩy từ các trung tâm dữ liệu (Cloud Data Centers) xuống thẳng các thiết bị phần cứng cục bộ nằm ở rìa mạng (Network Edge) – tức là ngay tại tọa độ vật lý nơi dữ liệu thô được sinh ra. Thay vì vận chuyển một khối lượng lớn dữ liệu hình ảnh đến nơi có năng lực tính toán, Edge AI thay đổi cách tổ chức xử lý dữ liệu bằng cách mang thuật toán đến gần nhất với nguồn dữ liệu. Sự thay đổi kiến trúc này biến các thiết bị ngoại vi từ những "cảm biến thụ động" (passive sensors) chỉ biết thu và phát luồng video, trở thành các "nút tính toán thông minh"

(smart computational nodes) có khả năng tự xử lý, phân tích và đưa ra quyết định tức thì.

Hạ tầng vật lý để hiện thực hóa triết lý Edge AI chính là các thiết bị biên (Edge Devices). Trong bối cảnh hệ thống giám sát phương tiện giao thông, các thiết bị này thường là các hệ thống máy tính nhúng dạng System-on-Chip (SoC) chuyên dụng cho AI, tiêu biểu như dòng NVIDIA Jetson. Đặc điểm chung của các nền tảng này là chúng được tích hợp sẵn các lõi xử lý tăng tốc phần cứng chuyên biệt như GPU nhúng, NPU (Neural Processing Unit) hoặc TPU (Tensor Processing Unit), cung cấp hiệu năng tính toán ma trận phù hợp cho các tác vụ AI tại biên, đồng thời vẫn duy trì định mức tiêu thụ điện năng thấp hơn và kích thước vật lý nhỏ gọn.

Bên cạnh sự tiến hóa vượt bậc về phần cứng vi mạch, sự trưởng thành của khái niệm Edge AI còn được thúc đẩy trực tiếp bởi các kỹ thuật tối ưu hóa phần mềm chuyên sâu. Bằng việc ứng dụng các thuật toán nén mô hình (như lượng tử hóa, cắt tỉa mạng) và các trình biên dịch tăng tốc (như TensorRT), các kiến trúc mạng nơ-ron phức tạp như YOLO (phát hiện đối tượng) hay MediaPipe (ước lượng tư thế) giờ đây đã giảm đáng kể rào cản về tài nguyên. Chúng có thể được biên dịch và thực thi hiệu quả ngay trên bộ nhớ hữu hạn của máy tính nhúng. Điều này cho phép hệ thống phân tích trực tiếp hàng chục khung hình mỗi giây ngay trong không gian cabin xe khép kín, đáp ứng các yêu cầu về khả năng suy luận thời gian thực (real-time analytics) của các bài toán nhận diện hành vi phức tạp.

1.2.2. So sánh Edge AI và Cloud AI

Để nhận diện rõ giá trị chiến lược của Edge AI, cần đặt nó lên bàn cân đối chiếu với kiến trúc điện toán đám mây truyền thống (Cloud AI) trong bối cảnh vận hành khắc nghiệt của môi trường giao thông.

Trong mô hình Cloud AI thuần túy, quy trình xử lý mang tính tập trung cao độ. Camera lắp đặt trên phương tiện chỉ đóng vai trò như một thiết bị ngoại vi thu nhận tín hiệu, thực hiện đóng gói và truyền phát luồng dữ liệu video liên tục qua hạ tầng mạng viễn thông (4G/5G) về máy chủ trung tâm. Tại đây, các cụm máy chủ hiệu năng cao sẽ bóc tách khung hình, chạy các mô hình học sâu để phát hiện vi phạm, và cuối cùng gửi ngược lệnh điều khiển hoặc cảnh báo về lại xe. Tuy nhiên, cách tiếp cận này vấp phải những "nút thắt cổ chai" về mặt viễn thông. Điểm yếu của nó là sự phụ thuộc lớn vào băng thông và độ ổn định của kết nối mạng. Việc truyền tải video độ phân giải cao liên tục 24/7 từ hàng nghìn phương tiện sẽ tiêu tốn một lượng lớn băng thông,

kéo theo làm tăng chi phí hạ tầng mạng. Ngoài ra, quá trình truyền phát hai chiều cũng tạo ra độ trễ khứ hồi (round-trip latency) đáng kể. Trong các tình huống khẩn cấp, chẳng hạn như khi tài xế ngủ gật hoặc rời mắt khỏi đường đi, sự chậm trễ chỉ tính bằng giây từ lúc dữ liệu rời xe đến khi nhận được lệnh cảnh báo có thể dẫn đến tai nạn. Thêm vào đó, khi phương tiện di chuyển qua các "vùng lùm" sóng viễn thông như đèo dốc, hầm chui hay khu vực ngoại ô hẻo lánh, hệ thống giám sát Cloud AI sẽ bị vô hiệu hóa.

Ngược lại, mô hình Edge AI mang đến một phương thức giải quyết bài toán khác biệt bằng triết lý phân tán. Thiết bị máy tính nhúng được lắp đặt trực tiếp trên xe sẽ thu nhận luồng video từ camera thông qua các giao thức truyền dẫn nội bộ với tốc độ cao. Dữ liệu hình ảnh được đưa thẳng vào bộ nhớ của thiết bị và được phân tích bởi các mô hình học sâu đã được biên dịch sẵn. Ưu điểm của cơ chế này là việc giảm đáng kể độ trễ truyền tải mạng, hỗ trợ phản hồi gần thời gian thực, cho phép hệ thống đưa ra quyết định và kích hoạt cảnh báo ngay tại hiện trường. Đồng thời, việc phân tích dữ liệu tại chỗ giúp hệ thống đạt được tính khả dụng (high availability), có khả năng hoạt động độc lập và bảo vệ tài xế liên tục ngay cả khi phương tiện bị ngắt kết nối Internet trong một khoảng thời gian nhất định.

Bên cạnh đó, Edge AI còn giải quyết bài toán chi phí viễn thông. Thay vì phải tải lên toàn bộ luồng video thô nặng nề, bộ xử lý tại biên chỉ cần trích xuất và gửi về trung tâm điều hành các siêu dữ liệu (metadata) nhẹ, bao gồm: chuỗi văn bản cảnh báo, thời gian, tọa độ GPS hiện tại, hoặc một vài hình ảnh/đoạn clip ngắn ghi lại để làm minh chứng. Cơ chế lọc dữ liệu tại nguồn này giúp tiết kiệm băng thông, biến việc triển khai hệ thống giám sát thời gian thực trên quy mô hàng nghìn phương tiện trở thành một bài toán khả thi về mặt kinh tế.

Bảng 1.1. So sánh Edge AI và Cloud AI

Tiêu chí	Edge AI	Cloud AI
Độ trễ (Latency)	< 50ms (xử lý tại chỗ)	200-500ms+ (phụ thuộc mạng)
Băng thông cần thiết	Thấp (chỉ gửi metadata)	Cao (truyền video thô)
Bảo mật dữ liệu	Dữ liệu tại thiết bị (riêng tư)	Dữ liệu trên server (rủi ro)
Hoạt động ngoại tuyến	Có (không cần mạng)	Không (phụ thuộc kết nối)
Chi phí viễn thông	Thấp	Cao (video streaming 24/7)

Năng lực tính toán	Giới hạn (phần cứng nhúng)	Lớn và có thể mở rộng tùy theo cấu hình hạ tầng
Khả năng mở rộng	Thêm thiết bị = thêm chi phí	Elastic scaling (thuê cloud)
Cập nhật mô hình	Cần triển khai từng thiết bị	Cập nhật tập trung 1 lần

1.2.3. Ưu điểm của Edge AI trong bài toán DMS

Việc định hướng ứng dụng triết lý Edge AI vào bài toán giám sát hành vi tài xế không phải là một sự lựa chọn ngẫu nhiên, mà xuất phát từ khả năng giải quyết trọn vẹn những điểm nghẽn kỹ thuật và vận hành mà điện toán đám mây để lại. Kiến trúc này mang đến bốn giá trị cốt lõi, tạo nên tính thực tiễn cao cho hệ thống khi triển khai trên diện rộng.

Giá trị tiên quyết và quan trọng nhất là khả năng cung cấp độ trễ thấp. Trong môi trường giao thông di chuyển với tốc độ cao, ranh giới giữa an toàn và tai nạn thường chỉ được quyết định trong vài phần mười giây. Bằng cách giảm sự phụ thuộc vào quá trình truyền dữ liệu qua mạng internet, Edge AI thiết lập một vòng lặp phản hồi khép kín (closed feedback loop) ngay bên trong không gian buồng lái. Chu trình từ thời điểm ống kính camera ghi nhận một sai lệch hành vi (như mắt nhắm lại hoặc tay cầm điện thoại) cho đến khi thiết bị phần cứng phát ra âm thanh cảnh báo có thể diễn ra trong thời gian tính bằng mili-giây. Khả năng phản ứng tức thời này đóng vai trò quan trọng, giúp tài xế kịp thời điều chỉnh lại hành vi trước khi phương tiện vượt khỏi tầm kiểm soát.

Thứ hai, Edge AI mang lại tính khả dụng và khả năng hoạt động ngoại tuyến (Offline Mode). Hạ tầng viễn thông trên thực tế không phải lúc nào cũng phủ sóng đồng đều, đặc biệt khi phương tiện di chuyển qua các địa hình phức tạp như đường đèo núi, hầm chui hoặc khu vực ngoại ô hẻo lánh. Với kiến trúc phân tán, mỗi thiết bị máy tính nhúng trên xe trở thành một nút tính toán tự trị (autonomous node). Ngay cả khi kết nối 4G/5G bị gián đoạn, thiết bị vẫn hoạt động, hệ thống cảnh báo vẫn duy trì chức năng cục bộ. Dữ liệu về sự kiện xảy ra trong khoảng thời gian mất kết nối sẽ được đưa vào bộ nhớ đệm cục bộ (local cache) và tự động đồng bộ hóa lên máy chủ đám mây theo cơ chế bất đồng bộ (asynchronous sync) ngay khi kết nối mạng được khôi phục.

Thứ ba, kiến trúc tại biên còn hỗ trợ bài toán bảo vệ quyền riêng tư dữ liệu (Data Privacy). Một rào cản lớn khi áp dụng DMS là tâm lý phản kháng của người lao động

do cảm giác bị giám sát hình ảnh liên tục. Với cách tiếp cận xử lý tại nguồn, luồng video thô có thể được phân tích trực tiếp trên thiết bị Edge và không cần truyền toàn bộ về máy chủ trung tâm. Trong hệ thống đề tài, Cloud chủ yếu tiếp nhận metadata như loại vi phạm, thời gian, thông tin thiết bị và một số hình ảnh hoặc đoạn video ngắn dùng làm bằng chứng. Cơ chế này vừa đáp ứng yêu cầu quản lý, vừa góp phần giảm nguy cơ rò rỉ dữ liệu hình ảnh cá nhân so với mô hình truyền tải video liên tục lên Cloud.

Cuối cùng, từ góc nhìn quản trị doanh nghiệp, Edge AI là chìa khóa để tối ưu hóa chi phí vận hành ở quy mô lớn. Nếu một doanh nghiệp logistics sở hữu một đội xe gồm hàng nghìn chiếc, việc truyền phát video liên tục theo mô hình Cloud sẽ tiêu tốn một khoản ngân sách lớn cho việc mua gói dung lượng viễn thông hàng tháng, đồng thời đòi hỏi đầu tư hệ thống máy chủ lưu trữ đắt đỏ. Bằng cách thực hiện cơ chế lọc tín hiệu thông minh ngay tại biên, hệ thống giảm đáng kể khối lượng dữ liệu truyền tải qua mạng. Sự tối ưu này biến DMS từ một dự án có chi phí duy trì đắt đỏ trở thành một giải pháp công nghệ mang lại hiệu quả kinh tế cao và dễ dàng mở rộng (scalability) cho các doanh nghiệp vận tải.

1.3. Kiến trúc Hybrid (Edge–Cloud)

1.3.1. Triết lý mô hình Hybrid

Mặc dù việc dịch chuyển năng lực tính toán xuống thiết bị biên (Edge AI) mang lại lợi thế về độ trễ và khả năng phản hồi thời gian thực, một kiến trúc thuần Edge (Pure Edge) vẫn bộc lộ những giới hạn khi áp dụng vào bài toán quản trị quy mô doanh nghiệp. Các thiết bị nhúng trên phương tiện bị giới hạn về năng lực lưu trữ tại chỗ, không thể duy trì cơ sở dữ liệu lịch sử trong thời gian dài. Hơn thế nữa, nếu thiếu đi một hệ thống trung tâm, mỗi thiết bị trên một chiếc xe sẽ trở thành một "ốc đảo thông tin" cô lập, khiến nhà quản lý không thể có cái nhìn toàn cảnh về tình trạng an toàn của toàn bộ đội xe. Do đó, để đáp ứng các yêu cầu của một hệ thống DMS cấp doanh nghiệp (Enterprise-grade DMS), kiến trúc lai Hybrid Edge-Cloud là một hướng tiếp cận phù hợp để cân bằng giữa xử lý thời gian thực tại Edge và quản trị tập trung trên Cloud.

Triết lý cốt lõi của kiến trúc Hybrid này được định hình dựa trên nguyên tắc phân chia khối lượng công việc (workload distribution) theo mô hình phân tán bất đối xứng, được tóm gọn qua định đề: "Xử lý chiến thuật tại Biên – Quản lý chiến lược trên Đám

mây". Sự phân cực này giúp khai thác điểm mạnh của từng hạ tầng, đồng thời giảm các điểm yếu chéo của chúng.

Tại phân hệ Biên (Edge Node), các thiết bị máy tính nhúng sẽ đóng vai trò giải quyết các tác vụ phản ứng nhanh, yêu cầu độ trễ thấp và ảnh hưởng trực tiếp đến an toàn lái xe. Nhiệm vụ của Edge là tiếp nhận luồng video thô (raw video stream) trực tiếp từ camera, thực thi các thuật toán trích xuất đặc trưng nặng nề (YOLO, MediaPipe Pose), tính toán luật hình học và đưa ra phản ứng tức thời để cảnh báo tài xế. Phân hệ này hoạt động như một màng lọc loại bỏ dữ liệu video không cần xử lý (khi tài xế lái xe an toàn) và chỉ đóng gói, truyền tải lên mạng những sự kiện (events) vi phạm đã được xác thực, dưới dạng siêu dữ liệu (metadata) nhẹ nhàng kèm theo một đoạn clip bằng chứng ngắn.

Ngược lại, phân hệ Đám mây (Cloud Node) được giảm đáng kể gánh nặng xử lý video thô thời gian thực, qua đó có thể tập trung tài nguyên để thực thi các nhiệm vụ quản trị và phân tích ở cấp hệ thống như lưu trữ tập trung, phân tích dữ liệu, hậu kiểm vi phạm và hỗ trợ ra quyết định quản trị đội xe. Cloud đóng vai trò là bộ não trung tâm, nơi tập hợp và đồng bộ hóa cơ sở dữ liệu từ hàng nghìn thiết bị Edge truyền về. Tại đây, hệ thống có thể cung cấp không gian lưu trữ tập trung để quản lý hồ sơ và bằng chứng vi phạm theo chính sách dữ liệu. Hơn thế nữa, sức mạnh tính toán của Cloud cho phép chạy các mô hình phân tích video chuyên sâu để hậu kiểm các clip nghi vấn, hoặc áp dụng các thuật toán khai phá dữ liệu (Data Mining) nhằm đánh giá, chấm điểm rủi ro (Driver Risk Scoring) cho từng cá nhân. Từ nền tảng dữ liệu này, Cloud cung cấp hệ thống RESTful API (xây dựng bằng FastAPI) để phục vụ dữ liệu cho ứng dụng giám sát (Admin Dashboard), hệ thống cho phép nhà quản lý truy vấn, lọc và xác thực các cảnh báo vi phạm theo thời gian thực.

Tóm lại, sự kết hợp giữa Edge và Cloud không phải là một phép cộng cơ học, mà là một sự bổ trợ phù hợp về mặt kiến trúc. Edge mang lại tốc độ và sự bảo vệ tức thời, trong khi Cloud mang lại tính liên kết, không gian lưu trữ lớn và năng lực quản trị vĩ mô. Thiết kế Hybrid này hỗ trợ khả năng mở rộng (scalability) cho hệ thống, do phần lớn tác vụ suy luận nặng được xử lý tại Edge, giúp giảm áp lực xử lý trực tiếp lên máy chủ trung tâm khi số lượng phương tiện tăng.

1.3.2. Vai trò và luồng xử lý tại Edge Node

Được định vị tại các phương tiện giao thông, phân hệ Edge chịu trách nhiệm thực thi toàn bộ chu trình giám sát khép kín ngay tại hiện trường. Thay vì vận hành theo

các tác vụ rời rạc, thiết bị nhúng được thiết kế để chạy một luồng xử lý dữ liệu (data pipeline) liên tục và liền mạch, bao gồm bốn tầng chức năng cốt lõi hoạt động nối tiếp nhau với tốc độ thời gian thực.

Khởi đầu chu trình là Tầng thu nhận và tiền xử lý tín hiệu (Data Acquisition & Preprocessing). Luồng video thô từ camera giám sát cabin được hệ thống tiếp nhận qua OpenCV VideoCapture (hỗ trợ webcam, RTSP hoặc tệp video). Tuy nhiên, dữ liệu ảnh gốc thường chứa nhiều thông tin dư thừa và có kích thước không đồng nhất. Tại đây, hệ thống thực hiện các thuật toán chuẩn hóa không gian (spatial normalization), bao gồm việc cắt xén (cropping), thay đổi kích thước (resizing) về một định dạng tiêu chuẩn (ví dụ: 640 x 640 pixels), chuyển đổi không gian màu và ma trận hóa thành các tensor dữ liệu để sẵn sàng làm đầu vào cho mạng nơ-ron. [2]

Dữ liệu sau khi được chuẩn hóa sẽ được chuyển tới Tầng nhận thức học sâu (Deep Learning Perception Layer). Sự khác biệt của hệ thống nằm ở việc vận hành song song và đa nhiệm các mô hình AI đã được tối ưu hóa. Ở luồng thứ nhất, mô hình nhận diện đối tượng đa lớp (YOLO) thực hiện quét toàn bộ khung hình để trích xuất tọa độ giới hạn (bounding boxes) và phân loại các vật thể mục tiêu như điện thoại, điều thuốc hay dải dây an toàn. Ở luồng thứ hai, mô hình ước lượng tư thế (MediaPipe Pose) tiến hành tái tạo cấu trúc giải phẫu của người lái, xác định chính xác tọa độ các điểm mốc (landmarks) như gốc vai, cổ tay, mũi và tai. Bằng cách thiết lập hệ tọa độ sinh trắc học này, hệ thống nội suy thành công ranh giới của các vùng quan tâm đặc biệt, bao gồm vùng thao tác của người lái (Driver ROI) và khu vực lồng ngực (Chest ROI).

Tuy nhiên, học sâu là chỉ cung cấp dữ liệu định vị rời rạc chứ chưa hiểu được hành vi. Do đó, toàn bộ kết quả thô từ tầng nhận thức sẽ được chuyển giao cho Động cơ luật suy luận (Behavior Rules Engine) - tầng xử lý mang tính quyết định của kiến trúc Edge. Tại đây, sự mơ hồ của dữ liệu đầu ra AI được giải quyết bằng một tập hợp các ràng buộc hình học và logic không gian có tính tất định (deterministic constraints). Cụ thể, thuật toán sẽ tính toán các khoảng cách Euclidean và đánh giá sự giao thoa không gian. Một cảnh báo sử dụng điện thoại chỉ được xác thực khi thỏa mãn đồng thời hai điều kiện: "Tọa độ điện thoại phải nằm gọn bên trong Driver ROI" và "Khoảng cách từ tâm điện thoại đến cổ tay hoặc vùng đầu phải nhỏ hơn một ngưỡng (threshold) định trước". Tương tự, lỗi không thắt dây an toàn chỉ được kết luận khi hệ thống phân tích không thấy sự tồn tại của dải dây đi cắt chéo qua khu vực Chest

ROI. Tầng lọc logic này đóng vai trò như một màng chắn độ tin cậy, giúp giảm thiểu các cảnh báo giả (False Positives) sinh ra do đồ vật của hành khách ghé phụ gây nhiễu.

Kết thúc chu trình là Tầng kích hoạt và giao tiếp dữ liệu (Actuation & Communication Layer). Ngay khi một vi phạm bị Động cơ luật suy luận xác nhận, Edge Node lập tức kích hoạt phần cứng ngoại vi (chuông còi, đèn LED) để đưa ra cảnh báo cho tài xế. Đồng thời, một tiến trình chạy ngầm (background thread) sẽ thực hiện việc đóng gói sự kiện. Khung hình chứa vi phạm được trích xuất, đính kèm với các thẻ siêu dữ liệu (metadata) gọn nhẹ bao gồm nhãn dán vi phạm, chuỗi thời gian (timestamp) và thông tin định danh phương tiện. Gói dữ liệu này sau đó được truyền lên phân hệ Đám mây (Cloud) thông qua giao thức HTTP REST API theo cơ chế bất đồng bộ (asynchronous payload transmission), giúp việc cập nhật dữ liệu diễn ra thông suốt mà không làm gián đoạn hay tạo ra độ trễ cho luồng phân tích video đang chạy liên tục của hệ thống.

1.3.3. Vai trò của Cloud Node

Phân hệ Đám mây (Cloud Node) là trung tâm giải quyết bài toán chiến lược và quản trị tổng thể. Để xử lý khối lượng dữ liệu lớn từ nhiều phương tiện đổ về, kiến trúc phần mềm tại Cloud Node được thiết kế phân lớp một cách chặt chẽ, bao gồm hai tầng chức năng cốt lõi hoạt động tương hỗ lẫn nhau.

Lớp nền tảng đầu tiên là Cổng giao tiếp ngoại vi (API Gateway) và Hệ thống quản trị dữ liệu. Bằng việc ứng dụng các bộ khung lập trình phía máy chủ (Backend Framework) hiện đại và tối ưu hóa hiệu năng cao cho xử lý bất đồng bộ như FastAPI, hệ thống thiết lập các điểm cuối giao tiếp (RESTful API endpoints) với tính sẵn sàng cao. Chức năng chính của tầng này là lắng nghe và tiếp nhận liên tục các gói tin (payloads) từ mạng lưới Edge Node gửi lên. Khi một gói tin được gửi đến, Backend sẽ tiến hành bóc tách, chuẩn hóa các thẻ siêu dữ liệu (metadata như: loại vi phạm, tọa độ GPS, thời gian) và trích xuất các tệp phương tiện (hình ảnh/video), sau đó ghi nhận vào Hệ cơ sở dữ liệu tập trung (Centralized Database). Bên cạnh chức năng tiếp nhận và lưu trữ dữ liệu, Cloud Node còn đảm nhiệm vai trò kích hoạt các luồng xử lý AI hậu kiểm chuyên sâu, điển hình là mô hình SlowFast, nhằm xác minh lại các đoạn video bằng chứng có tính chất phức tạp hoặc dễ gây nhầm lẫn. Việc chạy lại các đoạn video bằng chứng thông qua các mô hình AI trên Cloud giúp tạo ra một lớp xác thực thứ hai, củng cố tính chính danh của các vi phạm phức tạp trước khi lưu trữ.

Lớp thứ hai là Hệ thống giám sát và báo cáo trung tâm (Centralized Admin Dashboard). Nhằm chuyển hóa dữ liệu thô thành tri thức quản trị, Cloud Node cung cấp các API phục vụ cho hệ thống quản trị, cho phép nhà quản lý theo dõi tình trạng cảnh báo, thiết bị và tài xế trên toàn bộ đội xe. Thông qua các API này, dữ liệu vi phạm có thể được truy vấn, lọc theo nhiều tiêu chí như loại vi phạm, thiết bị, trạng thái xử lý và khoảng thời gian ghi nhận. Đồng thời, hệ thống cũng hỗ trợ truy xuất bằng chứng kỹ thuật số, thực hiện xác minh thủ công hoặc kích hoạt cơ chế hậu kiểm bằng mô hình SlowFast đối với các cảnh báo cần đánh giá thêm. Nhờ đó, Cloud Node không chỉ đóng vai trò lưu trữ dữ liệu tập trung mà còn là nền tảng hỗ trợ giám sát, báo cáo và ra quyết định quản trị ở cấp hệ thống.

1.3.4. Lợi thế và giá trị của kiến trúc Hybrid

Việc lựa chọn kiến trúc Hybrid Edge–Cloud trong đề tài xuất phát từ nhu cầu cân bằng giữa hai nhóm yêu cầu: phản hồi nhanh tại phương tiện và quản trị tập trung ở cấp hệ thống. Nếu chỉ sử dụng Cloud AI, hệ thống dễ phụ thuộc vào băng thông mạng và phát sinh độ trễ khi truyền video liên tục. Ngược lại, nếu chỉ sử dụng Edge AI, hệ thống có thể phản hồi nhanh tại chỗ nhưng gặp hạn chế trong lưu trữ, tổng hợp dữ liệu và giám sát toàn bộ đội xe.

Trong kiến trúc Hybrid, phân hệ Edge đảm nhiệm các tác vụ cần phản hồi nhanh như thu nhận khung hình, suy luận mô hình, áp dụng luật hình học và kích hoạt cảnh báo cục bộ. Phân hệ Cloud tập trung vào các chức năng mang tính quản trị như lưu trữ dữ liệu cảnh báo, truy xuất bằng chứng, hỗ trợ hậu kiểm và cung cấp giao diện giám sát. Nhờ cách phân chia này, hệ thống có thể giảm áp lực xử lý video thô trên máy chủ trung tâm, đồng thời vẫn duy trì khả năng theo dõi và quản lý dữ liệu vi phạm ở quy mô lớn.

Bên cạnh đó, kiến trúc Hybrid còn hỗ trợ khả năng mở rộng hệ thống. Khi số lượng phương tiện tăng, phần lớn tác vụ suy luận nặng vẫn được xử lý tại từng thiết bị Edge, trong khi Cloud chủ yếu tiếp nhận các bản ghi sự kiện và bằng chứng cần thiết. Điều này giúp giảm khối lượng dữ liệu truyền tải so với mô hình truyền toàn bộ video lên Cloud, đồng thời tạo nền tảng thuận lợi cho việc quản lý, thống kê và phân tích hành vi tài xế trong các giai đoạn phát triển tiếp theo.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ

2.1. Bài toán phát hiện đối tượng và mô hình YOLO

2.1.1. Tổng quan bài toán Object Detection

Phát hiện đối tượng (Object Detection) là một trong những bài toán nền tảng và có ý nghĩa thực tiễn rất lớn trong lĩnh vực Thị giác máy tính (Computer Vision). Mục tiêu của bài toán này không chỉ dừng lại ở việc xác định trong ảnh có xuất hiện đối tượng thuộc lớp nào, mà còn yêu cầu hệ thống phải định vị chính xác vị trí của từng đối tượng đó trong không gian ảnh. Nói cách khác, Object Detection là sự kết hợp giữa hai bài toán cơ bản: phân loại ảnh (Image Classification) và định vị đối tượng (Object Localization).

Đầu ra của một hệ thống phát hiện đối tượng thường bao gồm ba thành phần chính:

- Nhãn lớp (Class label): Cho biết đối tượng thuộc loại nào, chẳng hạn như điện thoại, thuốc lá, người, hoặc dây an toàn.
- Hộp giới hạn (Bounding box): Thể hiện vị trí và kích thước của đối tượng trong ảnh thông qua các tham số tọa độ không gian.
- Độ tin cậy (Confidence score): Phản ánh mức độ chắc chắn của mô hình đối với dự đoán đó (thường có giá trị từ 0 đến 1).

Trong biểu diễn toán học và lập trình thông dụng, một hộp giới hạn (bounding box) được mô tả bởi bốn giá trị tọa độ là (x, y, w, h) . Trong đó, x và y biểu diễn tọa độ tâm (hoặc tọa độ góc trên cùng bên trái) của hộp giới hạn, còn w và h tương ứng là chiều rộng (width) và chiều cao (height) của vật thể. Các tham số này cho phép hệ thống xác định chính xác vị trí và tỷ lệ của vật thể trong khung hình để phục vụ cho các bước xử lý tiếp theo như tính toán vị trí tương đối, lọc vùng quan tâm và suy luận hành vi.

Xét về bản chất, Object Detection là bài toán phức tạp hơn nhiều so với phân loại ảnh thông thường. Trong bài toán phân loại, mô hình chỉ cần đưa ra một nhãn tổng thể cho toàn bộ bức ảnh. Tuy nhiên, đối với Object Detection, mô hình phải xử lý đồng thời nhiều nhiệm vụ: dự đoán có bao nhiêu đối tượng trong ảnh, xác định nhãn của từng đối tượng, và khoanh vùng chính xác vị trí của chúng. Điều này càng trở nên khó khăn trong các tình huống thực tế có nhiều vật thể chồng lấn (overlapping),

kích thước nhỏ, bị che khuất (occlusion) hoặc xuất hiện trong môi trường ánh sáng không ổn định.

Đối với hệ thống giám sát hành vi tài xế, Object Detection giữ vai trò đặc biệt quan trọng vì đây là lớp xử lý đầu tiên giúp hệ thống “nhìn thấy” các dấu hiệu trực tiếp liên quan đến hành vi vi phạm. Ví dụ, nếu mô hình không phát hiện được chiếc điện thoại trên tay tài xế hoặc không nhận diện được điểm chốt của dây an toàn, toàn bộ chuỗi thuật toán suy luận hành vi phía sau sẽ không còn ý nghĩa. Do đó, bài toán phát hiện đối tượng trong bối cảnh DMS không chỉ đòi hỏi độ chính xác cao mà còn yêu cầu tốc độ xử lý (FPS) đủ nhanh để đáp ứng tiêu chí cảnh báo thời gian thực.

Bên cạnh đó, bài toán phát hiện đối tượng trong cabin xe mang những đặc thù và thách thức riêng. Thứ nhất, các đối tượng cần phát hiện thường có kích thước rất nhỏ, ví dụ như phần đầu lọc của điều thuốc hoặc một phần nhỏ của chiếc điện thoại bị che lấp bởi ngón tay tài xế. Thứ hai, điều kiện quan sát trong xe thay đổi liên tục tùy thuộc vào thời gian trong ngày, cường độ ánh sáng chiếu vào kính, góc đặt camera và sự dịch chuyển tư thế của người lái. Thứ ba, nhiều vật thể trong cabin có hình dáng hình học tương tự nhau, dễ gây nhầm lẫn (False Positive) cho mô hình; chẳng hạn chai nước, ví tiền, sạc dự phòng có thể bị nhận nhầm là điện thoại di động. Những yếu tố này khiến việc thiết kế dữ liệu và lựa chọn kiến trúc mô hình Object Detection phù hợp trở thành vấn đề then chốt của dự án.

Hiện nay, các phương pháp Object Detection sử dụng Deep Learning thường được chia thành hai nhóm chính là mô hình hai giai đoạn (Two-stage Detector) và mô hình một giai đoạn (One-stage Detector). Nhóm Two-stage, tiêu biểu là kiến trúc Faster R-CNN [15], thường cho độ chính xác cao do trải qua hai bước độc lập: trích xuất các vùng đề xuất (Region Proposals) và sau đó phân loại kết hợp tinh chỉnh tọa độ từng vùng. Tuy nhiên, kiến trúc công kênh khiến tốc độ suy luận của nhóm này khá chậm, khó đáp ứng yêu cầu triển khai trên các thiết bị nhúng (Edge Devices). Ngược lại, nhóm One-stage (tiêu biểu như họ mô hình YOLO, SSD [10]) xem bài toán phát hiện đối tượng là một bài toán hồi quy (regression problem), dự đoán trực tiếp tọa độ bounding box và xác suất lớp trên toàn bộ bức ảnh chỉ trong một lần chạy mạng nơ-ron. Nhờ đó, nhóm One-stage đạt được tốc độ cao. Trong bài toán DMS yêu cầu độ trễ thấp để cảnh báo tức thời, ưu tiên tốc độ nhưng vẫn giữ được độ chính xác ổn định là yếu tố tiên quyết, do đó họ mô hình YOLO được đánh giá là sự lựa chọn phù hợp nhất.

Từ góc nhìn kiến trúc hệ thống, Object Detection không chỉ giúp nhận diện sự hiện diện của vật thể, mà còn tạo nền tảng dữ liệu không gian đầu vào cho các tầng xử lý ngữ cảnh cao hơn (như Pose Estimation và Behavior Rules Engine). Trong hệ thống của đề tài này, YOLO đóng vai trò phát hiện các đối tượng mục tiêu, sau đó tọa độ của chúng được kết hợp trực tiếp với thông tin khung xương người lái để đưa ra quyết định cuối cùng. Điều đó cho thấy Object Detection là thành phần lõi trung tâm trong toàn bộ chuỗi phân tích hành vi tài xế.

2.1.2. Kiến trúc mô hình YOLO

YOLO (You Only Look Once)[14] là một họ mô hình phát hiện đối tượng thuộc nhóm mạng một giai đoạn (One-stage Detector), được đánh giá cao trong cộng đồng nghiên cứu thị giác máy tính nhờ khả năng tối ưu hóa sự cân bằng (trade-off) giữa độ chính xác (Accuracy/mAP) và tốc độ suy luận (Inference speed). Khác với các phương pháp hai giai đoạn truyền thống (như Faster R-CNN[15]) phải tách biệt quá trình trích xuất vùng đề xuất (Region Proposal) và quá trình phân loại, mạng YOLO thiết lập lại bài toán phát hiện đối tượng dưới dạng một bài toán hồi quy duy nhất (single regression problem). Toàn bộ quá trình từ ảnh đầu vào đến việc dự đoán tọa độ hộp giới hạn (bounding box) và xác suất phân lớp được thực hiện đồng thời chỉ trong một lần lan truyền tiến (single forward pass) qua mạng nơ-ron.

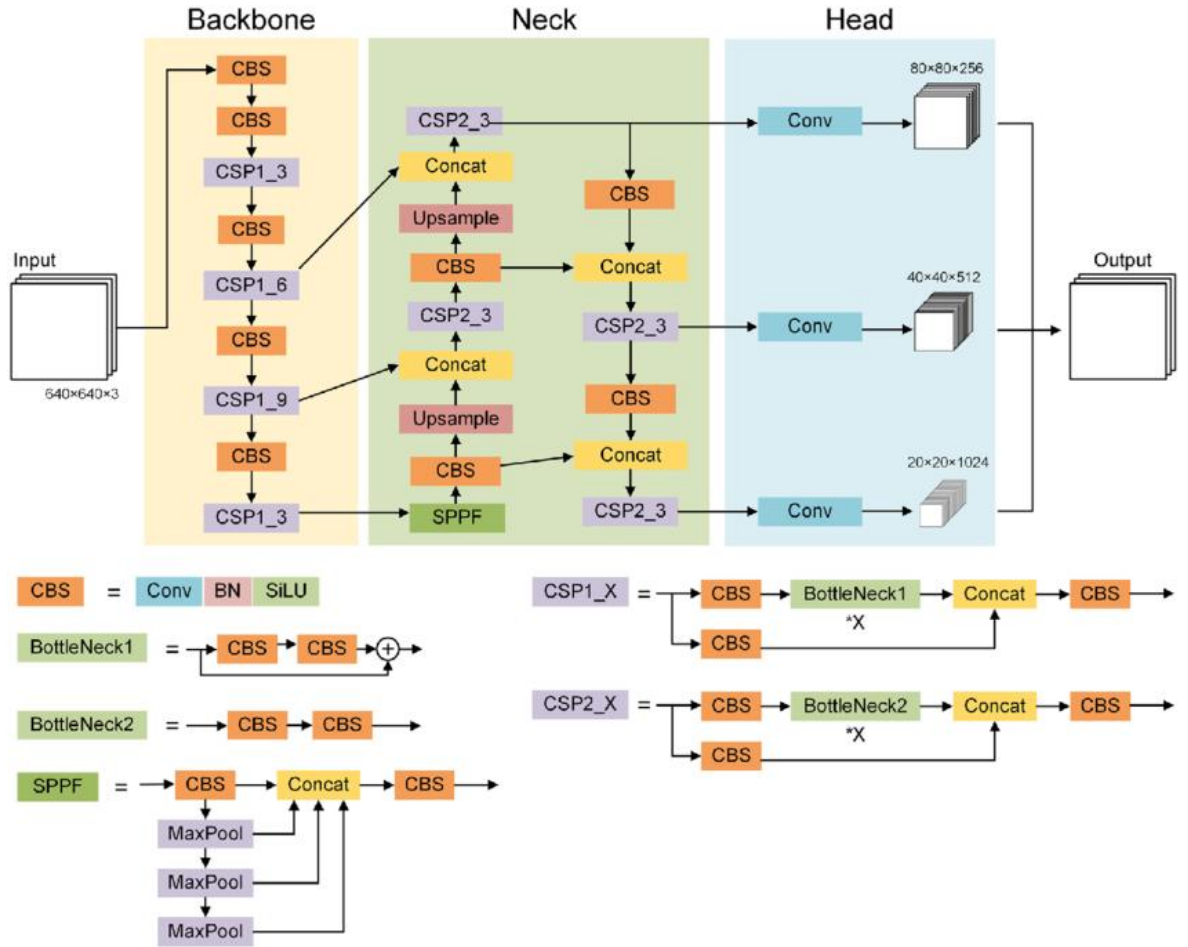
Về mặt nguyên lý, YOLO phân hoạch ảnh đầu vào [14] thành một lưới không gian (grid) có kích thước $S \times S$. Nếu tâm của một đối tượng thực tế (ground truth) rơi vào một ô lưới (grid cell) cụ thể, ô lưới đó sẽ chịu trách nhiệm chính trong việc phát hiện đối tượng này. Tại mỗi ô lưới, mô hình sẽ dự đoán B hộp giới hạn (thường dựa trên các hộp neo - Anchor boxes định nghĩa trước) cùng với điểm số tin cậy (Confidence score). Điểm tin cậy này phản ánh hai yếu tố: xác suất có đối tượng tồn tại trong hộp dự đoán và mức độ chính xác của hộp dự đoán đó so với hộp thực tế, được đo lường bằng chỉ số IoU (Intersection over Union). Chỉ số này được định nghĩa là tỷ lệ giữa diện tích phần giao nhau và diện tích phần hợp của hai hộp giới hạn, được tính theo công thức:

$$IoU = \frac{Area\ of\ Overlap}{Area\ of\ Union}$$

Giá trị IoU càng cao cho thấy hộp dự đoán càng sát với hộp nhãn thực tế. Trong thực tế, các biến thể như GIoU, DIoU và CIoU thường được sử dụng nhằm cải thiện khả năng tối ưu hóa vị trí và kích thước của bounding box.

Đồng thời, mỗi ô lưới cũng dự đoán $P(Class_i | Object)$ là xác suất có điều kiện của các lớp đối tượng (Conditional Class Probabilities).

Các phiên bản YOLO hiện đại đều được thiết kế dựa trên một kiến trúc module hóa, bao gồm ba thành phần chính: mạng trích xuất đặc trưng (Backbone), mạng kết hợp đặc trưng (Neck) và mạng dự đoán (Head). Mạng xương sống thường là một mạng nơ-ron tích chập sâu (CNN) làm nhiệm vụ trích xuất đặc trưng hình ảnh. Các mạng Backbone hiện đại (như CSPDarknet) sử dụng các khối tích chập kết hợp cơ chế tàn dư (Residual connections) để học được các đặc trưng từ mức độ thấp (cạnh, góc, kết cấu) đến các đặc trưng ngữ nghĩa phức tạp ở mức độ cao, đồng thời hạn chế hiện tượng suy giảm đạo hàm (vanishing gradient) khi mạng quá sâu [6]. Tiếp theo, Mạng kết hợp đặc trưng có nhiệm vụ kết hợp và tổng hợp các bản đồ đặc trưng [9] từ nhiều tầng không gian khác nhau của Backbone. Cơ chế này đóng vai trò quan trọng trong việc bảo toàn đặc trưng đa tỉ lệ (multi-scale), giúp mô hình nhận diện tốt cả những vật thể có kích thước nhỏ (như đầu lọc thuốc) lẫn những đối tượng có kích thước lớn hơn (như vùng ngực chứa dây an toàn). Cuối cùng, Mạng dự đoán là nơi thực hiện quá trình ánh xạ để xuất ra các thông tin đầu ra gồm tọa độ (x, y, w, h) , điểm tin cậy và xác suất phân loại lớp cho mọi đối tượng trong khung hình.



Hình 2.1.2. Kiến trúc mô hình YOLO gồm Backbone, Neck và Head

Trong quá trình huấn luyện, mô hình YOLO được tối ưu hóa thông qua một hàm mất mát đa nhiệm (Multi-task Loss Function), là sự tổng hợp của ba thành phần chính:

- Thành phần thứ nhất là mất mát định vị (L_{box}), dùng để đánh giá sai số giữa tọa độ hộp dự đoán và hộp nhãn thực tế, thường sử dụng các biến thể của IoU (như GIoU, CIoU) để tối ưu hóa độ lệch tâm, kích thước và tỷ lệ khung hình.
- Thành phần thứ hai là mất mát phân loại (L_{cls}), đánh giá sai số trong việc phân loại đối tượng bằng hàm Cross-Entropy.
- Thành phần thứ ba là mất mát độ tin cậy (L_{obj}), nhằm phạt mô hình khi dự đoán sai sự tồn tại của đối tượng ở các vùng nền trống hoặc không tự tin ở những vùng có đối tượng thật.

Do thiết kế cho phép nhiều ô lưới gần nhau cùng đưa ra dự đoán về một vật thể duy nhất, đầu ra của mô hình thường chứa nhiều hộp giới hạn chồng lấp. Để giải quyết vấn đề này, thuật toán Non-Maximum Suppression (NMS) được áp dụng ở

bước hậu xử lý. Thuật toán sẽ giữ lại hộp giới hạn có độ tin cậy cao nhất và loại bỏ các hộp xung quanh có chỉ số IoU vượt quá một ngưỡng (threshold) nhất định, đảm bảo mỗi đối tượng trên thực tế chỉ tương ứng với một bounding box duy nhất.

Trong phạm vi đề tài này, việc lựa chọn kiến trúc YOLO (ưu tiên các phiên bản có quy mô tham số từ Nano đến Medium) là một thiết kế có chủ đích giúp giải quyết các mâu thuẫn: mô hình phải đủ độ sâu để phát hiện các vật thể khó (nhỏ, bị che khuất một phần trong môi trường ánh sáng cabin phức tạp) nhưng cũng phải đủ nhẹ để triển khai khả thi trên nền tảng tính toán Edge AI (NVIDIA Jetson) với tốc độ khung hình (FPS) đáp ứng tiêu chuẩn thời gian thực. Thông qua kỹ thuật Transfer Learning (Học chuyển giao), mô hình YOLO được huấn luyện vi chỉnh (fine-tuning) trên tập dữ liệu chuyên biệt của đề tài để nhận diện 4 lớp đối tượng mục tiêu: *phone*, *smoking*, *seatbelt*, *no-seatbelt*. Tuy nhiên, do đặc thù của mô hình One-stage ưu tiên tốc độ, hiện tượng cảnh báo sai (False Positive) là không thể tránh khỏi hoàn toàn. Chính vì vậy, đầu ra không gian của YOLO không được sử dụng như một quyết định cuối cùng mà đóng vai trò là tầng dữ liệu sơ cấp, được kết hợp song song với hệ thống ước lượng tư thế MediaPipe Pose để thiết lập các ràng buộc hình học, từ đó nội suy ra hành vi vi phạm một cách chính xác nhất.

2.1.3. Ứng dụng YOLO trong bài toán DMS

Trong kiến trúc của hệ thống giám sát hành vi tài xế được đề xuất, YOLO không được sử dụng như một bộ phân loại hành vi độc lập (Behavior Classifier), mà đóng vai trò là tầng trích xuất đặc trưng không gian sơ cấp (Primary Spatial Feature Extractor). Nhiệm vụ cốt lõi của mô hình là định vị nhanh và chính xác vị trí của các đối tượng hoặc trạng thái có liên quan trực tiếp đến các hành vi vi phạm an toàn giao thông. Cụ thể, mô hình được huấn luyện để nhận diện các lớp mục tiêu chính bao gồm điện thoại (phone), dấu hiệu hút thuốc (smoking) và trạng thái dây an toàn (seatbelt / no-seatbelt).

Đóng góp quan trọng của YOLO không nằm ở việc đưa ra kết luận cuối cùng, mà ở khả năng số hóa không gian cabin thành các thông tin tọa độ có cấu trúc, đóng vai trò là dữ liệu đầu vào cho các tầng suy luận phía sau. Điều này cho phép hệ thống chuyển từ bài toán nhận diện đơn thuần sang bài toán suy luận hành vi dựa trên quan hệ không gian (spatial reasoning).

Đối với hành vi sử dụng điện thoại, YOLO chịu trách nhiệm phát hiện và khoanh vùng (bounding box) vị trí của điện thoại trong khung hình. Tuy nhiên, sự tồn tại của

điện thoại không đồng nghĩa với việc tài xế đang vi phạm, bởi thiết bị có thể nằm trên bảng điều khiển, ghế phụ hoặc do hành khách cầm. Do đó, đầu ra của YOLO được sử dụng như một tín hiệu kích hoạt (trigger signal), trả lời cho câu hỏi “điện thoại có xuất hiện trong không gian hay không và ở đâu”. Trên cơ sở đó, hệ thống tiếp tục khai thác thông tin tư thế từ MediaPipe Pose để xác định mối quan hệ hình học giữa điện thoại và các điểm khớp (keypoints) của người lái, từ đó suy luận liệu thiết bị có nằm trong vùng thao tác của tài xế hay không.

Đối với hành vi hút thuốc, YOLO được sử dụng như một bộ lọc sơ cấp nhằm phát hiện các vùng nghi vấn (candidate regions) chứa các vật thể nhỏ như điếu thuốc. Đây là một bài toán rất thách thức do kích thước đối tượng rất nhỏ, dễ bị mất đặc trưng trong quá trình resize ảnh, đồng thời thường bị che khuất một phần (occlusion) bởi tay hoặc khuôn mặt. Ngoài ra, các vật thể có hình dạng tương tự như bút, ống hút hoặc các vật mảnh cầm tay có thể gây nhiễu đáng kể, dẫn đến hiện tượng nhận diện nhầm (false positive). Trong bối cảnh đó, YOLO đóng vai trò như một cơ chế sàng lọc nhanh, xác định các ứng viên tiềm năng để các tầng suy luận phía sau tiếp tục kiểm chứng bằng cách đối chiếu với vị trí tay, miệng hoặc vùng đầu của tài xế, qua đó nâng cao độ tin cậy của quyết định cuối cùng.

Đối với trạng thái dây an toàn, YOLO có thể được áp dụng theo hai chiến lược chính. Chiến lược thứ nhất là nhận diện trực tiếp dải dây an toàn xuất hiện trên cơ thể người lái. Cách tiếp cận này giúp mô hình bám sát đặc trưng hình ảnh cụ thể, tuy nhiên dễ gặp khó khăn trong các trường hợp dây bị che khuất, có màu sắc tương đồng với trang phục hoặc trong điều kiện ánh sáng kém. Chiến lược thứ hai là xây dựng lớp đối lập như no-seatbelt, cho phép mô hình học cách nhận diện trạng thái không thắt dây dựa trên bố cục vùng ngực và vai. Phương pháp này tận dụng được thông tin ngữ cảnh rộng hơn nhưng lại nhạy cảm với các yếu tố gây nhiễu như dây đeo túi, họa tiết sọc chéo trên áo hoặc bóng đổ. Do đó, việc lựa chọn chiến lược gán nhãn phù hợp và thiết kế dữ liệu huấn luyện đóng vai trò quyết định đến hiệu quả của mô hình.

Một yếu tố quan trọng để YOLO trở thành lựa chọn phù hợp cho bài toán DMS là tốc độ suy luận nhanh, đáp ứng yêu cầu xử lý thời gian thực trên các nền tảng Edge AI. Trong môi trường giao thông, hệ thống cần phân tích liên tục luồng video và đưa ra cảnh báo trong thời gian rất ngắn. Nếu độ trễ xử lý quá lớn, hệ thống sẽ mất đi khả năng phản ứng kịp thời trước các tình huống nguy hiểm. Với đặc điểm của mạng one-stage, YOLO tối ưu hóa chi phí tính toán, cho phép xử lý nhiều khung hình mỗi giây

(FPS), từ đó phù hợp với việc triển khai trên các thiết bị nhúng có tài nguyên hạn chế như NVIDIA Jetson tại các nút biên.

Bên cạnh đó, YOLO giữ vai trò trung tâm trong kiến trúc Hybrid Edge–Cloud của hệ thống. Tại tầng Edge, mô hình thực hiện phát hiện nhanh để xác định các tình huống khả nghi và kích hoạt các cơ chế suy luận cục bộ. Các kết quả này sau đó có thể được chuyển lên tầng Cloud để xử lý chuyên sâu hơn, chẳng hạn như xác thực lại hành vi hoặc lưu trữ bằng chứng. Cách tiếp cận phân tầng này giúp tận dụng ưu điểm về tốc độ của YOLO, đồng thời khắc phục hạn chế về độ chính xác trong các tình huống phức tạp.

Để YOLO phát huy hiệu quả trong môi trường cabin, việc xây dựng bộ dữ liệu huấn luyện cần được thực hiện với tiêu chuẩn cao. Dữ liệu phải phản ánh đầy đủ sự biến thiên của môi trường thực tế, bao gồm đa dạng góc đặt camera, điều kiện chiếu sáng, sự khác biệt về hình thể, trang phục và hành vi của tài xế. Đặc biệt, việc bổ sung các mẫu âm tính khó (hard negatives), chẳng hạn như hình ảnh tài xế cầm chai nước, bút, ống hút hoặc điện thoại của hành khách, đóng vai trò quan trọng trong việc giảm thiểu hiện tượng cảnh báo sai, nâng cao độ ổn định và khả năng tổng quát hóa của mô hình khi triển khai thực tế.

Tóm lại, trong kiến trúc hệ thống DMS đề xuất, YOLO đóng vai trò là thành phần trước tiên và quan trọng trong việc quét và số hóa không gian cabin với tốc độ cao. Mặc dù đầu ra của mô hình chưa đủ để đưa ra kết luận cuối cùng về hành vi, nhưng khi được tích hợp với Pose Estimation, các luật suy luận hình học (spatial logic rules) và cơ chế xác thực nhiều tầng, YOLO trở thành nền tảng cốt lõi giúp xây dựng một hệ thống giám sát hành vi tài xế chính xác, thời gian thực và có tính ứng dụng cao trong thực tế.

2.2. Ước lượng tư thế và MediaPipe Pose

2.2.1. Tổng quan bài toán Pose Estimation

Ước lượng tư thế người (Human Pose Estimation) là một bài toán thị giác máy tính chuyên sâu, nhằm mục đích định vị và biểu diễn cấu trúc không gian của cơ thể người thông qua việc xác định tọa độ của các điểm mốc giải phẫu quan trọng (anatomical landmarks/keypoints). Các điểm mốc này thường tương ứng với các khớp nối hoặc bộ phận đặc trưng như mũi, mắt, tai, vai, khuỷu tay, cổ tay, hông và đầu gối. Tùy thuộc vào kiến trúc mạng và yêu cầu của bài toán, hệ thống có thể xuất ra tọa độ 2D trên mặt phẳng ảnh số hoặc mở rộng ra không gian 3D để cung cấp thêm

thông tin về chiều sâu hình học. Khác với bài toán phát hiện đối tượng (Object Detection) vốn chỉ tập trung vào việc khoanh vùng sự hiện diện của vật thể bằng các hộp giới hạn, Pose Estimation hướng tới việc mô hình hóa cấu trúc đồ thị (graph structure) và trạng thái vận động động học của con người.

Sự khác biệt về bản chất này khiến Pose Estimation đóng một vai trò không thể thay thế trong hệ thống giám sát hành vi tài xế. Trong thực tế, nhiều hành vi nguy hiểm không thể được định nghĩa và kết luận chỉ thông qua sự hiện diện của một vật thể đơn lẻ. Chẳng hạn, khi mô hình YOLO phát hiện một chiếc điện thoại di động trong khung hình, dữ liệu đó chưa đủ cơ sở để khẳng định tài xế đang vi phạm quy định, bởi thiết bị có thể đang nằm trên bảng điều khiển hoặc do hành khách ghé phụ cầm. Để chuyển hóa từ việc "phát hiện vật thể" sang "nhận diện hành vi", hệ thống cần một hệ quy chiếu không gian để đánh giá mối tương quan hình học giữa vật thể đó và cơ thể người lái. Thông qua việc trích xuất các điểm mốc như cổ tay, khuỷu tay và tai, Pose Estimation cung cấp dữ liệu hình học thiết yếu để hệ thống lập luận xem chiếc điện thoại có đang nằm trong tay tài xế và được đưa lên sát tai hay không, từ đó đưa ra kết luận hành vi một cách chính xác.

Bên cạnh khả năng cung cấp ngữ cảnh hình học, Pose Estimation còn thể hiện sự ưu việt trong việc giải quyết bài toán che khuất cục bộ (partial occlusion) vốn rất phổ biến trong môi trường buồng lái. Khi điều khiển phương tiện, một phần cơ thể của tài xế thường xuyên bị che khuất bởi vô lăng, phần tựa lưng của ghế, hoặc chính cánh tay của họ. Mặc dù dữ liệu hình ảnh bị thiếu hụt, các mô hình Pose Estimation hiện đại vẫn có khả năng học được cấu trúc giải phẫu tổng thể để nội suy tương đối chính xác các điểm mốc bị khuất dựa trên các điểm mốc hiển thị rõ ràng (như vùng đầu và vai). Năng lực nội suy này giúp hệ thống luôn duy trì được việc theo dõi và khoanh vùng chính xác khu vực thao tác của người lái (Driver ROI), đảm bảo tính liên tục của quá trình giám sát.

Về mặt phương pháp luận, các kiến trúc học sâu giải quyết bài toán Pose Estimation thường được chia thành hai trường phái chính: tiếp cận từ trên xuống (Top-down) và tiếp cận từ dưới lên (Bottom-up). Phương pháp Top-down hoạt động theo cơ chế hai bước: trước tiên sử dụng một bộ phát hiện đối tượng để khoanh vùng từng người trong ảnh, sau đó mới áp dụng mạng ước lượng tư thế trên từng vùng cắt (crop) đó. Ngược lại, phương pháp Bottom-up dự đoán toàn bộ các điểm mốc trong ảnh cùng một lúc, sau đó sử dụng các thuật toán nhóm (grouping) để ghép nối các

điểm mốc thuộc về cùng một cá thể. Đối với bài toán DMS, do đặc thù không gian cabin hẹp, số lượng đối tượng (tài xế) cố định và nằm ở vị trí trung tâm, phương pháp tiếp cận theo hướng Top-down với các luồng xử lý nhẹ (lightweight pipelines) thường được ưu tiên để tối ưu hóa chi phí tính toán trên thiết bị biên.

Xét trên khía cạnh xử lý chuỗi thời gian (video processing), một thách thức lớn của Pose Estimation là tính nhất quán theo thời gian (temporal consistency). Nếu các tọa độ điểm mốc dao động ngẫu nhiên (jitter) giữa các khung hình liên tiếp, các thuật toán suy luận không gian dựa trên khoảng cách sẽ liên tục sinh ra nhiễu, dẫn đến cảnh báo sai. Do đó, các bộ khung (framework) được lựa chọn triển khai thực tế không chỉ yêu cầu khả năng trích xuất đặc trưng không gian tốt, mà còn phải được tích hợp các bộ lọc làm mượt (smoothing filters) để ổn định tọa độ theo thời gian. Đây cũng là tiền đề quan trọng định hướng cho việc lựa chọn và ứng dụng MediaPipe Pose vào dự án.

Tóm lại, trong kiến trúc của hệ thống đề xuất, Pose Estimation không tồn tại độc lập hay thay thế Object Detection, mà đóng vai trò là một tầng trích xuất ngữ cảnh hình học liền mạch. Sự giao thoa giữa dữ liệu bounding box của vật thể và dữ liệu tọa độ giải phẫu của cơ thể người tạo nên một nền tảng vững chắc, giúp hệ thống không chỉ "nhìn" thấy các yếu tố rời rạc mà thực sự "hiểu" được cấu trúc hành vi phức tạp của tài xế trong môi trường vận hành thực tế.

2.2.2. Framework MediaPipe Pose

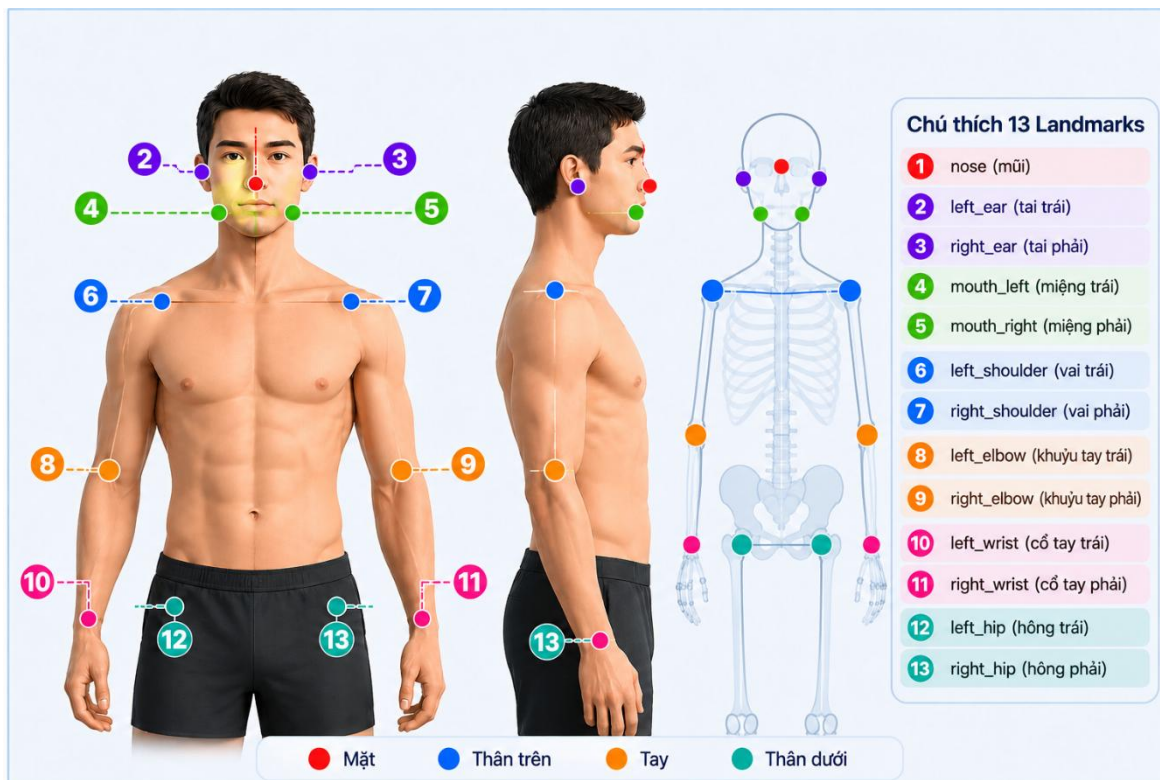
MediaPipe là một bộ khung [11] (framework) mã nguồn mở đa nền tảng do Google phát triển, chuyên biệt cho việc xây dựng các luồng xử lý dữ liệu (pipelines) đa phương tiện theo thời gian thực. MediaPipe Pose được đánh giá là một giải pháp phù hợp cho bài toán ước lượng tư thế người, đặc biệt được thiết kế để vận hành trên các thiết bị có tài nguyên tính toán hạn chế (Edge AI). MediaPipe Pose được xây dựng dựa trên kiến trúc mạng BlazePose – một mạng nơ-ron tích chập siêu nhẹ (ultra-lightweight CNN). Thay vì phải quét toàn bộ bức ảnh ở độ phân giải cao trong mọi khung hình, BlazePose vận hành theo cơ chế hai luồng (two-stage pipeline) nối tiếp. Ở bước đầu tiên, một mạng dò tìm nhanh (Detector) sẽ xác định vùng chứa cơ thể người (Region of Interest - ROI). Sau đó, một mạng hồi quy (Tracker) sẽ chỉ tập trung xử lý bên trong vùng ROI này để trích xuất tọa độ điểm mốc. Cách tiếp cận theo vết này giúp giảm thiểu chi phí tính toán, cho phép hệ thống đạt được tốc độ khung hình (FPS) tốt trên các thiết bị biên như NVIDIA Jetson [1].

MediaPipe Pose cung cấp khả năng phát hiện 33 điểm mốc [11] giải phẫu (anatomical landmarks) bao phủ toàn bộ cơ thể. Tuy nhiên, trong ngữ cảnh của hệ thống giám sát hành vi tài xế, hệ thống không khai thác toàn bộ 33 điểm mốc mà áp dụng kỹ thuật lọc đặc trưng (feature filtering). Cụ thể, chỉ trích xuất 13 điểm mốc then chốt, bao gồm: mũi (nose), tai trái/phải (left_ear, right_ear), vai trái/phải (left_shoulder, right_shoulder), khuỷu tay trái/phải (left_elbow, right_elbow), cổ tay trái/phải (left_wrist, right_wrist), hông trái/phải (left_hip, right_hip) và miệng trái/phải (mouth_left, mouth_right). Các điểm mốc này được lọc theo chỉ số visibility (ngưỡng mặc định 0.35) để loại bỏ các tọa độ bị nhiễu hoặc che khuất. Cách này giúp hệ thống tái cấu trúc hình học của phần thân trên, định vị vùng lồng ngực (Chest ROI) và theo dõi quỹ đạo chuyển động của tay tài xế ngay cả khi một phần cơ thể bị vô lăng hoặc góc quay camera che khuất.

Bảng 2.1. Danh sách 13 điểm mốc MediaPipe Pose sử dụng trong hệ thống

STT	Tên điểm mốc	Vùng cơ thể	Vai trò trong hệ thống
1	nose (mũi)	Mặt	Xác định vị trí đầu, tính khoảng cách face proximity
2	left_ear (tai trái)	Mặt	Bổ sung ngữ cảnh vùng mặt, xác định hướng đầu
3	right_ear (tai phải)	Mặt	Bổ sung ngữ cảnh vùng mặt, xác định hướng đầu
4	mouth_left (miệng trái)	Mặt	Xác định hành vi hút thuốc (gần miệng)
5	mouth_right (miệng phải)	Mặt	Xác định hành vi hút thuốc (gần miệng)
6	left_shoulder (vai trái)	Thân trên	Xác định Driver ROI, Chest ROI, shoulder width
7	right_shoulder (vai phải)	Thân trên	Xác định Driver ROI, Chest ROI, shoulder width
8	left_elbow (khuỷu tay trái)	Tay	Ngữ cảnh thao tác tay, ước tính vị trí cổ tay
9	right_elbow (khuỷu tay phải)	Tay	Ngữ cảnh thao tác tay, ước tính vị trí cổ tay
10	left_wrist (cổ tay trái)	Tay	Tính proximity với điện thoại/thuốc lá
11	right_wrist (cổ tay phải)	Tay	Tính proximity với điện thoại/thuốc lá
12	left_hip (hông trái)	Thân dưới	Bổ sung Driver ROI, fallback shoulder width

13	right_hip (hông phải)	Thân dưới	Bổ sung Driver ROI, fallback shoulder width
----	-----------------------	-----------	---



Hình 2.2.2. Minh họa 13 landmarks được sử dụng trong hệ thống

So với việc xử lý trên ảnh tĩnh rời rạc, MediaPipe Pose có ưu thế khi được áp dụng trên chuỗi dữ liệu video liên tục. Framework này hỗ trợ cơ chế làm mượt theo thời gian nhằm tận dụng thông tin từ các khung hình liền trước, từ đó góp phần giảm hiện tượng dao động nhiều (jittering) của các điểm mốc trong quá trình ước lượng tư thế. Nhờ đó, cấu trúc tư thế của tài xế có thể được biểu diễn ổn định hơn giữa các khung hình liên tiếp. Sự ổn định theo thời gian này có ý nghĩa quan trọng đối với độ chính xác của tầng suy luận hành vi, bởi chỉ một sai lệch nhỏ của điểm mốc cũng có thể dẫn đến việc hệ thống tính toán sai khoảng cách không gian (ví dụ: khoảng cách từ tay đến mặt), từ đó sinh ra các cảnh báo giả.

Đặt trong kiến trúc tổng thể của đề tài, MediaPipe Pose được lựa chọn triển khai như một giải pháp bù đắp những hạn chế của mô hình YOLO trong bài toán. Dữ liệu tọa độ giải phẫu từ MediaPipe đóng vai trò là tham số đầu vào thiết yếu cho Động cơ luật suy luận (Behavior Rules Engine). Tại đây, các phép toán hình học sẽ được thực thi để kiểm chứng xem vật thể (điện thoại, điều thuốc) do YOLO phát hiện có nằm

trong vùng thao tác của người lái (Driver ROI) hay không, hoặc dải an toàn có đi cắt ngang qua vùng ngực theo đúng quy chuẩn hay không. Sự kết hợp giữa công nghệ phát hiện vật thể và công nghệ ước lượng tư thế giúp đáp ứng các yêu cầu về tốc độ cũng như độ tin cậy của hệ thống DMS trong thực tiễn.

2.2.3. Ứng dụng MediaPipe Pose trong phân tích hành vi tài xế

Trong kiến trúc của hệ thống giám sát hành vi tài xế, MediaPipe Pose được sử dụng để bổ sung ngữ cảnh hình học cho các kết quả phát hiện từ YOLO. Thay vì chỉ xác định sự xuất hiện của điện thoại, điều thuốc hoặc dây an toàn, hệ thống cần phân tích mối quan hệ không gian giữa các đối tượng này với cơ thể tài xế. Nhờ dữ liệu điểm mốc cơ thể, hệ thống có thể chuyển từ mức nhận diện vật thể sang mức suy luận hành vi, từ đó đánh giá chính xác hơn liệu một đối tượng có thực sự liên quan đến hành vi vi phạm hay không.

Ứng dụng mang tính nền tảng đầu tiên của MediaPipe Pose trong dự án là khả năng thiết lập vùng người lái (Driver Region of Interest - Driver ROI). Dựa trên tọa độ không gian của các điểm mốc chủ chốt như mũi, tai, gốc vai và cổ tay, hệ thống có thể phác họa và khoanh vùng chính xác vị trí tương đối của người điều khiển phương tiện. Việc xác lập Driver ROI là một cơ chế phân hoạch không gian hiệu quả, cho phép hệ thống giảm thiểu các luồng dữ liệu nhiễu đến từ ghế phụ (hành khách cầm điện thoại) hoặc các đồ vật có hình dáng tương đồng nằm trên bảng điều khiển. Nhờ đó, năng lực tính toán của thiết bị Edge được tập trung vào đúng đối tượng mục tiêu, nâng cao độ ổn định trong môi trường cabin phức tạp.

Tiếp nối luồng xử lý không gian, dữ liệu điểm mốc từ MediaPipe Pose được ứng dụng để nội suy vùng ngực (Chest ROI), phục vụ trực tiếp cho bài toán đánh giá trạng thái an toàn thụ động (thắt dây an toàn). Từ tọa độ hai vai và trục xương sống thân trên, thuật toán hình học có thể dự đoán được quỹ đạo chuẩn xác mà dải dây an toàn phải vắt ngang qua nếu tài xế tuân thủ quy định. Khi YOLO trả về kết quả phát hiện hoặc không phát hiện được dải dây (seatbelt/no-seatbelt), việc đối chiếu tọa độ bounding box này với ranh giới của Chest ROI sẽ cung cấp một lớp xác thực chéo (cross-validation), giúp loại bỏ các báo động giả do nếp gấp áo hoặc quai đeo túi xách gây ra.

Đối với các hành vi mất tập trung thao tác như sử dụng điện thoại, MediaPipe Pose cung cấp công cụ để đo lường tương quan khoảng cách Euclidean giữa vật thể và cơ thể người. Cụ thể, sau khi YOLO khoanh vùng được chiếc điện thoại, hệ thống

sẽ tính toán khoảng cách từ tâm của bounding box đến tọa độ cổ tay và tai của tài xế. Nếu khoảng cách này nằm trong một ngưỡng giới hạn cho phép (thuộc vùng thao tác chủ động), hệ thống mới kết luận tài xế đang sử dụng điện thoại. Cơ chế lọc ngữ cảnh này giảm thiểu nguy cơ nhận diện nhầm, biến những tín hiệu thị giác thô thành các bằng chứng vi phạm có tính logic cao.

Trong trường hợp nhận diện hành vi hút thuốc, điều thuốc có đặc thù là kích thước pixel nhỏ, rất dễ bị che khuất và hòa lẫn vào nền ảnh. Tuy nhiên, hành vi hút thuốc lại luôn gắn liền với một mẫu động học đặc trưng: quỹ đạo tay đưa lên hội tụ tại vùng miệng. Bằng cách giám sát khoảng cách liên kết giữa điểm mốc cổ tay và điểm mốc khuôn mặt (mũi/miệng), hệ thống có thể củng cố niềm tin cho dự đoán của YOLO. Sự kết hợp này giúp phân biệt rõ ràng giữa hành vi hút thuốc thực sự và các hành động gây nhiễu khác như tài xế cầm bút, ngậm tăm hoặc đưa tay lên gãi cằm.

Một giá trị khác của MediaPipe Pose khi triển khai trong thực tế là khả năng khắc phục hiện tượng che khuất cục bộ (partial occlusion). Trong buồng lái, các góc khuất do vô lăng hoặc do chính cánh tay người lái tạo ra là điều không thể tránh khỏi. Dù không thể nhìn thấy toàn bộ các khớp nối, mạng nơ-ron của MediaPipe vẫn có khả năng dựa vào cấu trúc giải phẫu tổng thể để nội suy tương đối các điểm mốc bị che lấp. Nhờ năng lực này, hệ thống duy trì được tính liên tục trong việc theo dõi hành vi, thay vì bị gián đoạn mỗi khi tài xế thay đổi tư thế lái.

Cuối cùng, việc ứng dụng MediaPipe Pose đóng vai trò là nguồn dữ liệu hình học quan trọng để vận hành Động cơ luật suy luận (Behavior Rules Engine) của hệ thống DMS. Các tọa độ điểm mốc được cập nhật theo thời gian thực giúp hệ thống xây dựng các luật không gian như đo khoảng cách giữa tay và khuôn mặt, xác định vật thể có nằm trong vùng người lái hay không. Cách tiếp cận này kết hợp ưu điểm của học sâu trong nhận diện mẫu với luật hình học trong suy luận hành vi, qua đó giúp hệ thống tăng độ tin cậy, dễ kiểm soát logic và thuận tiện hơn khi tinh chỉnh trong quá trình triển khai thực tế.

2.3. Nhận diện hành động (Action Recognition)

2.3.1. Tổng quan bài toán Action Recognition

Nhận diện hành động (Action Recognition) là một bài toán phân tích thị giác máy tính ở cấp độ cao, tập trung vào việc xác định và phân loại một hành động hoặc chuỗi hành vi diễn ra liên tục theo thời gian. Khác biệt căn bản so với các bài toán xử lý ảnh tĩnh như phát hiện đối tượng (Object Detection) hay ước lượng tư thế (Pose

Estimation), Action Recognition không chỉ khai thác cấu trúc không gian (spatial information) tại một khung hình đơn lẻ, mà còn phải mô hình hóa sự biến thiên động học (temporal information) xuyên suốt một chuỗi các khung hình liên tiếp. Nói cách khác, hệ thống phải trích xuất được các đặc trưng không gian - thời gian (spatio-temporal features) để hiểu trọn vẹn ngữ cảnh của luồng video.

Sự cần thiết của việc tích hợp chiều thời gian xuất phát từ tính đa nghĩa của các khung hình tĩnh. Trong môi trường buồng lái, một hình ảnh chụp lại khoảnh khắc tài xế đưa tay lên gần mặt có thể tương ứng với vô số trạng thái khác nhau: nghe điện thoại, hút thuốc, gãi mặt, lau mồ hôi hoặc điều chỉnh kính. Nếu chỉ sử dụng mô hình trích xuất đặc trưng không gian, hệ thống rất dễ rơi vào bẫy cảnh báo sai (False Positive) do thiếu đi thông tin về chu kỳ và quỹ đạo chuyển động. Tuy nhiên, khi đối chiếu qua lăng kính thời gian, mỗi hành vi sẽ bộc lộ một "chữ ký động học" (kinematic signature) riêng biệt. Ví dụ, hành vi hút thuốc bao gồm một chuỗi thao tác tuần hoàn: đưa tay cầm vật lên miệng, duy trì trong khoảng thời gian ngắn, nhả khói và hạ tay xuống. Tương tự, hành vi dùng điện thoại được đặc trưng bởi việc duy trì thiết bị ở sát tai trong một khoảng thời gian dài. Việc phân tích chuỗi động học này giúp hệ thống phá vỡ sự nhập nhằng của ảnh tĩnh và xác thực hành vi với độ tin cậy cao.

Về mặt phương pháp luận, các kỹ thuật nhận diện hành động đã trải qua nhiều giai đoạn phát triển. Trước kỷ nguyên của học sâu, các hệ thống chủ yếu dựa vào đặc trưng thủ công kết hợp với thuật toán học máy cổ điển, tiêu biểu là việc sử dụng quang sai (Optical Flow) để mô phỏng sự dịch chuyển của điểm ảnh. Với sự bùng nổ của mạng nơ-ron nhân tạo, các phương pháp hiện đại tập trung vào kiến trúc học sâu như mạng tích chập kết hợp bộ nhớ ngắn hạn (CNN-LSTM), mạng tích chập 3D (3D CNN), hoặc các kiến trúc đa luồng nhằm phân tách và xử lý song song thông tin không gian và thời gian. Mặc dù đạt được độ chính xác cao nhờ khả năng tự động học biểu diễn đặc trưng từ dữ liệu thô, các kiến trúc này đi kèm với chi phí tính toán lớn và yêu cầu năng lực xử lý GPU rất lớn, vượt quá giới hạn của các bo mạch nhúng thông thường.

Chính vì rào cản về tài nguyên phần cứng, trong khuôn khổ kiến trúc Hybrid Edge-Cloud của đề tài, mô hình Action Recognition không được triển khai thường trực trên thiết bị biên (Edge). Thay vào đó, nó được đặt tại trung tâm xử lý đám mây (Cloud Node), đóng vai trò như một tầng xác thực chuyên sâu (Deep Verification Layer). Quy trình diễn ra như sau: khi hệ thống Edge phân tích các khung hình tĩnh

và phát hiện các tình huống nghi vấn thông qua YOLO và MediaPipe Pose, một đoạn video ngắn (video clip) chứa khoảnh khắc đó sẽ được trích xuất và gửi lên Cloud. Tại đây, mô hình nhận diện hành động sẽ tiến hành quét toàn bộ dải thời gian để xác nhận đó là hành vi vi phạm thực sự hay chỉ là một thao tác vô hại. Sự phân tầng này không chỉ giải quyết được bài toán quá tải tính toán tại biên mà còn thiết lập một quy trình nhận thức AI toàn diện: từ việc "nhìn thấy" đối tượng, đến việc "hiểu" ngữ cảnh không gian, và cuối cùng là "xác thực" chuỗi hành vi theo trục thời gian.

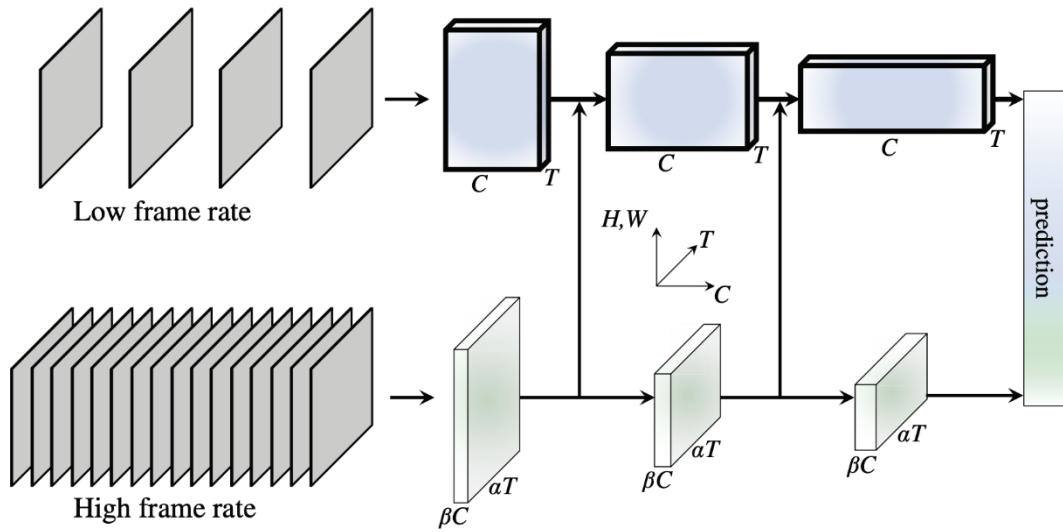
2.3.2. Mô hình SlowFast

SlowFast là một kiến trúc mạng nơ-ron sâu dành cho bài toán phân tích video, được phát triển nhằm khai thác hiệu quả đồng thời thông tin không gian và thời gian trong chuỗi khung hình. Ý tưởng của mô hình là tách việc xử lý video thành hai nhánh song song có nhịp độ khác nhau, lấy cảm hứng từ cơ chế xử lý thông tin thị giác trong sinh học: một nhánh tập trung vào thông tin nội dung và hình dạng tổng thể, nhánh còn lại tập trung vào chuyển động nhanh [5].

Kiến trúc SlowFast gồm hai pathway chính:

- Slow Pathway hoạt động trên chuỗi khung hình được lấy mẫu thưa hơn, tức là số frame đầu vào ít hơn trong cùng một khoảng thời gian. Mục tiêu của nhánh này là học các đặc trưng ngữ nghĩa không gian ổn định như bố cục cảnh, hình dạng vật thể, cấu trúc cơ thể và bối cảnh tổng quát. Vì xử lý ít khung hình hơn, Slow Pathway có thể dành nhiều năng lực mô hình hơn để học đặc trưng sâu.
- Fast Pathway hoạt động trên chuỗi khung hình dày hơn, tức là giữ được nhịp thay đổi chuyển động tốt hơn. Nhánh này tập trung vào thông tin thời gian, nghĩa là những biến đổi nhanh và tinh tế trong hành động. Mặc dù xử lý nhiều frame hơn, Fast Pathway thường được thiết kế nhẹ hơn về số kênh đặc trưng để hạn chế chi phí tính toán.

Hai pathway này không vận hành tách biệt hoàn toàn mà có cơ chế trao đổi thông tin thông qua các kết nối ngang (lateral connections). Nhờ đó, mô hình có thể kết hợp được cả sự ổn định của đặc trưng không gian lẫn độ nhạy với chuyển động theo thời gian. Đây là một điểm rất mạnh của SlowFast so với những kiến trúc chỉ tập trung đơn thuần vào một khía cạnh.



Hình 2.3.2. Kiến trúc SlowFast 2-pathway gồm Slow Pathway, Fast Pathway và lateral connections

Trong bối cảnh DMS, SlowFast đặc biệt phù hợp cho các nhiệm vụ xác thực hành vi ở tầng sâu hơn. Ví dụ, một clip ngắn được trích xuất khi Edge nghi ngờ tài xế đang hút thuốc có thể được gửi đến tầng xử lý chuyên sâu để SlowFast đánh giá. Lúc này, nhánh Slow có thể nắm bắt bối cảnh như vị trí khuôn mặt, tay và vật thể nhỏ; còn nhánh Fast có thể theo dõi chuỗi chuyển động tay lên xuống gần miệng. Sự kết hợp này giúp mô hình phân biệt tốt hơn giữa hành vi hút thuốc thật và các cử động tương tự nhưng vô hại.

Một lý do quan trọng khác khiến SlowFast được đánh giá cao là nó không buộc phải hy sinh toàn bộ tốc độ để đổi lấy chất lượng. So với một số mô hình 3D CNN nặng nề xử lý đồng đều mọi khung hình, SlowFast tận dụng cách chia vai trò giữa hai nhánh để tối ưu hiệu quả học đặc trưng. Tuy nhiên, so với YOLO hoặc MediaPipe Pose, đây vẫn là mô hình có chi phí suy luận cao hơn đáng kể. Vì thế, trong kiến trúc hệ thống của đề tài, SlowFast phù hợp hơn với vai trò mô hình chuyên sâu ở Cloud hoặc mô hình xác thực lại cảnh báo, thay vì chạy liên tục trên thiết bị Edge.

Từ góc độ thiết kế hệ thống, việc sử dụng SlowFast như một lớp xác minh hậu kiểm là hợp lý. Edge đảm nhiệm phát hiện nhanh và cảnh báo tức thời nhờ các mô hình nhẹ. Khi phát sinh sự kiện nghi ngờ hoặc cần lưu bằng chứng chất lượng cao, hệ thống có thể gửi đoạn video ngắn lên Cloud để SlowFast đánh giá chính xác hơn. Cách bố trí này vừa tận dụng được ưu điểm của mô hình video mạnh, vừa không làm quá tải tài nguyên tính toán tại biên.

2.3.3. Đặc trưng không gian – thời gian

Khái niệm đặc trưng không gian – thời gian là nền tảng lý thuyết quan trọng của các mô hình phân tích video. Trong xử lý ảnh tĩnh, mô hình chủ yếu trích xuất đặc trưng không gian, tức là các thông tin liên quan đến hình dạng, màu sắc, biên, kết cấu và vị trí tương đối trong một khung ảnh đơn lẻ. Tuy nhiên, đối với video, chỉ đặc trưng không gian là chưa đủ. Một hành động chỉ có thể được hiểu đầy đủ khi xem xét thêm sự thay đổi của các đặc trưng đó qua thời gian. Chính sự kết hợp này tạo thành đặc trưng không gian – thời gian.

Phát biểu một cách đơn giản, đặc trưng không gian cho biết “đang có gì trong khung hình”, còn đặc trưng thời gian cho biết “nó đang thay đổi như thế nào”. Trong bài toán giám sát hành vi tài xế, một khung hình có thể cho thấy tay người lái đang ở gần mặt, nhưng chỉ khi quan sát nhiều khung hình liên tiếp, hệ thống mới nhận ra đó là một chuyển động lặp lại của hành vi hút thuốc hoặc dùng điện thoại.

Trong các mô hình học sâu dành cho video, đặc trưng không gian – thời gian thường được học thông qua các lớp tích chập 3D hoặc các cơ chế xử lý chuỗi. Khác với tích chập 2D chỉ hoạt động trên chiều cao và chiều rộng của ảnh, tích chập 3D mở rộng thêm một chiều thời gian, cho phép bộ lọc học trực tiếp các mẫu chuyển động qua chuỗi frame. Nhờ vậy, mô hình không chỉ nhận ra một vật thể hoặc tư thế, mà còn hiểu được nhịp điệu và hướng vận động của chúng.

Vai trò của đặc trưng không gian – thời gian đặc biệt rõ trong việc phân biệt các hành vi có bối cảnh tĩnh khá giống nhau. Ví dụ, tay đưa lên gần đầu có thể là nghe điện thoại, chỉnh tóc hoặc gãi đầu. Nếu chỉ xét hình ảnh tĩnh, các trường hợp này đôi khi rất khó tách biệt. Tuy nhiên, khi xét theo thời gian, mỗi hành vi lại có quy luật chuyển động khác nhau về quỹ đạo tay, thời lượng duy trì, vị trí kết thúc hoặc sự phối hợp với các vật thể liên quan. Đây là lý do tại sao Action Recognition thường đạt hiệu quả cao hơn trong việc xác nhận hành vi thực sự.

Trong hệ thống của đề tài, khái niệm đặc trưng không gian – thời gian có thể được hiểu như tầng thông tin sâu hơn dùng để xác thực hành vi sau khi đã có các tín hiệu sơ cấp từ YOLO và MediaPipe Pose. YOLO cung cấp thông tin về vật thể và vị trí tức thời. MediaPipe Pose cung cấp thông tin tư thế và quan hệ hình học cơ thể. SlowFast hoặc các mô hình nhận diện hành động khác khai thác tiếp chuỗi biến đổi theo thời gian để đưa ra nhận định mạnh hơn. Sự phân tầng này giúp hệ thống vừa nhanh ở giai đoạn đầu, vừa sâu ở giai đoạn xác thực.

Tóm lại, đặc trưng không gian – thời gian là cơ sở để chuyển từ phân tích ảnh đơn sang hiểu video và hành vi. Đối với các hệ thống DMS hiện đại, đây là lớp thông tin có tiềm năng rất lớn để cải thiện độ chính xác, đặc biệt trong các tình huống mà một khung hình đơn lẻ chưa đủ làm bằng chứng kết luận.

2.4. Tối ưu hóa mô hình cho thiết bị Edge

2.4.1. Quantization (Lượng tử hóa)

Một trong những rào cản kỹ thuật lớn nhất khi triển khai các mô hình mạng nơ-ron sâu trên thiết bị biên (Edge Devices) là sự hạn chế về dung lượng bộ nhớ, băng thông, điện năng tiêu thụ và năng lực tính toán. Theo tiêu chuẩn mặc định, các mô hình sau khi huấn luyện thường lưu trữ trọng số (weights) và giá trị kích hoạt (activations) dưới định dạng dấu phẩy động độ chính xác đơn 32-bit (FP32). Mặc dù định dạng này cung cấp dải biểu diễn số học rộng và chính xác, nó lại tạo ra gánh nặng tính toán. Đối với các hệ thống nhúng yêu cầu xử lý luồng video thời gian thực, sự quá tải về băng thông bộ nhớ (memory bandwidth) do phải liên tục đọc/ghi các ma trận FP32 là nguyên nhân chính dẫn đến hiện tượng sụt giảm tốc độ khung hình. Lượng tử hóa (Quantization) là một kỹ thuật tiềm năng để giải quyết vấn đề này. [7]

Về bản chất toán học, lượng tử hóa là quá trình ánh xạ (mapping) một dải giá trị liên tục có độ chính xác cao (như FP32) sang một tập hợp các giá trị rời rạc có độ chính xác thấp hơn (số bit ít hơn). Hai định dạng lượng tử hóa được ứng dụng phổ biến nhất trong thực tế là bán độ chính xác FP16 (16-bit Floating Point) và số nguyên INT8 (8-bit Integer). Việc chuyển đổi này mang lại lợi ích kép: thứ nhất, nó nén kích thước vật lý của mô hình xuống từ 2 đến 4 lần, giải phóng đáng kể không gian lưu trữ và RAM; thứ hai, các phép toán ma trận trên số nguyên (INT8) đòi hỏi chu kỳ xung nhịp ít hơn và tiêu thụ ít điện năng hơn nhiều so với phép toán dấu phẩy động. Nhờ đó, thông lượng tính toán (throughput) của hệ thống phần cứng được đẩy lên mức cao hơn.

Xét về phương pháp luận, quy trình lượng tử hóa có thể được thực thi thông qua hai hướng tiếp cận chính. Hướng thứ nhất là Lượng tử hóa sau huấn luyện (Post-Training Quantization - PTQ). Kỹ thuật này thực hiện việc chuyển đổi kiểu dữ liệu sau khi mô hình đã hội tụ hoàn toàn. Để thiết lập các tham số tỷ lệ (scale) và điểm 0 (zero-point) cho quá trình ép kiểu từ số thực sang số nguyên, PTQ đòi hỏi một tập dữ liệu hiệu chỉnh nhỏ (calibration dataset) chạy qua mạng nhằm thống kê dải phân bố động (dynamic range) của các giá trị kích hoạt. Kỹ thuật này đơn giản, triển khai

nhanh và tiết kiệm tài nguyên. Hướng tiếp cận thứ hai là Lượng tử hóa trong quá trình huấn luyện (Quantization-Aware Training - QAT). Trong phương pháp này, các node lượng tử hóa giả (fake quantization nodes) được chèn trực tiếp vào đồ thị tính toán trong lúc huấn luyện. Mạng nơ-ron sẽ tự động tính toán và thích nghi với các nhiễu lượng tử (quantization noise) trong quá trình lan truyền ngược (backpropagation), giúp bảo toàn độ chính xác tốt hơn so với PTQ, dù quy trình thực hiện phức tạp và tốn kém thời gian hơn đáng kể. [7]

Trong bối cảnh xây dựng hệ thống giám sát hành vi tài xế trên các thiết bị nhúng như NVIDIA Jetson, lượng tử hóa là điều kiện tiên quyết để đạt được ngưỡng thời gian thực. Hệ thống phải xử lý đồng thời cả mô hình YOLO và MediaPipe Pose, xử lý liên tục nhiều khung hình mỗi giây. Nếu không ép kiểu dữ liệu xuống FP16 hoặc INT8, thiết bị Edge sẽ nhanh chóng bị quá tải, dẫn đến độ trễ hệ thống tăng và làm mất đi khả năng cảnh báo sớm.

Tuy nhiên, lượng tử hóa luôn đi kèm với bài toán đánh đổi giữa hiệu năng phần cứng và chất lượng dự đoán. Khi ép dải giá trị xuống không gian biểu diễn hẹp hơn (chỉ còn 256 giá trị đối với INT8), mô hình bắt buộc phải chịu sai số làm tròn (truncation error). Đối với các bài toán có tính thử thách cao trong dự án, chẳng hạn như phát hiện điều thuốc nhỏ hoặc nhận diện khuôn mặt bị che khuất trong điều kiện thiếu sáng, sự mất mát thông tin này có thể dẫn đến việc tăng tỷ lệ bỏ sót (False Negative) hoặc nhận diện nhầm (False Positive). Do đó, chiến lược triển khai thực tế đòi hỏi sự tinh chỉnh kỹ lưỡng: cần đánh giá đo lường chỉ số mAP (mean Average Precision) trước và sau lượng tử hóa để tìm ra điểm cân bằng tối ưu, đảm bảo hệ thống vừa duy trì được tốc độ suy luận nhanh, vừa không đánh đổi tính mạng và sự an toàn của người lái.

2.4.2. TensorRT

TensorRT là một bộ trình biên dịch (compiler) và môi trường thực thi (runtime) hiệu năng cao dành riêng cho các mạng nơ-ron sâu, được NVIDIA nghiên cứu và phát triển để tối ưu hóa trên các kiến trúc GPU của hãng. Trong khi các framework học sâu bậc cao như PyTorch hay TensorFlow tập trung vào tính linh hoạt để hỗ trợ quá trình huấn luyện (training), thì TensorRT lại được thiết kế với mục tiêu chính: cải thiện thông lượng suy luận và giảm độ trễ suy luận. Đối với các hệ thống Edge AI vận hành trên hệ sinh thái phần cứng NVIDIA, đặc biệt là dòng vi mạch nhúng Jetson (như Jetson Nano, Xavier, hoặc Orin), TensorRT được xem là giải pháp phù hợp để

chuyển hóa các mô hình từ phòng thí nghiệm thành các ứng dụng chạy thời gian thực trên xe khách.

Quy trình tối ưu hóa của TensorRT thường bắt đầu bằng việc xuất mô hình đã huấn luyện sang một định dạng biểu diễn trung gian chuẩn hóa, phổ biến nhất là đồ thị ONNX (Open Neural Network Exchange). Từ đồ thị không phụ thuộc nền tảng này, trình biên dịch của TensorRT sẽ phân tích cấu trúc mạng và tái cấu trúc nó thành một "engine" thực thi (executable engine) phù hợp với vi kiến trúc phần cứng đang biên dịch. Trong quá trình xây dựng engine này, hệ thống tự động áp dụng một số các phép biến đổi đồ thị phức tạp, nổi bật nhất là kỹ thuật hợp nhất các lớp đồ thị (Layer/Tensor Fusion), cho phép gộp các phép toán nối tiếp nhau (ví dụ: Convolution, Batch Normalization và ReLU) thành một hạt nhân tính toán (kernel) duy nhất. Sự hợp nhất này giúp loại bỏ đáng kể chi phí rào cản (overhead) và giảm thiểu số lần đọc/ghi dữ liệu lặp lại vào bộ nhớ toàn cục (VRAM).

Bên cạnh việc tái cấu trúc đồ thị, TensorRT còn tích hợp tính năng tự động tinh chỉnh nhân tính toán (Kernel Auto-tuning). Trình biên dịch sẽ tiến hành chạy thử nghiệm (benchmarking) hàng loạt các thuật toán và thư viện tối ưu phần cứng (cuDNN, cuBLAS) trực tiếp trên GPU mục tiêu để chọn ra lộ trình thực thi có thời gian hoàn thành ngắn nhất. Đồng thời, quá trình cấp phát và tái sử dụng bộ nhớ động (Memory Optimization) cũng được tính toán trước nhằm tránh sự phân mảnh vùng nhớ trong quá trình chạy thực tế. Đặc biệt, TensorRT cung cấp các công cụ hiệu chuẩn độ chính xác (Precision Calibration), phù hợp với kỹ thuật lượng tử hóa đã đề cập ở phần trước. Nó cho phép chuyển đổi liền mạch các phép toán từ FP32 xuống FP16 hoặc INT8 bằng cách tận dụng các lõi Tensor Cores (nếu có) trên GPU, mang lại hiệu năng gia tốc lớn.

Trong hệ thống giám sát hành vi tài xế của đề tài, việc ứng dụng TensorRT để biên dịch mô hình YOLO và các mô hình thị giác khác được đánh giá là phù hợp. Sự can thiệp của TensorRT giúp giải quyết trực tiếp bài toán sụt giảm tốc độ khung hình (FPS drop) khi hệ thống phải xử lý luồng video liên tục. Thay vì phải chạy ở mức vài khung hình mỗi giây với mô hình PyTorch nguyên bản, một engine TensorRT INT8/FP16 có thể vận hành tốt với tốc độ cao, đáp ứng yêu cầu phát cảnh báo thời gian thực.

Tuy nhiên, sự tối ưu hóa của TensorRT cũng đi kèm với một số ràng buộc mang tính đặc thù. Thứ nhất, tính di động (portability) của mô hình bị loại bỏ phần lớn: một

engine TensorRT được biên dịch trên thiết bị NVIDIA Jetson Nano sẽ không thể mang sang chạy trên Jetson Orin hay các card đồ họa PC thông thường, buộc hệ thống phải thực hiện lại quá trình biên dịch (build engine) trên từng kiến trúc phần cứng đích. Thứ hai, các toán tử (operators) tùy chỉnh hoặc quá mới trong mạng học sâu có thể chưa được TensorRT hỗ trợ ở cấp độ nhân (kernel), đòi hỏi kỹ sư phải viết thêm các plugin mở rộng bằng C++/CUDA hoặc chấp nhận việc fallback (lùi về) xử lý chậm trên CPU. Cuối cùng, khi kết hợp TensorRT với lượng tử hóa INT8, bộ dữ liệu hiệu chỉnh (calibration dataset) phải có tính đại diện cao đối với môi trường cabin xe, nếu không, hiện tượng sụp đổ độ chính xác (accuracy degradation) sẽ xảy ra, làm vô hiệu hóa khả năng nhận diện vi phạm.

Tóm lại, trong thiết kế của phân hệ Edge, nếu Lượng tử hóa là quá trình tối ưu hóa về mặt biểu diễn số học lý thuyết, thì TensorRT chính là công cụ hiện thực hóa sức mạnh tối ưu đó lên cấu trúc vi mạch vật lý. Sự kết hợp giữa hai kỹ thuật này tạo thành một tầng Middleware vững chắc, thu hẹp khoảng cách giữa sự phức tạp của mạng nơ-ron sâu và sự hạn chế của phần cứng nhúng.

2.4.3. Tối ưu hiệu năng trên thiết bị nhúng

Việc triển khai thành công một hệ thống AI trên thiết bị vi mạch nhúng không chỉ dừng lại ở việc tối ưu hóa nội tại mạng nơ-ron (thông qua Lượng tử hóa hay TensorRT), mà còn phụ thuộc vào kiến trúc phần mềm tổng thể. Trong thực tế, một mô hình sở hữu tốc độ suy luận lý thuyết cao vẫn có thể gây ra độ trễ (latency) lớn nếu hệ thống gặp các "nút thắt cổ chai" (bottlenecks) tại các khâu ngoại vi như giải mã luồng video (video decoding), sao chép dữ liệu giữa các phân vùng bộ nhớ, hoặc cơ chế giao tiếp liên tiến trình (IPC). Do đó, tối ưu hóa hiệu năng thiết bị nhúng phải được tiếp cận một cách toàn cục.

Trong nguyên mẫu hiện tại, pipeline xử lý vận hành theo vòng lặp đơn (single-thread) và được tối ưu bằng cơ chế bỏ khung có chủ đích cùng kích hoạt MediaPipe Pose theo điều kiện (chỉ chạy khi có detection liên quan). Việc tách luồng I/O/AI/hiển thị bằng đa luồng là định hướng mở rộng trong các phiên bản tối ưu hóa sau.

Bên cạnh kiến trúc đa luồng, chiến lược trích mẫu thời gian (temporal sampling) hay còn gọi là bỏ khung có chủ đích (controlled frame skipping) là một kỹ thuật thực tiễn mang lại hiệu quả cao. Thay vì ép thiết bị nhúng suy luận toàn bộ 30 khung hình/giây (FPS) từ nguồn phát, hệ thống có thể được cấu hình để chỉ lấy mẫu 5 đến 10 khung hình/giây. Do đặc tính động học của các hành vi vi phạm (như dùng điện

thoại, hút thuốc) thường kéo dài qua nhiều giây, việc giảm tần số lấy mẫu hoàn toàn không làm mất đi các hành vi có giá trị, đồng thời giải phóng đến 70% năng lực tính toán của GPU/CPU. Song song với đó, chiến lược chuẩn hóa không gian (spatial normalization) cũng được áp dụng. Việc hạ độ phân giải ảnh gốc xuống kích thước đầu vào tiêu chuẩn của mô hình (như 640 x 640) kết hợp với kỹ thuật cắt vùng quan tâm (ROI cropping) giúp tối ưu hóa số lượng phép toán dấu phẩy động (FLOPs) mà không làm suy giảm khả năng phát hiện các vật thể nhỏ như điều thuốc.

Một khía cạnh kỹ thuật khác là bài toán quản trị bộ nhớ vật lý. Trên các bo mạch Edge AI (như họ NVIDIA Jetson), bộ nhớ RAM được thiết kế theo kiến trúc chia sẻ thống nhất (Unified Memory Architecture) giữa CPU và GPU. Bằng cách thiết kế phần mềm tận dụng kỹ thuật Zero-copy, CPU và GPU có thể truy cập trực tiếp vào cùng một vùng nhớ chứa ảnh đầu vào mà không cần thực hiện các lệnh sao chép dữ liệu (Memcpy). Hơn nữa, việc quản lý vòng đời của các tensor và bộ đệm (buffer pools) được kiểm soát chặt nhằm tái sử dụng vùng nhớ liên tục, ngăn chặn hiện tượng rò rỉ bộ nhớ (memory leak) gây tràn RAM sau một thời gian dài hoạt động.

Cuối cùng, tối ưu hóa trên thiết bị Edge không chỉ là bài toán về tốc độ khung hình (FPS), mà là bài toán về độ tin cậy và sự bền bỉ. Một hệ thống DMS vận hành trên phương tiện giao thông phải đối mặt với môi trường nhiệt độ biến thiên liên tục. Nếu kiến trúc phần mềm ép phần cứng chạy ở mức tải 100% không ngừng nghỉ, vi xử lý sẽ nhanh chóng bị quá nhiệt và tự động kích hoạt cơ chế điều tiết nhiệt (thermal throttling). Cơ chế này sẽ ép xung nhịp hệ thống xuống mức thấp để bảo vệ linh kiện, dẫn đến hiện tượng giật lag. Do vậy, việc kết hợp đồng bộ các kỹ thuật đa luồng, bỏ khung có chủ đích và quản lý bộ nhớ chính là giải pháp chính để duy trì một ngưỡng nhiệt độ an toàn, đảm bảo thiết bị biên vận hành 24/7 một cách trơn tru, bền bỉ và giữ kết nối liền mạch với trung tâm dữ liệu đám mây.

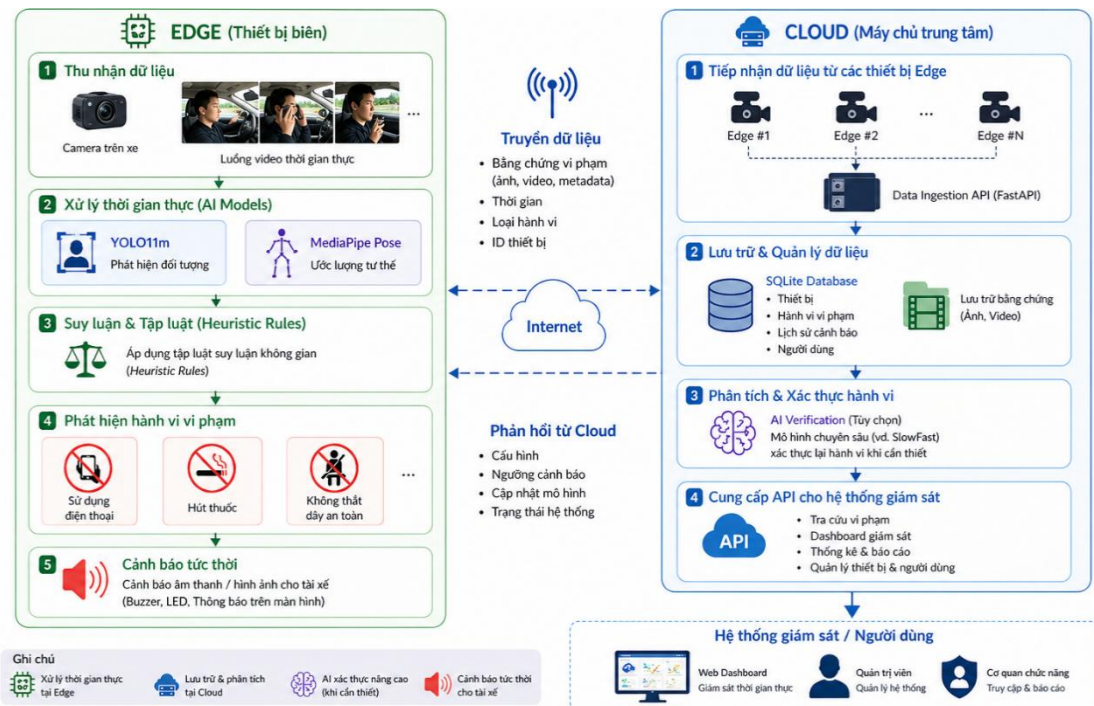
CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1. Kiến trúc tổng thể hệ thống

3.1.1. Mô hình hệ thống tổng thể

Trên cơ sở các phân tích ở Chương 1 và Chương 2, hệ thống được thiết kế theo kiến trúc Hybrid Edge–Cloud gồm hai phân hệ chính: Edge Node và Cloud Node. Mục này tập trung mô tả cấu trúc kỹ thuật tổng thể của hệ thống, bao gồm cách các thành phần được tổ chức, vai trò của từng phân hệ và cách dữ liệu được luân chuyển giữa thiết bị biên và máy chủ đám mây.

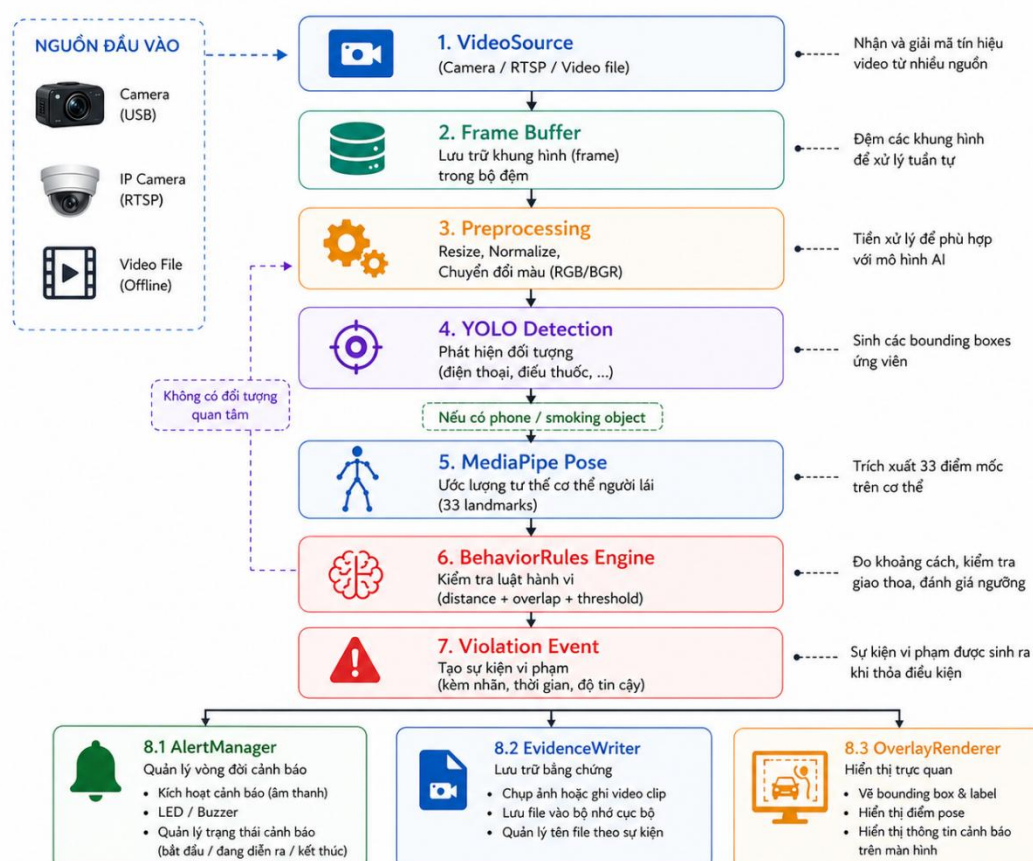
Trong mô hình tổng thể, Edge Node được triển khai tại phương tiện để tiếp nhận luồng video, thực hiện suy luận AI và tạo cảnh báo cục bộ. Cloud Node tiếp nhận dữ liệu sự kiện từ Edge, lưu trữ thông tin cảnh báo, quản lý bằng chứng và cung cấp dữ liệu cho giao diện giám sát. Cách tổ chức này giúp tách rõ nhóm tác vụ cần phản hồi nhanh tại phương tiện và nhóm tác vụ phục vụ quản trị tập trung.



Hình 3.1.1. Kiến trúc tổng thể hệ thống Hybrid Edge–Cloud

Hình 3.1.1 minh họa kiến trúc tổng thể của hệ thống Hybrid Edge–Cloud, trong đó luồng xử lý bắt đầu từ camera tại phương tiện, đi qua các thành phần xử lý tại Edge, sau đó đồng bộ dữ liệu cảnh báo và bằng chứng cần thiết lên Cloud để phục vụ lưu trữ, đối soát và giám sát.

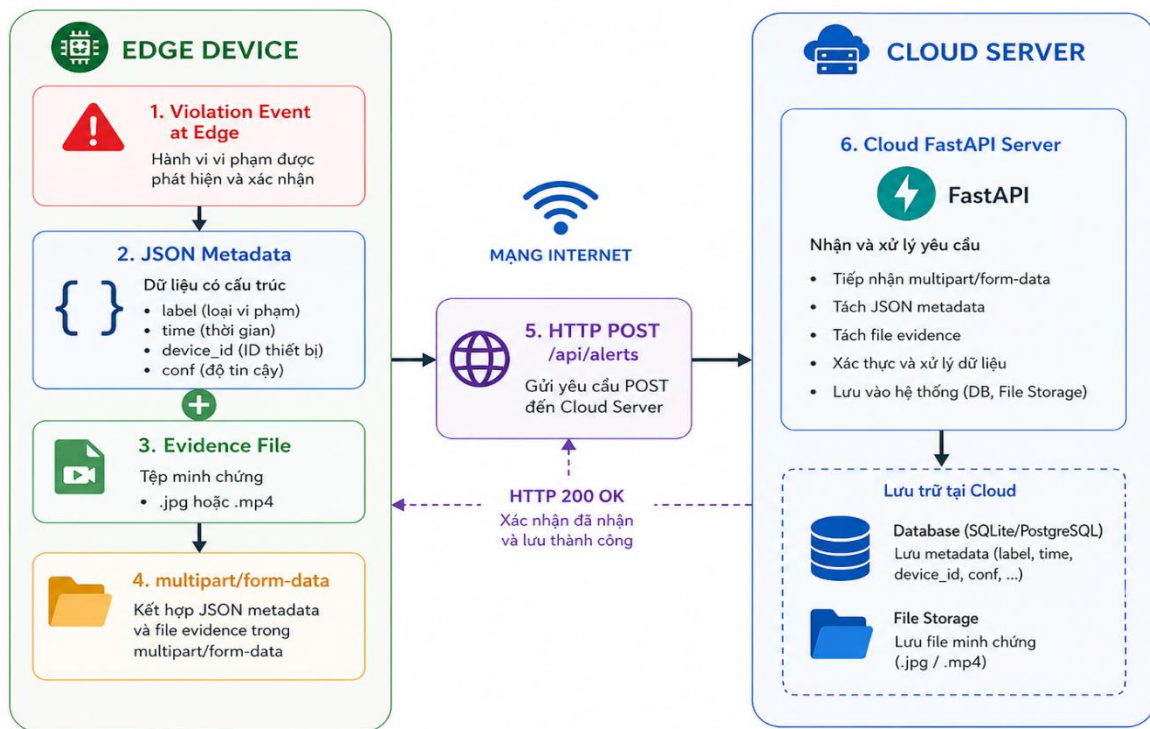
3.1.2. Luồng dữ liệu và giao thức giao tiếp



Hình 3.1.2. Luồng xử lý cục bộ tại Edge trước khi đồng bộ lên Cloud

Hình 3.1.2 mô tả luồng xử lý cục bộ tại thiết bị biên trước khi dữ liệu được đồng bộ lên Cloud. Luồng dữ liệu bắt đầu từ các nguồn đầu vào như camera USB, camera IP hoặc video ngoại tuyến, sau đó được đưa qua các bước đệm khung hình, tiền xử lý, nhận diện đối tượng bằng YOLO, ước lượng tư thế bằng MediaPipe Pose và kiểm tra luật hành vi bằng Behavior Rules Engine.

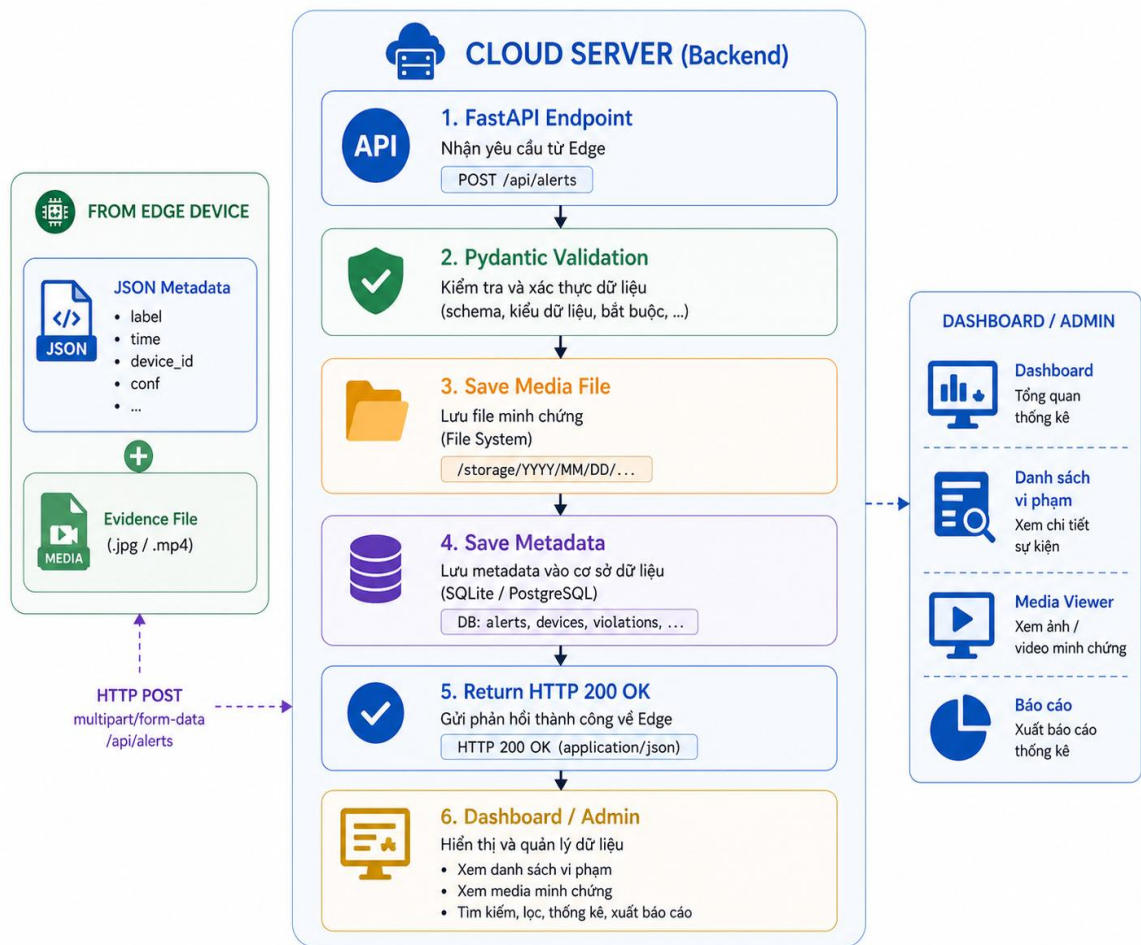
Kết quả cuối cùng của quá trình xử lý tại Edge là sự kiện vi phạm đã được xác định kèm theo các thông tin như nhãn vi phạm, thời gian và độ tin cậy. Từ sự kiện này, hệ thống có thể kích hoạt cảnh báo tại chỗ, lưu lại bằng chứng hình ảnh hoặc video, đồng thời hiển thị thông tin cảnh báo lên giao diện giám sát cục bộ.



Hình 3.1.3. Luồng đồng bộ dữ liệu từ Edge lên Cloud

Hình 3.1.3 trình bày luồng đồng bộ dữ liệu từ thiết bị biên lên máy chủ Cloud sau khi một sự kiện vi phạm được phát hiện. Tại Edge Device, hệ thống đóng gói dữ liệu thành hai phần chính gồm JSON metadata chứa thông tin mô tả vi phạm và evidence file chứa hình ảnh hoặc video bằng chứng.

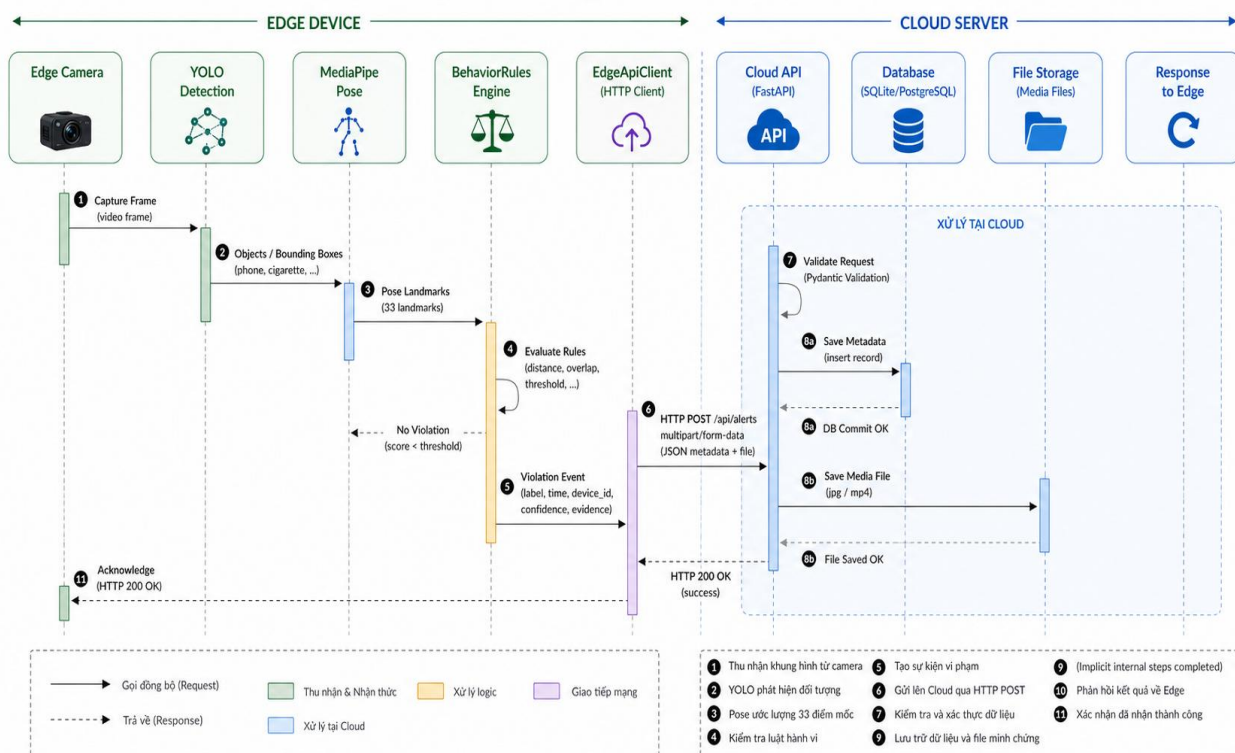
Hai thành phần này được kết hợp dưới dạng multipart/form-data và gửi lên Cloud Server thông qua phương thức HTTP POST tại endpoint `/api/alerts`. Sau khi tiếp nhận, Cloud Server xử lý yêu cầu bằng FastAPI, tách metadata và file bằng chứng, sau đó lưu trữ dữ liệu vào cơ sở dữ liệu và hệ thống lưu trữ tệp.



Hình 3.1.4. Pipeline xử lý dữ liệu tại Cloud

Hình 3.1.4 mô tả pipeline xử lý dữ liệu tại Cloud Server sau khi nhận được cảnh báo từ Edge Device. Dữ liệu đầu vào bao gồm JSON metadata và tệp bằng chứng được gửi qua endpoint FastAPI, sau đó được kiểm tra tính hợp lệ bằng Pydantic Validation nhằm đảm bảo đúng cấu trúc, kiểu dữ liệu và các trường bắt buộc.

Sau bước xác thực, hệ thống lưu tệp phương tiện vào File System, đồng thời ghi thông tin metadata vào cơ sở dữ liệu như SQLite hoặc PostgreSQL. Khi quá trình lưu trữ hoàn tất, Cloud Server trả về phản hồi HTTP 200 OK cho Edge và cung cấp dữ liệu cho Dashboard/Admin để phục vụ xem danh sách vi phạm, xem bằng chứng, tìm kiếm, lọc, thống kê và xuất báo cáo.



Hình 3.1.5. Sequence diagram luồng xử lý và đồng bộ cảnh báo vi phạm

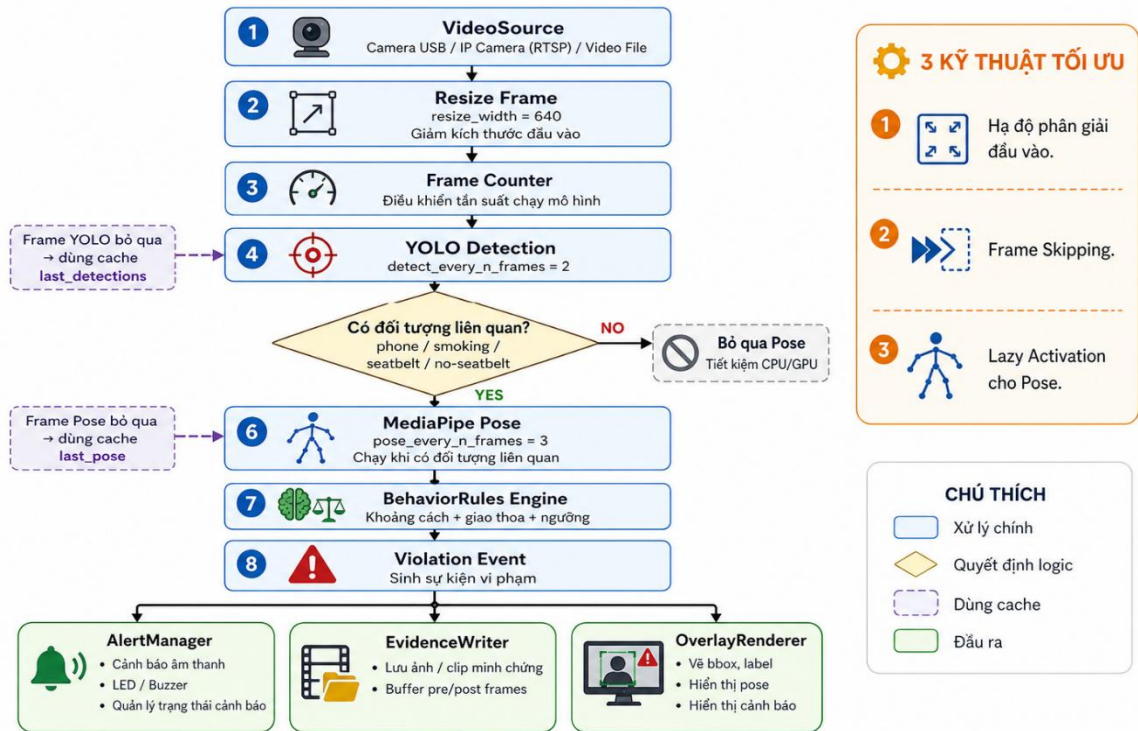
Hình 3.1.5 thể hiện sequence diagram của toàn bộ quá trình xử lý và đồng bộ cảnh báo vi phạm giữa Edge Device và Cloud Server. Ở phía Edge, camera thu nhận khung hình, YOLO phát hiện các đối tượng liên quan, MediaPipe Pose trích xuất điểm mốc cơ thể, sau đó Behavior Rules Engine đánh giá các điều kiện về khoảng cách, giao nhau và ngưỡng để xác định có phát sinh vi phạm hay không.

Khi một sự kiện vi phạm được tạo ra, EdgeApiClient gửi dữ liệu lên Cloud API thông qua HTTP POST. Ở phía Cloud, FastAPI tiếp nhận và xác thực yêu cầu, metadata được lưu vào cơ sở dữ liệu, file bằng chứng được lưu vào File Storage, sau đó hệ thống trả phản hồi thành công về thiết bị biên. Sơ đồ này cho thấy rõ sự phân chia trách nhiệm giữa Edge và Cloud: Edge đảm nhiệm nhận diện và suy luận thời gian thực, còn Cloud đảm nhiệm xác thực, lưu trữ và phục vụ dữ liệu quản trị.

3.2. Thiết kế phân hệ Edge

3.2.1. Pipeline xử lý tại Edge

Pipeline xử lý tại Edge từ lúc tiếp nhận luồng video đầu vào cho đến khi phát sinh sự kiện vi phạm và kích hoạt các mô-đun đầu ra được mô tả trong Hình 3.2.1.



Hình 3.2.1. Pipeline xử lý tại Edge

Pipeline xử lý tại Edge được tổ chức theo dạng tuần tự từ khâu tiếp nhận dữ liệu đầu vào, suy luận mô hình, đánh giá logic hành vi cho đến khi sinh cảnh báo và lưu bằng chứng. Toàn bộ luồng xử lý này được điều phối bởi lớp EdgePipeline, đóng vai trò kết nối các mô-đun chức năng trong phân hệ Edge.

Ở bước đầu tiên, hệ thống tiếp nhận luồng dữ liệu hình ảnh từ nhiều nguồn khác nhau thông qua mô-đun VideoSource, bao gồm camera USB, camera IP sử dụng giao thức RTSP hoặc tệp video ngoại tuyến. Cách tổ chức này giúp pipeline có tính linh hoạt cao, dễ dàng thích ứng với nhiều kịch bản triển khai và kiểm thử khác nhau.

Sau khi khung hình được đọc vào, hệ thống thực hiện bước Resize Frame để đưa ảnh đầu vào về kích thước phù hợp với cấu hình xử lý, chẳng hạn chiều rộng `resize_width = 640`. Việc giảm kích thước đầu vào giúp giảm số lượng điểm ảnh cần xử lý, từ đó tiết kiệm bộ nhớ và cải thiện tốc độ suy luận trên thiết bị Edge có tài nguyên hạn chế.

Tiếp theo, mô-đun Frame Counter được sử dụng để điều khiển tần suất chạy của các mô hình AI. Thay vì thực hiện suy luận ở mọi khung hình, hệ thống áp dụng cơ chế lấy mẫu theo chu kỳ nhằm giảm tải tính toán. Cụ thể, mô hình YOLO Detection chỉ được chạy sau mỗi số khung hình nhất định theo tham số `detect_every_n_frames`.

Ở các khung hình không chạy YOLO, hệ thống tái sử dụng kết quả gần nhất thông qua bộ nhớ đệm `last_detections`. Cơ chế này giúp giảm chi phí xử lý mà vẫn duy trì được tính liên tục của luồng nhận diện.

Từ kết quả phát hiện đối tượng của YOLO, hệ thống thực hiện bước kiểm tra logic để xác định xem có xuất hiện các đối tượng liên quan đến hành vi vi phạm hay không, chẳng hạn như phone, smoking, seatbelt hoặc no-seatbelt. Nếu không có đối tượng liên quan, pipeline sẽ bỏ qua bước ước lượng tư thế nhằm tiết kiệm tài nguyên xử lý. Ngược lại, nếu phát hiện đối tượng nghi vấn, hệ thống sẽ kích hoạt mô-đun MediaPipe Pose để trích xuất các điểm mốc cơ thể người lái. Tương tự như YOLO, mô hình Pose cũng được điều khiển theo chu kỳ thông qua tham số `pose_every_n_frames`, và các khung hình trung gian có thể sử dụng lại kết quả `last_pose`.

Sau khi có đầu ra từ YOLO và MediaPipe Pose, dữ liệu được đưa vào BehaviorRules Engine. Đây là tầng suy luận hành vi, nơi các luật hình học và ngữ cảnh không gian được áp dụng để đánh giá xem tài xế có thực hiện hành vi vi phạm hay không. Các luật này có thể dựa trên khoảng cách giữa vật thể và các điểm mốc cơ thể, mức độ giao thoa giữa các vùng quan tâm hoặc các ngưỡng điều kiện được xác lập trước.

Khi các điều kiện logic được thỏa mãn, hệ thống sinh ra một Violation Event. Sự kiện này là đầu ra quan trọng của pipeline tại Edge, chứa thông tin về loại vi phạm, thời điểm xảy ra và các dữ liệu liên quan phục vụ cảnh báo và lưu trữ.

Bảng 3.1. Cấu hình ngưỡng xác nhận vi phạm theo từng loại hành vi

Event Type	Mô tả	confirm_frames	Ngưỡng điểm	Fallback
using_phone	Sử dụng điện thoại khi lái xe	7	0.62	Không (cần pose)
smoking	Hút thuốc khi lái xe	7	0.62	Có (raw conf $\geq$ 0.70)
no_seatbelt	Không thắt dây an toàn	12	0.45 + margin 0.07	Không

Từ sự kiện vi phạm, hệ thống kích hoạt đồng thời ba mô-đun đầu ra. Thứ nhất, AlertManager chịu trách nhiệm phát cảnh báo cục bộ thông qua âm thanh, LED hoặc

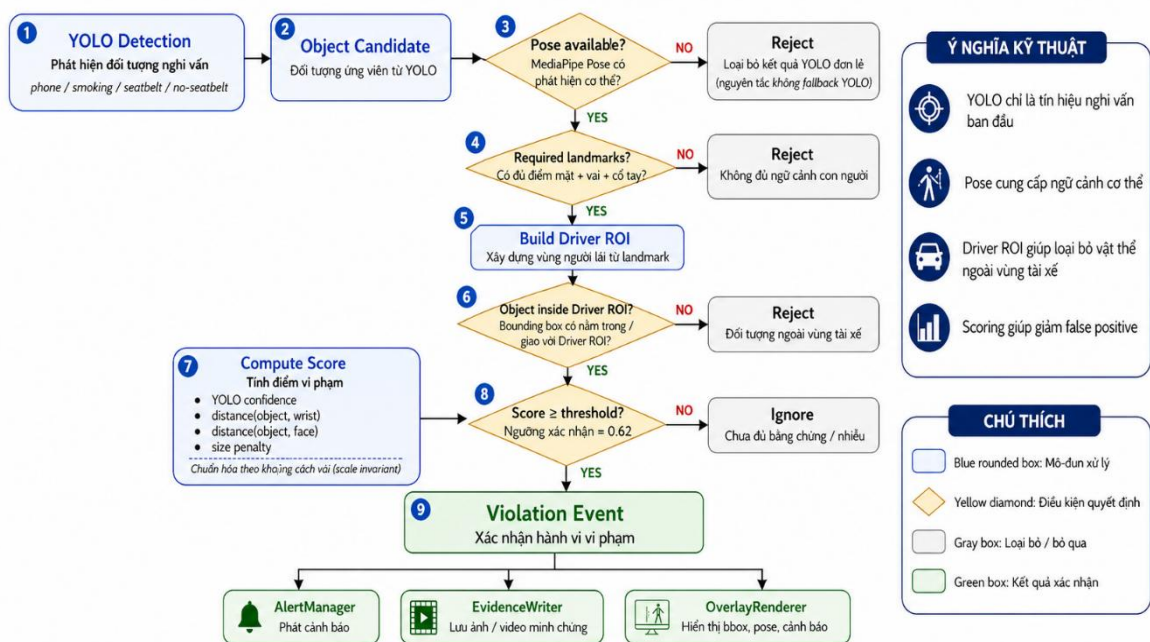
các tín hiệu nhắc nhở khác. Thứ hai, EvidenceWriter thực hiện lưu ảnh hoặc video clip minh chứng, đồng thời hỗ trợ bộ đệm các khung hình trước và sau sự kiện để phục vụ hậu kiểm. Thứ ba, OverlayRenderer đảm nhiệm việc hiển thị trực quan kết quả lên màn hình, bao gồm bounding box, nhãn đối tượng, điểm pose và trạng thái cảnh báo.

Thông qua cách tổ chức như trên, pipeline tại Edge không chỉ đảm bảo khả năng phát hiện hành vi vi phạm theo thời gian thực mà còn được tối ưu hóa rõ rệt về hiệu năng. Ba cơ chế tối ưu quan trọng được áp dụng gồm: hạ độ phân giải đầu vào, frame skipping và lazy activation cho Pose. Sự kết hợp này giúp hệ thống đạt được sự cân bằng giữa tốc độ suy luận, độ ổn định và khả năng triển khai thực tế trên thiết bị Edge.

Tóm lại, pipeline xử lý tại Edge là thành phần chính của phân hệ Edge, bảo đảm hệ thống có thể tiếp nhận dữ liệu, phân tích hành vi và đưa ra phản hồi kịp thời ngay trên phương tiện. Đây là nền tảng quan trọng để hệ thống tiếp tục thực hiện các cơ chế kết hợp YOLO và MediaPipe Pose cũng như phát hiện và cảnh báo vi phạm ở các mục tiếp theo.

3.2.2. Kết hợp YOLO và MediaPipe Pose

Trong hệ thống đề xuất, YOLO và MediaPipe Pose được kết hợp theo cơ chế suy luận hai tầng nhằm nâng cao độ chính xác trong phát hiện hành vi tài xế. YOLO đóng vai trò là tầng phát hiện nhanh các đối tượng có khả năng liên quan đến hành vi vi phạm như điện thoại, thuốc lá hoặc dây an toàn. Tuy nhiên, sự xuất hiện của một vật thể trong khung hình chưa đủ để kết luận tài xế đang thực hiện hành vi vi phạm. Vì vậy, MediaPipe Pose được sử dụng để bổ sung ngữ cảnh hình học của cơ thể người lái, giúp hệ thống đánh giá mối quan hệ không gian giữa vật thể và các bộ phận quan trọng như mặt, vai, cổ tay và vùng thân trên.

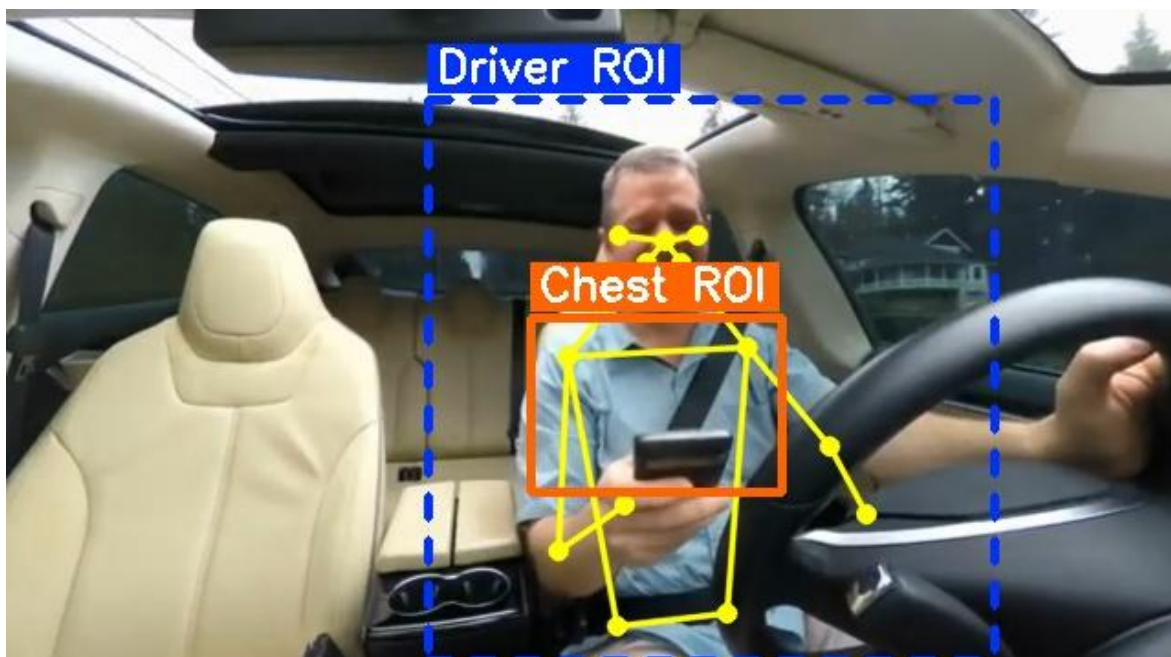


Hình 3.2.2. Cơ chế kết hợp YOLO và MediaPipe Pose trong suy luận hành vi

Nguyên tắc không fallback YOLO đơn lẻ được áp dụng cho hành vi sử dụng điện thoại: nếu không có tư thế (pose) hoặc không đủ điểm mốc (landmark) cần thiết thì loại bỏ. Đối với hành vi hút thuốc, hệ thống vẫn có cơ chế fallback dựa trên confidence thô khi không có pose, nhằm tránh bỏ sót trong điều kiện pose thất bại.

Để một phát hiện được đưa vào quá trình suy luận hành vi, hệ thống yêu cầu phải tồn tại tối thiểu các nhóm điểm mốc cơ thể quan trọng. Cụ thể, cần có ít nhất một điểm thuộc vùng mặt, một điểm thuộc vùng vai và một điểm cổ tay. Các điểm này đóng vai trò tạo hệ quy chiếu hình học cho tài xế trong khung hình. Điểm mặt được sử dụng để xác định vùng đầu và vùng miệng; điểm vai giúp ước lượng tỷ lệ cơ thể và xây dựng vùng người lái; trong khi điểm cổ tay hỗ trợ đánh giá mối liên hệ giữa vật thể và thao tác cầm nắm của tài xế.

Từ các điểm mốc thu được bởi MediaPipe Pose, hệ thống xây dựng vùng quan tâm của người lái, gọi là Driver ROI. Vùng này đại diện cho không gian cơ thể và vùng thao tác chính của tài xế trong cabin. Khi YOLO phát hiện một đối tượng, hệ thống kiểm tra xem bounding box của đối tượng đó có nằm trong hoặc giao với Driver ROI hay không. Nếu đối tượng nằm ngoài vùng người lái, kết quả phát hiện sẽ bị loại bỏ. Cơ chế này giúp hệ thống phân biệt giữa vật thể thật sự liên quan đến tài xế và vật thể nằm ở các vùng nhiễu như ghế phụ, bảng điều khiển hoặc nền phía sau.



Hình 3.2.3. Minh họa Driver ROI và Chest ROI trong khoang lái

Sau khi thỏa mãn điều kiện có đủ ngưỡng cảnh cơ thể và đối tượng nằm trong Driver ROI, hệ thống chuyển sang bước tính điểm vi phạm. Điểm vi phạm được xây dựng dựa trên nhiều yếu tố, bao gồm độ tin cậy của YOLO, khoảng cách giữa vật thể và cổ tay, khoảng cách giữa vật thể và vùng mặt, cũng như các ràng buộc về kích thước bounding box. Để đảm bảo hệ thống hoạt động ổn định với các tài xế có vóc dáng khác nhau hoặc góc camera khác nhau, khoảng cách giữa vật thể và các landmark được chuẩn hóa theo tỷ lệ vai của tài xế thay vì sử dụng trực tiếp giá trị pixel cao. Nhờ vậy, cùng một hành vi có thể được đánh giá nhất quán hơn trong nhiều điều kiện quan sát.

Về mặt nguyên lý, điểm vi phạm có thể được biểu diễn khái quát như sau:

$$Score = w1 \times YOLO_conf + w2 \times proximity(object, wrist) + w3 \times proximity(object, face) - w4 \times size_penalty$$

Trong đó, YOLO_conf phản ánh độ tin cậy của mô hình phát hiện đối tượng; proximity(object, wrist) và proximity(object, face) biểu diễn mức độ gần giữa vật thể với cổ tay và vùng mặt; còn size_penalty dùng để giảm điểm đối với các bounding box có kích thước bất thường hoặc không phù hợp với đặc trưng vật thể cần phát hiện.

Nếu điểm vi phạm vượt qua ngưỡng được thiết lập trước, ví dụ $score \geq 0.62$, hệ thống xác nhận hành vi là vi phạm và sinh ra một Violation Event. Ngược lại, nếu

điểm thấp hơn ngưỡng, phát hiện này chỉ được xem là nhiễu hoặc chưa đủ bằng chứng để cảnh báo. Nhờ sự kết hợp giữa YOLO, MediaPipe Pose và cơ chế scoring theo ngữ cảnh, hệ thống có khả năng giảm cảnh báo sai tốt hơn so với phương pháp chỉ dựa trên phát hiện đối tượng đơn lẻ.

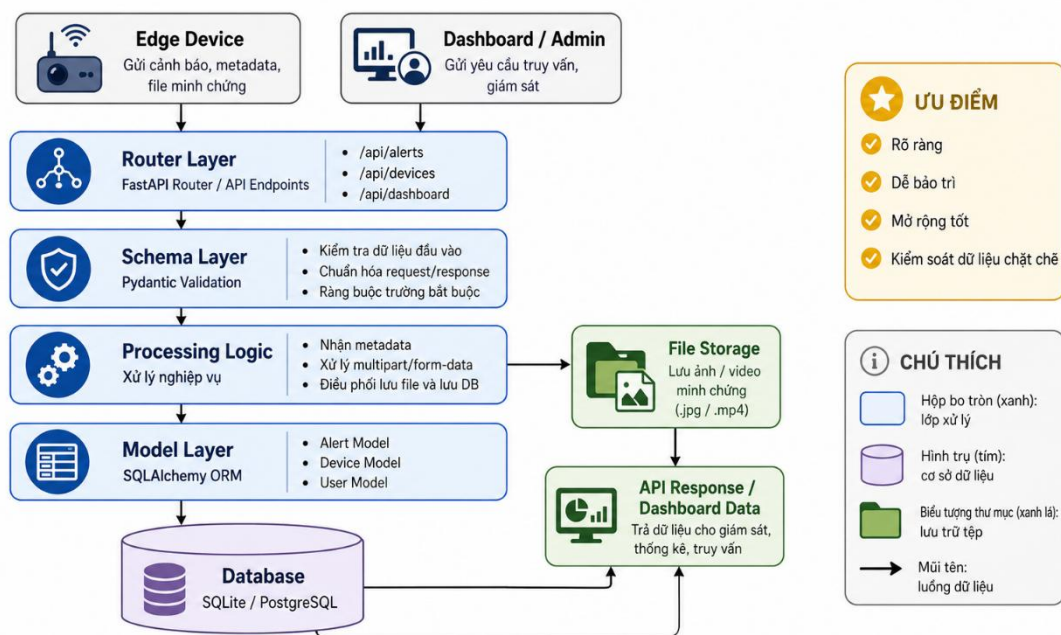
Bảng 3.2. Các tham số cấu hình chính của YOLO và Behavior Rules Engine

Tham số	Giá trị	Mô tả
conf_threshold	0.35	Ngưỡng tin cậy tối thiểu của YOLO
iou_threshold	0.45	Ngưỡng IoU cho Non-Maximum Suppression
detect_every_n_frames	2	Chỉ chạy YOLO mỗi N frame (frame skipping)
pose_every_n_frames	3	Chỉ chạy MediaPipe Pose mỗi N frame
resize_width	640	Chiều rộng resize đầu vào trước khi suy luận
alert_cooldown_sec	4	Thời gian chờ giữa 2 alert cùng loại (giây)
phone_score_threshold	0.62	Ngưỡng điểm xác nhận hành vi dùng điện thoại
smoking_score_threshold	0.62	Ngưỡng điểm xác nhận hành vi hút thuốc
phone_confirm_frames	7	Số frame liên tiếp cần để xác nhận dùng điện thoại
smoking_confirm_frames	7	Số frame liên tiếp cần để xác nhận hút thuốc
no_seatbelt_confirm_frames	12	Số frame liên tiếp cần để xác nhận không thắt dây an toàn
seatbelt_conf_threshold	0.45	Ngưỡng tin cậy cho phát hiện dây an toàn
seatbelt_margin	0.07	Chênh lệch tối thiểu giữa seatbelt và no-seatbelt
buffer_size	150	Kích thước frame buffer (deque) cho EvidenceWriter
min_visibility	0.35	Ngưỡng visibility để giữ landmark từ MediaPipe
gamma	1.18	Hệ số brighten gamma cho ảnh trước khi pose estimation

3.2.3. Cơ chế phát hiện và cảnh báo

3.3. Thiết kế phân hệ Cloud

3.3.1. Kiến trúc Backend



Hình 3.3.1. Kiến trúc phân lớp của Backend Cloud

Trong phân hệ Cloud, Backend đóng vai trò là trung tâm tiếp nhận, xử lý và quản lý dữ liệu được gửi lên từ các thiết bị Edge. Sau khi phân hệ Edge phát hiện một sự kiện vi phạm, các thông tin liên quan như loại hành vi, thời gian, độ tin cậy, mã thiết bị và tệp minh chứng sẽ được đóng gói và gửi đến Cloud thông qua API. Do đó, Backend cần được thiết kế theo hướng rõ ràng, dễ mở rộng và có khả năng kiểm soát dữ liệu chặt chẽ.

Trong hệ thống đề xuất, Backend được xây dựng bằng FastAPI, một framework Python hiện đại, phù hợp cho việc phát triển các dịch vụ REST API. FastAPI được lựa chọn vì có hiệu năng tốt, hỗ trợ xử lý bất đồng bộ, tích hợp chặt chẽ với Pydantic để kiểm tra dữ liệu đầu vào và có khả năng tự động sinh tài liệu API. Những đặc điểm này phù hợp với yêu cầu của hệ thống DMS, nơi Cloud cần tiếp nhận dữ liệu từ nhiều thiết bị Edge và cung cấp API cho giao diện giám sát.

Về mặt tổ chức mã nguồn, Backend được thiết kế theo kiến trúc phân lớp nhằm tách biệt trách nhiệm giữa các thành phần. Kiến trúc này bao gồm ba lớp chính: Router Layer, Schema Layer và Model Layer.

Router Layer chịu trách nhiệm định nghĩa các điểm cuối API của hệ thống. Đây là lớp tiếp nhận các request từ thiết bị Edge hoặc từ giao diện Dashboard. Ví dụ, khi Edge gửi dữ liệu cảnh báo lên Cloud, request được gửi tới POST /alerts

(multipart/form-data); còn giao diện Dashboard truy vấn danh sách qua GET /api/alerts.

Schema Layer được xây dựng dựa trên Pydantic, có vai trò kiểm tra và chuẩn hóa dữ liệu đầu vào. Các dữ liệu như nhân vi phạm, thời gian, mã thiết bị, độ tin cậy hoặc thông tin tệp minh chứng cần được kiểm tra trước khi lưu vào hệ thống. Việc sử dụng schema giúp giảm lỗi dữ liệu, tránh thiếu trường bắt buộc và đảm bảo dữ liệu gửi từ Edge lên Cloud có cấu trúc thống nhất.

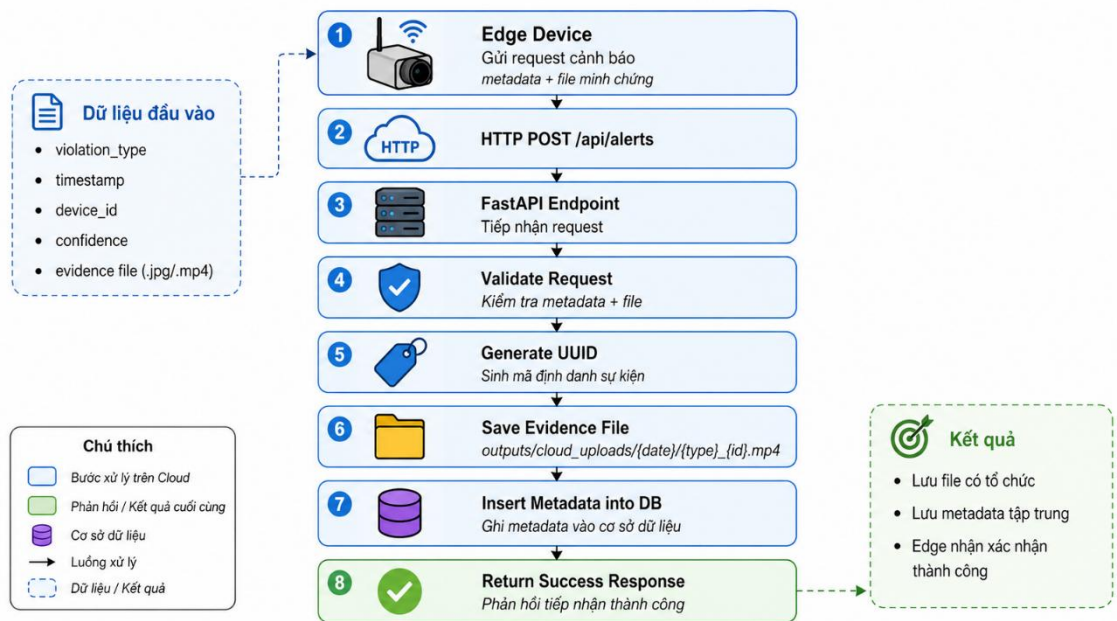
Model Layer đại diện cho cấu trúc dữ liệu được lưu trữ trong cơ sở dữ liệu. Trong hệ thống, lớp này được xây dựng thông qua ORM SQLAlchemy, giúp ánh xạ các bảng dữ liệu thành các lớp đối tượng trong Python. Cách tiếp cận này giúp việc thao tác với cơ sở dữ liệu trở nên rõ ràng hơn, đồng thời giảm sự phụ thuộc trực tiếp vào các câu lệnh SQL thủ công.

Về tổng thể, Router Layer, Schema Layer và Model Layer phối hợp với nhau để tiếp nhận request, kiểm tra dữ liệu đầu vào và ghi nhận thông tin cảnh báo vào hệ thống lưu trữ. Trong mục này, trọng tâm trình bày là cách Backend được tổ chức theo các lớp chức năng nhằm tách biệt trách nhiệm giữa định tuyến API, kiểm tra dữ liệu và thao tác cơ sở dữ liệu. Quy trình tiếp nhận metadata và file minh chứng từ Edge được trình bày chi tiết trong mục 3.3.2.

Kiến trúc phân lớp này mang lại nhiều lợi ích cho quá trình phát triển và bảo trì hệ thống. Thứ nhất, việc tách biệt Router, Schema và Model giúp mã nguồn rõ ràng hơn, mỗi lớp chỉ đảm nhiệm một nhóm trách nhiệm cụ thể. Thứ hai, hệ thống dễ mở rộng khi cần bổ sung API mới, thêm trường dữ liệu hoặc thay đổi cơ sở dữ liệu. Thứ ba, cách tổ chức này giúp giảm lỗi trong quá trình phát triển, do dữ liệu đầu vào được kiểm tra trước khi đưa vào tầng lưu trữ. Nhờ đó, Backend Cloud có thể vận hành ổn định và đóng vai trò nền tảng cho các chức năng lưu trữ, truy vấn, thống kê và giám sát hành vi tài xế.

3.3.2. Cơ chế nhận và xử lý dữ liệu

Sau khi phân hệ Edge phát hiện một hành vi vi phạm, hệ thống sẽ đóng gói dữ liệu cảnh báo và gửi lên Cloud thông qua API. Gói dữ liệu này bao gồm hai thành phần chính: metadata và file minh chứng. Metadata chứa các thông tin có cấu trúc như loại vi phạm, thời gian xảy ra, mã thiết bị, độ tin cậy và các thông tin bổ sung khác. File minh chứng có thể là ảnh hoặc video clip ngắn ghi lại thời điểm xảy ra vi phạm.



Hình 3.3.2. Quy trình tiếp nhận và xử lý dữ liệu cảnh báo tại Cloud

Tại phía Cloud, request từ Edge được tiếp nhận bởi API endpoint của FastAPI. Backend tiến hành kiểm tra dữ liệu đầu vào để đảm bảo request có đầy đủ các trường bắt buộc và file minh chứng hợp lệ. Việc kiểm tra này giúp hạn chế lỗi dữ liệu, tránh ghi nhận các cảnh báo thiếu thông tin hoặc không đúng định dạng.

Sau khi dữ liệu được xác thực, hệ thống lưu metadata vào cơ sở dữ liệu và sử dụng khóa id tự tăng làm định danh cảnh báo. Ở phiên bản hiện tại không sinh UUID riêng cho sự kiện.

Tiếp theo, file minh chứng được lưu vào thư mục lưu trữ của Cloud theo cấu trúc phân cấp dựa trên ngày phát sinh và loại vi phạm. Ví dụ, đường dẫn lưu trữ có thể có dạng:

outputs/cloud_uploads/{frames|clips|events}/{filename}

Trong đó {frames|clips|events} là nhóm lưu trữ tương ứng, còn {filename} là tên file gốc mà Edge gửi lên (ví dụ using_phone_123.jpg hoặc using_phone_123.mp4). Cách tổ chức này phù hợp với cơ chế upload hiện tại và giúp truy vết bằng chứng theo loại dữ liệu.

Song song với quá trình lưu file, Backend ghi các thông tin metadata vào cơ sở dữ liệu. Các trường dữ liệu có thể bao gồm mã sự kiện, loại vi phạm, thời gian, mã thiết bị, điểm tin cậy, đường dẫn file minh chứng và trạng thái xử lý. Việc tách biệt giữa metadata trong cơ sở dữ liệu và file minh chứng trong hệ thống tệp giúp giảm

tải cho database, đồng thời phù hợp với đặc điểm dữ liệu đa phương tiện có kích thước lớn.

Bảng 3.3. Quy trình tiếp nhận và xử lý dữ liệu cảnh báo tại Cloud

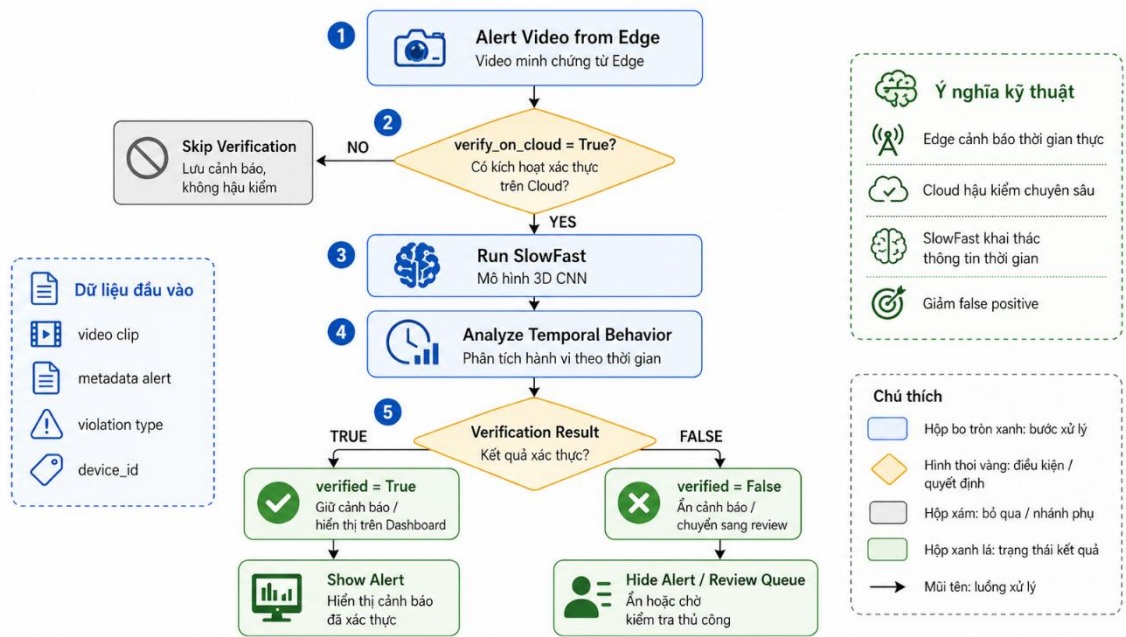
Bước	Thành phần xử lý	Kết quả
1	Edge Device	Gửi request cảnh báo gồm metadata và file minh chứng
2	HTTP POST /alerts	Truyền dữ liệu từ Edge lên Cloud (multipart/form-data)
3	FastAPI Endpoint	Tiếp nhận request từ thiết bị Edge
4	Validate Request	Kiểm tra metadata và file minh chứng
5	Database (auto-increment id)	Sinh id tự tăng cho cảnh báo khi lưu DB
6	File Storage	Lưu file vào outputs/cloud_uploads/frames clips events/{file name}
7	Database	Ghi metadata cảnh báo vào cơ sở dữ liệu
8	API Response	Trả phản hồi thành công cho Edge

Cuối cùng, sau khi hoàn tất quá trình lưu file và ghi database, Cloud trả về response thành công cho thiết bị Edge. Response này xác nhận rằng dữ liệu cảnh báo đã được tiếp nhận và lưu trữ đầy đủ. Nhờ đó, Edge có thể đánh dấu sự kiện là đã đồng bộ thành công và tiếp tục quá trình giám sát thời gian thực mà không bị gián đoạn.

Quy trình nhận và xử lý dữ liệu trên Cloud giúp hệ thống đảm bảo ba yêu cầu quan trọng: dữ liệu được tiếp nhận có kiểm soát, file minh chứng được lưu trữ có tổ chức, và metadata được quản lý tập trung để phục vụ tra cứu, thống kê và hiển thị trên dashboard.

3.3.3. Xác thực hành vi và giảm cảnh báo sai

Trong hệ thống đề xuất, phân hệ Cloud không chỉ đóng vai trò lưu trữ dữ liệu cảnh báo mà còn có thể thực hiện bước xác thực lại hành vi nhằm giảm thiểu cảnh báo sai. Cơ chế này được gọi là Cloud Verification. Khác với phân hệ Edge, vốn ưu tiên tốc độ xử lý và phản hồi tức thời, Cloud có lợi thế về tài nguyên tính toán và khả năng phân tích dữ liệu sau khi sự kiện đã được ghi nhận. Vì vậy, Cloud phù hợp để chạy các mô hình học sâu có độ phức tạp cao hơn nhằm hậu kiểm các cảnh báo được gửi từ Edge.



Hình 3.3.3. Cơ chế xác thực hành vi tại Cloud

Cơ chế xác thực trên Cloud được kích hoạt khi người dùng chủ động gọi API verify cho từng cảnh báo (endpoint POST /alerts/{id}/verify) và dịch vụ SlowFast khả dụng. Trong file config.yaml, tham số verify_on_cloud được đặt ở chế tắt (comment) theo mặc định. Ở phiên bản hiện tại, SlowFast chưa fine-tune chuyên sâu cho hành vi tài xế. Hệ thống sử dụng mô hình SlowFast đã huấn luyện sẵn trên bộ dữ liệu Kinetics và thực hiện ánh xạ các nhãn hành động có ý nghĩa gần tương đồng sang nhãn của dự án. Vì vậy, kết quả từ SlowFast chỉ được dùng như một lớp hỗ trợ hậu kiểm trên Cloud, không thay thế cơ chế phát hiện chính tại Edge.

Về mặt xử lý, video minh chứng được tách thành chuỗi frame đầu vào. Mô hình SlowFast sau đó phân tích sự thay đổi theo thời gian của tư thế, chuyển động tay, vùng mặt và sự xuất hiện của các vật thể liên quan. Kết quả đầu ra của mô hình là nhãn dự đoán hoặc xác suất cho biết hành vi trong đoạn video có thực sự là vi phạm hay không. Cách tiếp cận này đặc biệt hữu ích trong các trường hợp Edge có thể sinh cảnh báo nhầm do vật thể gây nhiễu, góc quay bất lợi hoặc tín hiệu Pose chưa ổn định.

Nếu kết quả xác thực từ Cloud cho thấy hành vi là hợp lệ, bản ghi cảnh báo sẽ được giữ lại và đánh dấu verified = True. Ngược lại, nếu mô hình xác định rằng cảnh báo không đủ cơ sở, hệ thống cập nhật trạng thái verified = False. Khi đó, cảnh báo có thể bị ẩn khỏi giao diện giám sát chính hoặc được đưa vào nhóm cần xem xét thủ công, tùy theo chính sách vận hành của hệ thống. Cơ chế này giúp hạn chế việc hiển

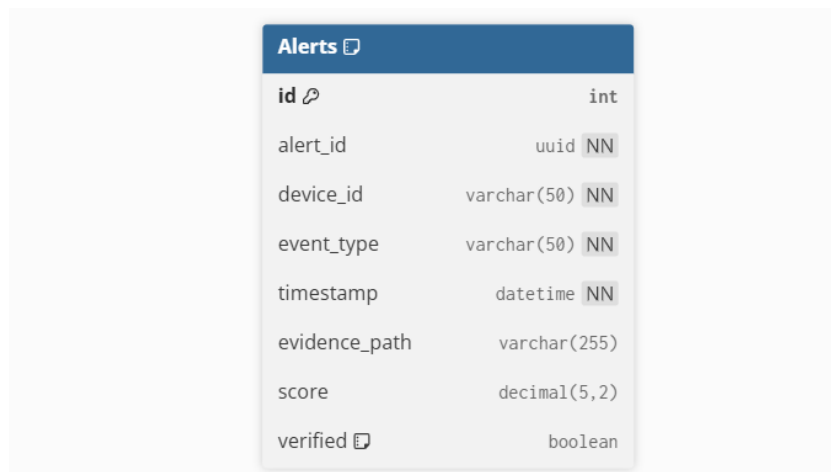
thị các cảnh báo sai cho người quản lý, đồng thời giữ cho dữ liệu thống kê phản ánh chính xác hơn hành vi thực tế của tài xế.

Việc bổ sung tầng xác thực tại Cloud tạo nên cơ chế kiểm tra hai lớp. Tầng Edge chịu trách nhiệm phát hiện nhanh và cảnh báo tức thời, trong khi tầng Cloud thực hiện hậu kiểm chuyên sâu để nâng cao độ tin cậy dữ liệu. Nhờ đó, hệ thống vừa đảm bảo yêu cầu thời gian thực trong môi trường lái xe, vừa giảm thiểu hiện tượng false positive trong quá trình quản lý và báo cáo.

Như vậy, Cloud Verification đóng vai trò như tầng kiểm chứng bổ sung sau Edge, giúp cân bằng giữa tốc độ cảnh báo thời gian thực và độ tin cậy của dữ liệu quản lý. Cơ chế này đặc biệt quan trọng trong các hệ thống DMS triển khai thực tế, nơi việc giảm cảnh báo sai có ý nghĩa trực tiếp đối với trải nghiệm người dùng và tính tin cậy của hệ thống.

3.4. Thiết kế cơ sở dữ liệu, API và giao diện giám sát

3.4.1. Thiết kế cơ sở dữ liệu



Alerts	
id	int
alert_id	uuid NN
device_id	varchar(50) NN
event_type	varchar(50) NN
timestamp	datetime NN
evidence_path	varchar(255)
score	decimal(5,2)
verified	boolean

Hình 3.4.1. Cấu trúc cơ sở dữ liệu bảng Alerts

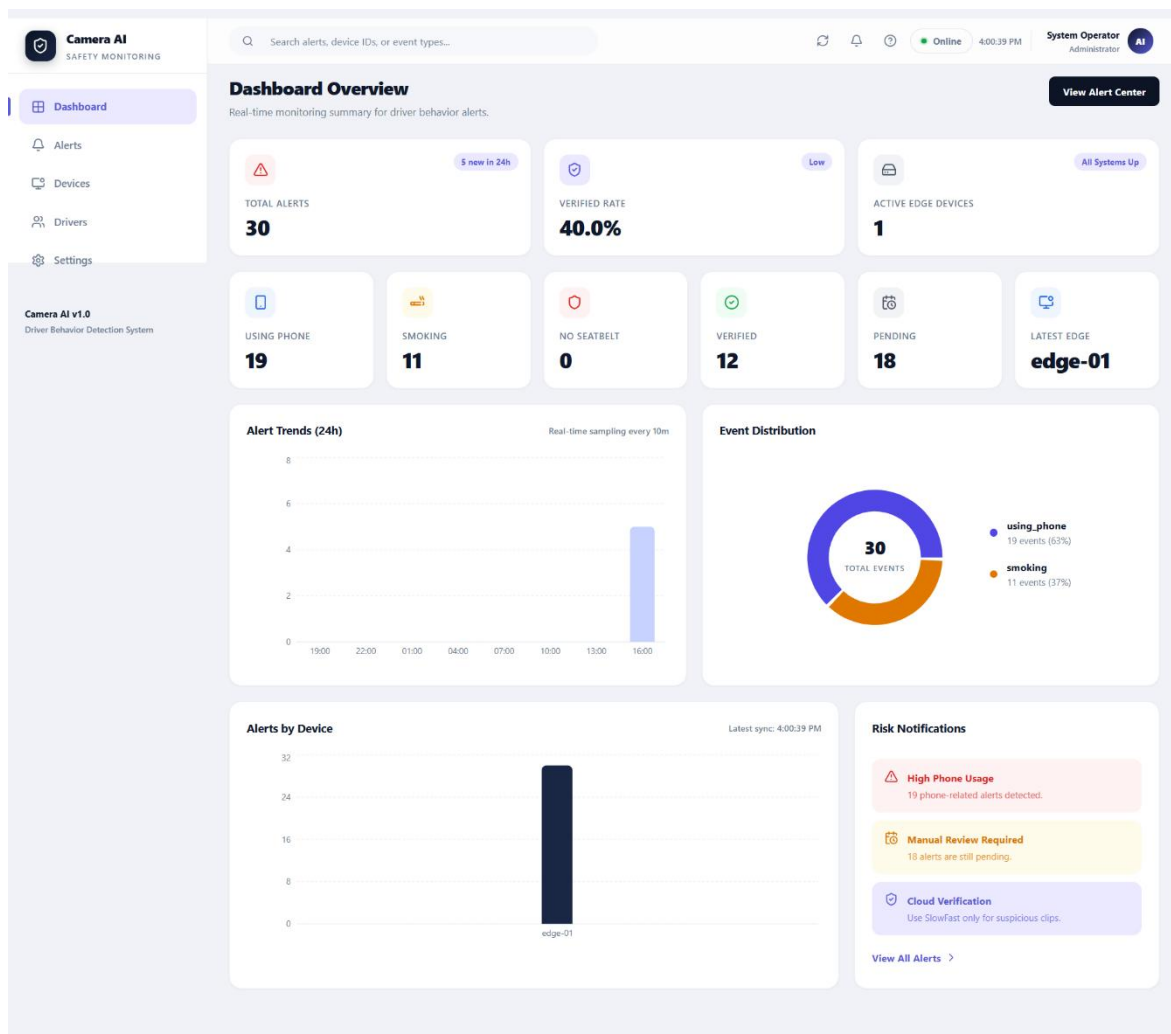
Bảng chính: Alerts. Bảng Alerts chứa toàn bộ thông tin về các sự kiện vi phạm được phát hiện bởi phân hệ Edge và gửi lên Cloud. Mỗi bản ghi cảnh báo bao gồm metadata (loại vi phạm, thời gian, độ tin cậy, mã thiết bị), siêu dữ liệu bằng chứng (đường dẫn ảnh, video clip, file JSON), và thông tin đánh giá (trạng thái xác thực, người đánh giá, ghi chú). Chi tiết các trường dữ liệu được trình bày trong bảng bên dưới.

Bảng 3.4. Cấu trúc các trường dữ liệu của bảng Alerts

Trường	Mô tả
id	khóa chính (auto-increment)
event_type	loại vi phạm
timestamp	thời gian (ISO string)
confidence	độ tin cậy
frame_index	chỉ số frame
source_device	mã thiết bị
notes	ghi chú
frame_path	đường dẫn ảnh
clip_path	đường dẫn video
event_json_path	đường dẫn file JSON
verified	trạng thái xác thực
review_status	trạng thái kiểm duyệt
verified_by	nguồn xác thực
reviewer_notes	ghi chú đánh giá
created_at	thời điểm tạo
reviewed_at	thời điểm review

3.4.2. Giao diện giám sát (Alert Center)

Giao diện giám sát được xây dựng theo kiến trúc Single Page Application bằng React 19, sử dụng Vite 7 làm công cụ xây dựng. Hệ thống bao gồm năm trang chức năng chính: Dashboard (tổng quan với biểu đồ xu hướng và phân bố sự kiện), Alerts Center (quản lý cảnh báo với bộ lọc đa tiêu chí, phân trang, xác thực hàng loạt), Devices, Drivers và Settings. Dữ liệu được tự động làm mới mỗi 5 giây. Người quản lý có thể xem bằng chứng kỹ thuật số (ảnh/clip), thực hiện đánh giá thủ công (manual review) hoặc kích hoạt xác thực tự động bằng mô hình SlowFast. Thư viện Recharts được sử dụng để vẽ biểu đồ Bar Chart (xu hướng cảnh báo 24h) và Pie Chart (phân bố sự kiện). Lucide React cung cấp hệ thống icon thống nhất cho toàn bộ giao diện.



Hình 3.4.2. Dashboard tổng quan hệ thống giám sát

Camera AI
SAFETY MONITORING

Dashboard

Alerts

Drivers

Settings

Camera AI v1.0

Driver Behavior Detection System

Search alerts, device IDs, or event types...

Online

1:37:00 PM

System Operator Administrator

Export CSV

Delete (0)

Batch Verify (0)

Alerts Center

Review driver behavior alerts, evidence clips, and Cloud verification results.

EVENT TYPE

DEVICE ID

STATUS

START DATE

END DATE

ROWS

ALL TYPES

All Devices

ALL STATUS

DD/MM/YYYY

DD/MM/YYYY

10

Apply

Reset

Recent Alerts

26 records

	ALERT ID	TIMESTAMP	TYPE	CONFIDENCE	DEVICE	STATUS	ACTIONS
☐	#AL-00033	2026-05-23T16:03:27	Smoking	100%	edge-01	UNCONFIRMED	View Verify
☐	#AL-00032	2026-05-23T16:02:24	Phone Use	100%	edge-01	VERIFIED	View Verify
☐	#AL-00031	2026-05-23T15:58:18	Smoking	100%	edge-01	UNCONFIRMED	View Verify
☐	#AL-00030	2026-05-23T15:58:10	Smoking	88%	edge-01	UNCONFIRMED	View Verify
☐	#AL-00029	2026-05-23T15:57:45	Smoking	100%	edge-01	UNCONFIRMED	View Verify
☐	#AL-00028	2026-05-23T15:57:09	Phone Use	100%	edge-01	VERIFIED	View Verify
☐	#AL-00027	2026-05-23T15:57:05	Phone Use	100%	edge-01	VERIFIED	View Verify
☐	#AL-00026	2026-05-03T20:10:54	Smoking	100%	edge-01	UNCONFIRMED	View Verify
☐	#AL-00025	2026-05-03T20:10:08	Phone Use	100%	edge-01	VERIFIED	View Verify
☐	#AL-00024	2026-05-03T20:07:35	Smoking	100%	edge-01	UNCONFIRMED	View Verify

Page 1 of 3 (26 records)

Previous

1

2

3

Next

Hình 3.4.3. Alerts Center quản lý danh sách cảnh báo

Alert Details — #EV-028

Phone Use

Timestamp: 2026-05-23T15:57:09

0:00

View JSON Event

Verify with SlowFast AI

SlowFast Verification Result

No verification results yet. Run SlowFast or verify manually.

Mark False Positive

Save as Unconfirmed

Confirm Alert

TECHNICAL METADATA

Alert ID

#EV-028

Source Device

edge-01

AI Confidence

100%

Frame Index

407

Status

Verified

REVIEWER NOTES

Enter observation details...

Hình 3.4.4. Evidence Modal dùng để xem và xác thực bằng chứng cảnh báo

59

CHƯƠNG 4: XÂY DỰNG VÀ THỰC NGHIỆM

4.1. Môi trường và công cụ triển khai

Quá trình hiện thực hóa, lập trình và kiểm thử nguyên mẫu (prototype) của hệ thống giám sát hành vi tài xế được tiến hành trong môi trường phát triển cục bộ (local environment) trên nền tảng hệ điều hành Windows. Việc lựa chọn môi trường Windows trong giai đoạn này mang lại lợi thế lớn về giao diện trực quan, cho phép các kỹ sư dễ dàng gỡ lỗi (debugging), theo dõi luồng video đầu ra và quản lý mã nguồn trước khi đóng gói và triển khai thực tế lên các bo mạch nhúng Linux. Ngôn ngữ lập trình chính là Python – nền tảng ngôn ngữ sở hữu hệ sinh thái thư viện phong phú và phổ biến hiện nay trong lĩnh vực trí tuệ nhân tạo, thị giác máy tính và phát triển dịch vụ Web (Backend). Để đảm bảo tính toàn vẹn của mã nguồn, toàn bộ hệ thống được chạy trong một môi trường ảo hóa (virtual environment - venv). Phương pháp này tuân thủ nguyên lý cô lập không gian phụ thuộc (dependency isolation), giúp quản lý chặt chẽ phiên bản của từng thư viện, giảm tối thiểu rủi ro xung đột phần mềm trên máy tính phát triển.

Về mặt công nghệ, hệ thống là sự tích hợp của nhiều thư viện được phân chia theo hai phân hệ. Tại phân hệ Edge (Xử lý tại biên), hệ thống sử dụng framework Ultralytics YOLO cho các tác vụ huấn luyện và suy luận mạng nơ-ron phát hiện đối tượng. Cùng với YOLO là thư viện MediaPipe Pose do Google phát triển, đảm nhận nhiệm vụ trích xuất 13 điểm mốc giải phẫu chính của cơ thể người lái. Để hỗ trợ luồng dữ liệu cho hai mô hình AI này, thư viện mã nguồn mở OpenCV được sử dụng để can thiệp vào xử lý đa phương tiện, chịu trách nhiệm cho các tác vụ I/O hình ảnh như: đọc luồng video, nội suy thay đổi kích thước (resizing), vẽ các hộp giới hạn/thông số lên khung hình (overlay rendering) và xuất tệp video bằng chứng. [2]

Tại phân hệ Cloud (Đám mây), kiến trúc dịch vụ được xây dựng dựa trên FastAPI – bộ khung phát triển API hiện đại có tốc độ phản hồi cao. FastAPI được kết hợp với thư viện Pydantic để thực thi cơ chế chuẩn duyệt kiểu dữ liệu đầu vào (schema validation), và thư viện SQLAlchemy để triển khai mô hình Ánh xạ quan hệ đối tượng (ORM - Object-Relational Mapping). Cấu trúc này cho phép lưu trữ và truy vấn nhật ký vi phạm một cách an toàn trên hệ quản trị cơ sở dữ liệu SQLite. Sự kết hợp của FastAPI, Pydantic và SQLAlchemy tạo ra một backend nhẹ nhưng tuân thủ các tiêu chuẩn bảo mật và thiết kế RESTful.

Hỗ trợ cho quá trình viết mã là môi trường phát triển tích hợp Visual Studio Code, kết hợp cùng hệ thống quản lý phiên bản Git nhằm theo dõi thay đổi trong vòng đời phát triển phần mềm (SDLC). Toàn bộ các cấu hình hệ thống được khai báo trong một tệp cấu hình độc lập config.yaml. Tệp này đóng vai trò như một bảng điều khiển trung tâm, cho phép kỹ sư tùy chỉnh linh hoạt các siêu tham số (hyperparameters) tại thời điểm chạy (runtime) như: ngưỡng điểm tin cậy (confidence threshold), tỷ lệ bỏ khung hình (frame skip rate), kích thước chuẩn hóa ảnh đầu vào, và địa chỉ IP của máy chủ Cloud. Cơ chế tách rời logic và cấu hình này giúp quá trình tinh chỉnh (fine-tuning) hệ thống trên nhiều kịch bản thực tế diễn ra nhanh chóng mà không cần biên dịch lại mã nguồn.

Cuối cùng, hệ thống được thiết kế để tự động nhận diện và tận dụng nền tảng điện toán song song CUDA (nếu thiết bị có trang bị card đồ họa GPU NVIDIA) nhằm tăng tốc quá trình suy luận ma trận. Trong kịch bản không có GPU phần cứng, hệ thống vẫn duy trì tính khả dụng (high availability) bằng cách tự động chuyển đổi sang chế độ suy luận thuần vi xử lý trung tâm (CPU-only inference). Tuy nhiên, tốc độ xử lý (FPS) sẽ có sự sụt giảm.

4.2. Thu thập, gán nhãn và tiền xử lý dữ liệu

Trong lĩnh vực học sâu nói chung và bài toán thị giác máy tính nói riêng, chất lượng và độ đa dạng của tập dữ liệu đóng vai trò tiên quyết, quyết định trực tiếp đến năng lực tổng quát hóa của mô hình khi triển khai vào môi trường thực tế. Nhằm đáp ứng tính phức tạp của không gian buồng lái, hệ thống dữ liệu của đề tài được xây dựng dựa trên chiến lược kết hợp đa nguồn, bao gồm ba nguồn cung cấp chiến lược nhằm bù trừ khiếm khuyết cho nhau.

Nguồn dữ liệu đầu tiên và quan trọng nhất được trích xuất từ cuộc thi học máy "State Farm Distracted Driver Detection"¹ trên nền tảng Kaggle. Đây là bộ dữ liệu chuẩn (benchmark dataset) cung cấp khoảng 102 ngàn khung hình chất lượng cao, ghi lại chân thực các hành vi của người lái xe trong môi trường buồng lái thực tế với đa dạng góc máy và góc độ vô lăng. Nơi thứ hai được tổng hợp từ nền tảng cộng đồng Roboflow Universe có một số nguồn như hao-0u4gu/driver-behavior-monitoring, dms-vewel/seatbelt-smjqj, project-h7ym6/smoking-kq44t sau đó thu thập các hình ảnh mang tính đặc thù nhằm giải quyết các bài toán khó của hệ thống, bao gồm: các

¹ Kaggle, "State Farm Distracted Driver Detection", truy cập tại: <https://www.kaggle.com/c/state-farm-distracted-driver-detection>, ngày truy cập: 20/3/2026

hình thái cảm điện thoại di động, đặc trưng hình học của dải dây an toàn, và đặc biệt là sự xuất hiện của điều thuốc (vật thể nhỏ và dễ bị che khuất). Cuối cùng, nhằm giải quyết bài toán lệch miền dữ liệu (domain shift) và tăng cường độ chính xác khi hệ thống chạy thực tế, tác giả đã tiến hành thu thập bổ sung một tập dữ liệu tự hành (custom local data) trong các kịch bản ánh sáng và tư thế khác nhau giúp mô hình học được các đặc trưng quang học sát với môi trường thử nghiệm cục bộ, từ đó giảm thiểu hiện tượng overfitting.

Sau khi tổng hợp, toàn bộ dữ liệu thô được đưa vào quy trình thiết lập bản thể học. Bộ nhãn của mô hình YOLO được phân loại thành bốn lớp đối tượng trọng tâm: phone, smoking, seatbelt, và no-seatbelt. Sự phân chia này đáp ứng trực tiếp hai mục tiêu giám sát: nhóm phone và smoking đại diện cho các tác nhân gây xao nhãng chủ động, trong khi nhóm seatbelt và no-seatbelt phục vụ đánh giá tính tuân thủ quy tắc an toàn thụ động. Các hình ảnh chưa có nhãn được xử lý thủ công thông qua các phần mềm chuyên dụng như LabelImg và giao diện gán nhãn của Roboflow, sau đó được xuất ra dưới định dạng văn bản chuẩn của YOLO (.txt) để tương thích với luồng huấn luyện của thư viện Ultralytics. Ở tầng runtime, hệ thống chuẩn hóa nhãn hành vi thành using_phone, smoking và no_seatbelt để thống nhất với pipeline cảnh báo và lưu trữ.

Một thách thức nữa khi tích hợp dữ liệu đa nguồn là sự bất đồng nhất về cách đặt tên nhãn. Chẳng hạn, cùng một khái niệm nhưng các tập dữ liệu khác nhau có thể gán nhãn là mobile phone, cell phone, phone, hoặc no_seatbelt, no-seatbelt. Nếu đưa trực tiếp vào mạng nơ-ron, mô hình sẽ phân mảnh chúng thành các lớp độc lập, gây giảm độ chính xác. Do đó, hệ thống đã thực thi một bước chuẩn hóa đồng nhất, bằng cách ánh xạ các tên gọi khác nhau về cùng một nhãn thống nhất. Sau quá trình này, toàn bộ dữ liệu được quy về bốn lớp chính của đề tài gồm: phone, smoking, seatbelt và no-seatbelt.

Sau bước chuẩn hóa nhãn, dữ liệu ảnh tiếp tục được đưa qua giai đoạn tiền xử lý hình ảnh. Mọi khung hình đều được thay đổi kích thước (resizing) về một định dạng ma trận tensor tiêu chuẩn (ví dụ: 768x768 pixel) để tối ưu hóa chi phí tính toán GPU. Hệ thống tiến hành lọc bỏ các hình ảnh kém chất lượng (quá mờ do rung lắc chuyển động hoặc mất thông tin do thiếu sáng). Đối với các ảnh có độ tương phản kém do ngược sáng hoặc bóng đổ trong cabin, thuật toán hiệu chỉnh Gamma và cân bằng biểu đồ histogram được áp dụng để cải thiện độ sáng, độ tương phản và làm rõ hơn các

đặc trưng của đối tượng. Nhờ bước tiền xử lý này, dữ liệu đầu vào trở nên đồng nhất hơn, hỗ trợ mô hình học tốt hơn trong các điều kiện quan sát khác nhau.

Cuối cùng, tập dữ liệu hoàn thiện được phân rã ngẫu nhiên theo tỷ lệ tiêu chuẩn thành ba phần độc lập: Tập huấn luyện (Train set) để mạng nơ-ron cập nhật trọng số; Tập kiểm định (Validation set) để theo dõi sai số và ngăn chặn overfitting trong quá trình học; và Tập kiểm thử (Test set) được cô lập hoàn toàn để đánh giá khách quan năng lực của mô hình sau khi huấn luyện. Về mặt tổ chức cấu trúc dự án cục bộ, các dữ liệu dành riêng cho giai đoạn chạy thử nghiệm phần mềm được quy hoạch gọn gàng vào các thư mục `data/sample_videos`, `data/sample_images` và `data/test_cases`, tạo tiền đề thuận lợi cho việc kiểm tra hồi quy từng chức năng (unit testing) của hệ thống DMS sau này.

4.3. Huấn luyện và tối ưu mô hình

Trong kiến trúc của hệ thống, mạng YOLO đóng vai trò chính trong phát hiện đối tượng. Ở bản triển khai hiện tại, mô hình YOLO11m của Ultralytics được sử dụng thông qua trọng số `best.pt` (huấn luyện 100 epochs, `imgsz=768`, `batch=16` trên Google Colab với GPU). Quyết định kỹ thuật này xuất phát từ bài toán đánh đổi (trade-off) giữa độ chính xác (mAP) và tốc độ suy luận (FPS). Nếu sử dụng các phiên bản quá nhẹ (như Nano hay Small), hệ thống sẽ đối mặt với rủi ro bỏ sót (False Negative) các vật thể có kích thước nhỏ và dễ bị lấp khuất như điều thuốc hay dây đai an toàn chìm màu. Ngược lại, việc triển khai các mô hình cỡ lớn (Large/XLarge) sẽ tốn nhiều băng thông bộ nhớ của thiết bị Edge, khiến tốc độ xử lý sụt giảm và làm giảm khả năng cảnh báo thời gian thực. Do đó, việc chọn phiên bản YOLO được cân nhắc theo dữ liệu huấn luyện và giới hạn phần cứng.

Quá trình huấn luyện mạng nơ-ron được cấu hình dựa trên một không gian tham số (hyperparameters) được tinh chỉnh cẩn thận. Hình ảnh đầu vào được chuẩn hóa về ma trận vuông với tham số `imgsz=768`, đảm bảo đủ độ phân giải không gian để trích xuất đặc trưng vật thể nhỏ. Quá trình học được thiết lập với `epochs=100` và `batch=16`, phù hợp với giới hạn bộ nhớ đồ họa (VRAM) của thiết bị huấn luyện. Nhằm ngăn chặn hiện tượng overfitting, cơ chế dừng sớm (patience=20) được kích hoạt, cho phép hệ thống tự động ngừng quá trình cập nhật trọng số nếu hàm mất mát (loss function) trên tập validation không được cải thiện. Đặc biệt, kỹ thuật Huấn luyện độ chính xác hỗn hợp tự động (Automatic Mixed Precision - `amp=True`) được sử dụng để chuyển đổi linh hoạt giữa định dạng FP32 và FP16, giúp tăng tốc độ huấn luyện lên đáng kể

mà không làm suy giảm biểu diễn toán học. Quá trình huấn luyện diễn ra ngoài pipeline triển khai; mã nguồn hiện tại tập trung vào suy luận (inference) với trọng số best.pt.

Để đối phó với môi trường ánh sáng phức tạp và góc quay hẹp trong cabin xe, hệ thống áp dụng kỹ thuật Tăng cường dữ liệu (Data Augmentation). Thuật toán Mosaic ghép ngẫu nhiên 4 bức ảnh thành một, giúp mô hình học cách nhận diện vật thể ở nhiều tỷ lệ (scale) khác nhau; Mixup trộn lẫn các pixel để làm mờ ranh giới vật thể; trong khi phép biến đổi không gian màu HSV mô phỏng các điều kiện lóa sáng hoặc thiếu sáng. Kết thúc quá trình huấn luyện, bộ theo dõi sẽ tự động kết xuất hai tập trọng số quan trọng: best.pt (lưu trữ ma trận trọng số đạt hiệu năng tốt nhất trên tập kiểm định, được dùng để triển khai thực tế) và last.pt (lưu trạng thái vòng lặp cuối cùng để dự phòng phục hồi quá trình huấn luyện).

Việc có được một mô hình best.pt chính xác mới chỉ là một phần. Để hệ thống có thể vận hành tốt trên các thiết bị Edge giới hạn tài nguyên thì cần áp dụng thêm một số kỹ thuật tối ưu hóa tại thời điểm chạy (runtime optimization) như sau:

4.3.1. Tối ưu hóa bằng chiến lược trích mẫu thời gian (Frame Skipping)

Thay vì ép vi xử lý phải đáp ứng luồng suy luận trên toàn bộ 30 khung hình mỗi giây (FPS), hệ thống áp dụng cơ chế bỏ khung có chủ đích thông qua hai siêu tham số detect_every_n_frames (dành cho YOLO) và pose_every_n_frames (dành cho MediaPipe Pose). Chiến lược này dựa trên cơ sở hành vi của con người: các hành vi vi phạm như đưa điện thoại lên tai hay cầm điều thuốc hút thường kéo dài xuyên suốt nhiều giây. Việc phân tích từng mili-giây là một sự lãng phí tài nguyên. Bằng cách chỉ kích hoạt mô hình AI ở các khung hình nhất định và tái sử dụng (nội suy) kết quả tọa độ cho các khung hình trung gian, hệ thống được giảm tải mà vẫn duy trì tính liên tục và tính toàn vẹn của logic cảnh báo.

4.3.2. Tối ưu hóa bằng chuẩn hóa độ phân giải không gian

Chi phí tính toán của mạng tích chập tỷ lệ thuận với bình phương kích thước ảnh đầu vào. Thông qua tham số resize_width, hình ảnh từ camera được thu nhỏ có kiểm soát trước khi đi vào luồng xử lý. Việc tinh chỉnh tham số này cần sự đánh giá cẩn thận: kích thước không được quá nhỏ để tránh hiện tượng suy thoái đặc trưng vi thể (khiến điều thuốc hoặc viên điện thoại biến mất thành các pixel nhiễu), nhưng cũng không được quá lớn để bảo vệ tốc độ khung hình (FPS). Sự tối ưu này giúp giải phóng một lượng lớn băng thông RAM và giảm áp lực lên bộ đệm của thiết bị Edge.

4.3.3. Biên dịch và gia tốc phần cứng với TensorRT

Đối với các kịch bản triển khai trên hệ sinh thái NVIDIA (như bo mạch Jetson hoặc máy tính có GPU rời), tệp trọng số PyTorch nguyên bản (.pt) có thể được biên dịch lại thông qua script `scripts/export_tensorrt.py`. Ở phiên bản hiện tại, hệ thống mới cung cấp bước export engine; việc nạp và chạy trực tiếp TensorRT trong pipeline chưa được tích hợp.

Tóm lại, quá trình xây dựng hệ thống AI cho thiết bị Edge là một bài toán tối ưu đa mục tiêu. Các tối ưu runtime đã hiện hữu gồm: frame skipping, chuẩn hóa độ phân giải đầu vào và kích hoạt MediaPipe Pose theo điều kiện. TensorRT hiện mới ở mức export engine và nằm trong lộ trình tích hợp ở các phiên bản tiếp theo.

4.4. Triển khai hệ thống và kiểm thử

Sau khi hoàn tất quá trình huấn luyện mạng nơ-ron và định chuẩn các siêu tham số, hệ thống bước vào giai đoạn đóng gói và triển khai thực nghiệm theo đúng kiến trúc phân tán đã thiết kế, bao gồm hai phân hệ: Edge và Cloud.

4.4.1. Triển khai phân hệ Edge (Thiết bị biên)

Phân hệ Edge được hiện thực hóa thông qua lõi điều phối (core orchestrator) đặt tại tệp mã nguồn `app/edge/pipeline_yolo_pose.py`. Đây là thành phần trung tâm chịu trách nhiệm quản lý vòng đời (lifecycle) của toàn bộ quá trình xử lý video tại thiết bị biên. Hệ thống được thiết kế để khởi chạy linh hoạt thông qua giao diện dòng lệnh (CLI) với cú pháp tiêu chuẩn:

```
python -m app.edge.main_edge --source <đường_dẫn>
```

Cơ chế này cho phép thiết bị dễ dàng thay đổi nguồn dữ liệu đầu vào (input stream), từ việc trích xuất trực tiếp qua phần cứng camera, webcam cho đến việc đọc các tệp video ngoại tuyến hoặc tệp video mẫu nhằm phục vụ công tác gỡ lỗi (debugging).

Khi luồng thực thi (pipeline) bắt đầu, hệ thống thực hiện một chu trình vòng lặp liên tục và khép kín: đọc khung hình từ bộ đệm, tiền xử lý không gian (resize), và đưa qua mạng YOLO để trích xuất vật thể. Dựa trên cơ chế chu kỳ (frame skipping) đã cấu hình, MediaPipe Pose sẽ được kích hoạt xen kẽ để nội suy cấu trúc khung xương. Điểm hội tụ của luồng dữ liệu là khi kết quả từ cả hai mô hình được đẩy vào bộ xử lý luật logic để đánh giá hành vi vi phạm. Cuối cùng, hệ thống thực thi việc ghi nhật ký (logging), cắt tệp bằng chứng và đẩy gói tin cảnh báo lên đám mây. Về mặt giao diện,

hệ thống hỗ trợ kết xuất hình ảnh trực quan (overlay rendering) bao gồm hộp giới hạn, nhãn định danh, mức độ tin cậy và bộ khung xương người lái cùng chỉ số FPS. Tuy nhiên, trong môi trường triển khai thực tế trên xe, hệ thống hỗ trợ chế độ chạy ngầm (headless mode) thông qua cờ cấu hình `show_window=false`, giúp tiết kiệm tài nguyên đồ họa để tập trung lưu trữ tệp vào thư mục bằng chứng.



Hình 4.4.1. Pipeline Edge đang chạy với detection, pose và FPS

4.4.2. Triển khai phân hệ Cloud (Đám mây trung tâm)

Phân hệ Cloud được triển khai như một hệ sinh thái Backend hoàn chỉnh dựa trên bộ khung FastAPI, với điểm neo khởi động (entry point) nằm tại tệp `app/cloud/main_cloud.py`. Mã nguồn tầng máy chủ được tổ chức nghiêm ngặt theo mô hình MVC (Model-View-Controller) kết hợp ORM. Cụ thể, lớp giao tiếp mạng và định tuyến API được định nghĩa tại `app/cloud/api_routes.py`; trong khi tầng thao tác dữ liệu được tách bạch rõ ràng qua các tệp `database.py` (cấu hình kết nối), `models.py` (định nghĩa bảng cơ sở dữ liệu) và `crud.py` (thực thi các truy vấn thêm/đọc/sửa/xóa).

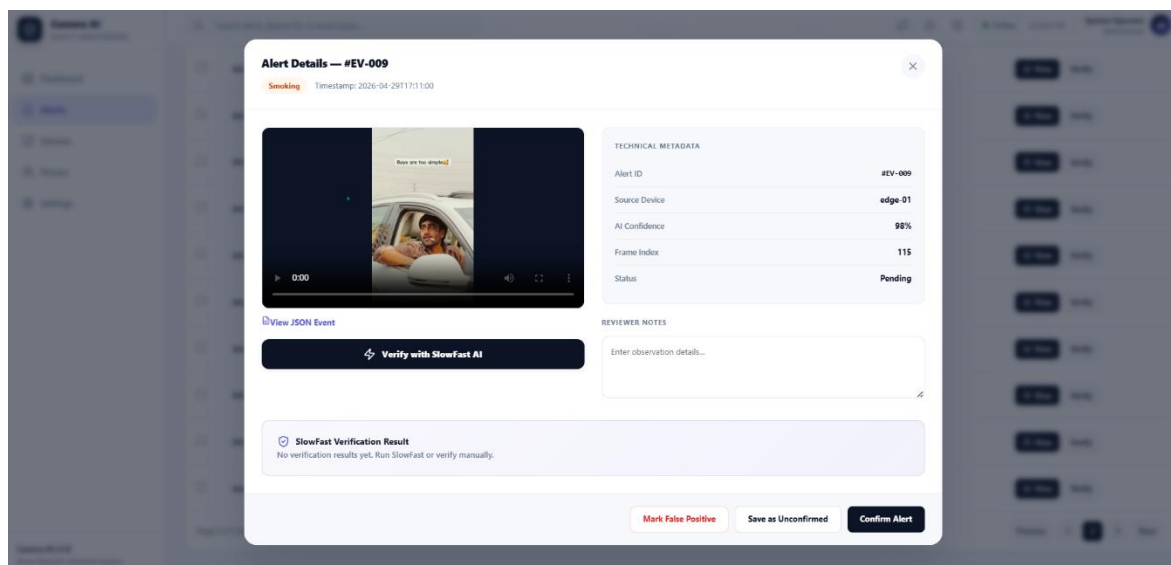
Nhiệm vụ chính của Backend là mở các cổng giao tiếp để tiếp nhận luồng dữ liệu từ hàng loạt Edge Node gửi lên. Để giải quyết bài toán băng thông, hệ thống xử lý các yêu cầu dưới định dạng tải trọng đa phần (multipart/form-data). Metadata được gửi dưới dạng các trường form (`event_type`, `timestamp`, `confidence`, `frame_index`, `source_device`, `notes`) và được lưu trực tiếp vào cơ sở dữ liệu SQLite, trong khi tệp

tin bằng chứng (binary files) được lưu trữ an toàn vào hệ thống tệp cục bộ theo cấu trúc `outputs/cloud_uploads/frames|clips|events/{filename}`. Đối với các đoạn video đã được vẽ đồ họa (overlay) trực tiếp từ Edge chuyển về, chúng sẽ được lưu trữ độc lập tại thư mục `outputs/edge_videos` để phục vụ công tác trích xuất và đối soát giao diện quản trị sau này.

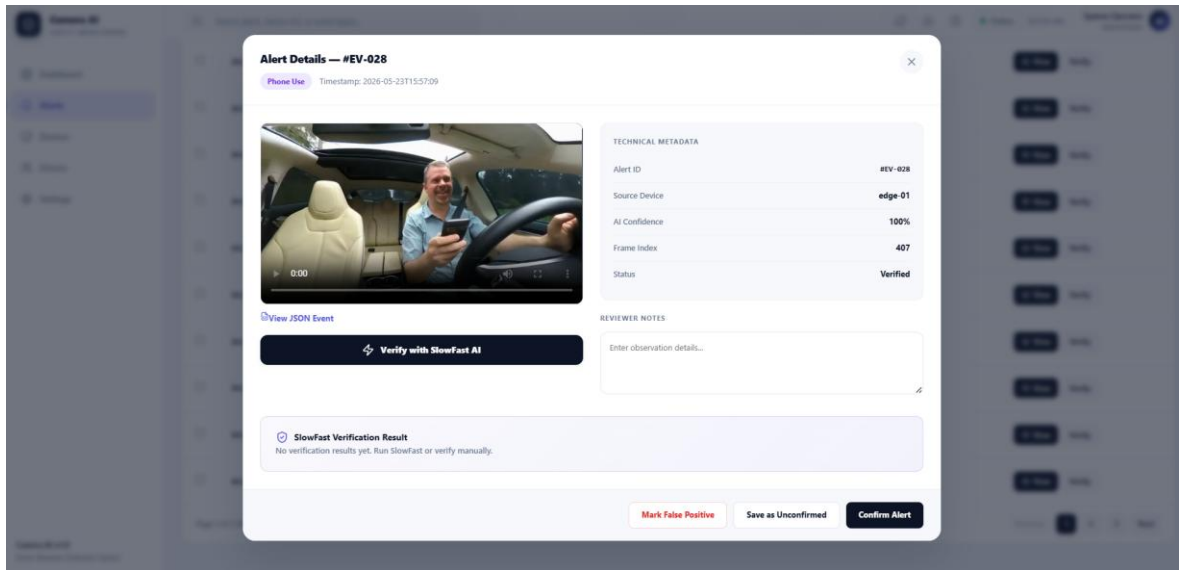
4.4.3. Thẩm định và kiểm thử kịch bản (System Validation)

Để đánh giá khả năng vận hành của hệ thống trong môi trường thực tế, quá trình thẩm định (validation) được tiến hành cẩn thận thông qua tập dữ liệu mô phỏng tại `data/sample_videos` và các tình huống được cô lập tại `data/test_cases`. Ma trận kịch bản kiểm thử (test matrix) được thiết kế đi từ các điều kiện lý tưởng đến các kịch bản biên (edge cases).

Các kịch bản kiểm thử nền tảng bao gồm: khung hình trống (không có người lái), tài xế thao tác chuẩn, tài xế sử dụng điện thoại, hút thuốc và không thắt dây an toàn. Tiếp theo là kiểm thử ở các tình huống gây nhiễu (False Positive Tests) - chẳng hạn như xuất hiện một vật thể có hình dáng tương đồng điện thoại di động nằm ở khu vực ghế phụ, hoặc khi cabin rơi vào trạng thái thiếu sáng, ngược sáng làm các đặc trưng quang học bị che khuất một phần.



Hình 4.4.3.1. Ví dụ kiểm thử phát hiện hút thuốc



Hình 4.4.3.2. Ví dụ kiểm thử phát hiện sử dụng điện thoại

Xuyên suốt quá trình chạy kịch bản, hệ thống được giám sát thông qua: Nhật ký thực thi (Runtime log) giúp theo dõi tiến trình chạy và điểm số tin cậy ở mức mã nguồn; Giao diện trực quan (Video overlay) giúp kiểm chứng tính chính xác của các hộp giới hạn tọa độ; và Bảng chứng kỹ thuật số (Evidence output) để đối soát các tệp được lưu trữ. Việc phân tích chéo giữa ba nguồn thông tin này mang lại cái nhìn toàn diện về hiệu năng của kiến trúc phần mềm. Đặc biệt, việc truy xuất thủ công các tệp bằng chứng giúp tác giả phát hiện ra những điểm mù của thuật toán, từ đó thực hiện vòng lặp tinh chỉnh (fine-tuning) các siêu tham số như ngưỡng cảnh báo (threshold), bộ luật không gian (rules) và chu kỳ bỏ khung hình (frame skipping) nhằm đạt được trạng thái vận hành tốt nhất có thể.

4.5. Đánh giá hiệu năng và kết quả

Để đo lường một cách khách quan và toàn diện năng lực của hệ thống DMS đề xuất, quá trình đánh giá thực nghiệm được triển khai dựa trên hai nhóm chỉ số đo lường (metrics) chính: Thông lượng hiệu năng xử lý (Processing Performance) và Độ chính xác trong định vị hành vi (Behavior Detection Accuracy).

4.5.1. Đánh giá hiệu năng xử lý (FPS)

Trong bối cảnh triển khai trên thiết bị Edge, thông lượng hệ thống được đo lường trực tiếp thông qua chỉ số Khung hình trên giây (Frames Per Second - FPS). Trong luồng thực thi mã nguồn, FPS không được tính toán rời rạc cho từng khung hình để tránh hiện tượng nhiễu động (fluctuation), mà được áp dụng thuật toán trung bình trượt để làm mượt thông qua biến fps_smooth. Chỉ số này phản ánh năng lực xử lý

tổng thể của đường ống (pipeline) từ lúc đọc ảnh, chạy AI, tính toán logic cho đến khi hiển thị (overlay).

Để chứng minh giá trị của các kỹ thuật tối ưu hóa đã trình bày ở phần trước, hệ thống được đặt vào môi trường kiểm thử chịu tải (stress test) với nhiều cấu hình siêu tham số khác nhau. Các biến số độc lập được đưa vào thử nghiệm bao gồm: kích hoạt đơn lẻ YOLO, kích hoạt đồng thời YOLO và MediaPipe Pose, bật/tắt cơ chế bỏ khung hình (Frame Skipping), và điều chỉnh độ phân giải đầu vào (Resize).

Bảng 4.1. Thông lượng xử lý (FPS) trung bình theo các kịch bản cấu hình

Kịch bản cấu hình	YOLO	MediaPipe Pose	Frame Skipping	Kích thước ảnh	FPS trung bình đo được
Cấu hình 1 (Cơ sở)	Bật	Tắt	Không	Gốc (848x480)	2.34
Cấu hình 2 (Toàn tải)	Bật	Bật	Không	Gốc (848x480)	2.20
Cấu hình 3 (Tối ưu chu kỳ)	Bật	Bật	Có (N=3)	Gốc (848x480)	3.04
Cấu hình 4 (Tối ưu toàn diện)	Bật	Bật	Có (N=3)	Resize (width=640)	4.75

Ghi chú: FPS được tính theo công thức $FPS = \frac{total_frames}{total_time_sec}$. Các cấu hình tương ứng được lưu trong thư mục backend/bench_configs/. Benchmark chạy trên máy tính cá nhân (CPU), trên thiết bị Edge (Jetson) FPS dự kiến cao hơn.

Qua phân tích số liệu, có thể thấy rõ quy luật đánh đổi tài nguyên. Khi kích hoạt thêm mô hình ước lượng tư thế MediaPipe, tải điện toán (computational load) tăng lên khiến FPS giảm nhẹ (từ 2.34 xuống 2.20 FPS) so với cấu hình chỉ chạy YOLO. Cơ chế bỏ khung hình có chủ đích (frame skipping với N=3) giúp giảm tần suất suy luận YOLO xuống còn 1/3, từ đó cải thiện thông lượng lên 3.04 FPS. Kết hợp với kỹ thuật thu nhỏ ảnh đầu vào (resize từ 848 xuống 640 pixel), thời gian suy luận trên mỗi frame được rút ngắn đáng kể, nâng thông lượng lên 4.75 FPS - tăng hơn 2 lần so với cấu hình cơ sở. Kết quả chứng minh hiệu quả của tổ hợp chiến lược tối ưu (frame skipping + resize) trong việc cân bằng giữa độ chính xác và hiệu năng trên thiết bị Edge.

4.5.2. Đánh giá độ chính xác thực tế

Bên cạnh thông lượng xử lý, tính chính xác của hệ thống là yếu tố quyết định mức độ tin cậy. Quá trình đánh giá được thực hiện thông qua tập dữ liệu kiểm thử bao gồm các video được gán nhãn sự thật tham chiếu từ trước.

Ngoài việc đánh giá theo từng hành vi vi phạm, đề tài cũng sử dụng các chỉ số đánh giá chuẩn của bài toán phát hiện đối tượng để phản ánh hiệu quả huấn luyện của mô hình YOLO. Các chỉ số này được trích xuất từ quá trình huấn luyện mô hình, bao gồm Precision, Recall, mAP50 và mAP50-95.

Bảng 4.2. Kết quả đánh giá mô hình YOLO trên tập kiểm thử

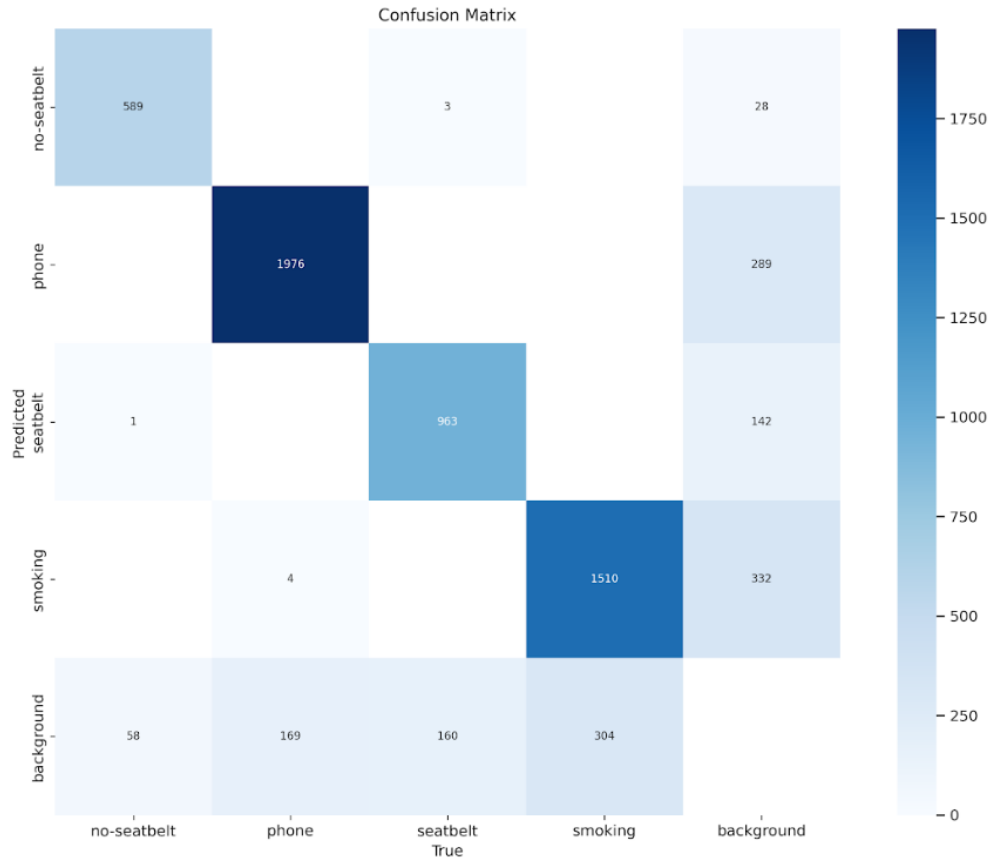
Chỉ số	Giá trị tốt nhất	Epoch
Precision	87,89%	99
Recall	82,90%	55
Recall	82,90%	55
mAP50-95	59,81%	100

Kết quả cho thấy mô hình YOLO đạt mAP50 cao nhất 88,93%, Precision cao nhất 87,89% và Recall cao nhất 82,90%. Điều này cho thấy mô hình có khả năng phát hiện tương đối tốt các đối tượng liên quan đến hành vi vi phạm trong môi trường cabin xe. Tuy nhiên, chỉ số mAP50-95 đạt 59,81%, thấp hơn đáng kể so với mAP50, phản ánh rằng độ chính xác định vị bounding box vẫn còn hạn chế khi đánh giá ở các ngưỡng IoU nghiêm ngặt hơn. Nguyên nhân chủ yếu đến từ đặc thù của bài toán như đối tượng nhỏ, che khuất một phần, thay đổi ánh sáng và góc nhìn camera trong khoang lái.

Kết quả dự đoán của hệ thống được đối chiếu để xây dựng Ma trận nhầm lẫn (Confusion Matrix), từ đó trích xuất 4 tham số cơ bản:

- True Positive (TP): Hệ thống phát hiện đúng hành vi vi phạm thực tế.
- False Positive (FP): Hệ thống cảnh báo sai (Báo có vi phạm nhưng thực tế không có).
- False Negative (FN): Hệ thống bỏ sót (Có vi phạm nhưng không phát hiện ra).
- True Negative (TN): Hệ thống im lặng chính xác khi tài xế lái xe an toàn.

Từ các tham số trên, hệ thống được đánh giá qua ba chỉ số học thuật tiêu chuẩn: Độ chuẩn xác $Precision = \frac{TP}{TP + FP}$, Độ bao phủ $Recall = \frac{TP}{TP + FN}$ và Điểm $F1 - score = \frac{2 \times Precision \times Recall}{Precision + Recall}$.



Hình 4.5.1. Ma trận nhầm lẫn của mô hình nhận diện hành vi trên tập kiểm thử

Bảng 4.3. Hiệu suất nhận diện hành vi trên tập kiểm thử

Loại hành vi vi phạm	TP	FP	FN	Precision (%)	Recall (%)	F1-Score (%)
Sử dụng điện thoại (Phone)	1165	90	88	92.86	92.98	92.92
Hút thuốc (Smoking)	523	102	87	83.69	85.74	84.70
Không thắt dây an toàn	600	47	109	92.73	84.59	88.47

Kết quả thực nghiệm cho thấy hệ thống đạt hiệu suất nhận diện tương đối tốt trên tập kiểm thử, với F1-Score lần lượt là 92,92% đối với hành vi sử dụng điện thoại, 84,70% đối với hành vi hút thuốc và 88,47% đối với hành vi không thắt dây an toàn. Trong đó, hành vi sử dụng điện thoại đạt kết quả cao nhất, còn hành vi hút thuốc có

F1-Score thấp hơn do đối tượng cần nhận diện có kích thước nhỏ và dễ bị che khuất trong môi trường cabin xe.

Các giá trị TP, FP và FN được ước lượng dựa trên số lượng instance trong tập test cùng với Precision và Recall do mô hình YOLO trả về. Precision, Recall và F1-Score được tính theo từng nhãn hành vi vi phạm.

4.5.3. Phân tích các trường hợp lỗi (Error Analysis)

Việc phân tích các ca cảnh báo lỗi (False Positives) và bỏ sót (False Negatives) giúp đánh giá được giới hạn của hệ thống. Trong quá trình thực nghiệm, các lỗi phổ biến được ghi nhận phân bổ vào ba nhóm nguyên nhân chính:

Nhiều thị giác và tương đồng hình học: Mạng YOLO thi thoảng phát sinh nhận diện nhầm (FP) các vật thể có dạng hình hộp chữ nhật màu tối (như ví tiền, sạc dự phòng, thẻ ATM) thành điện thoại di động.

Suy thoái đặc trưng do môi trường: Trong điều kiện ngược sáng gắt hoặc đi qua hầm tối, cấu trúc quang học của khung hình bị phá vỡ khiến MediaPipe Pose mất khả năng nội suy các điểm mốc (landmarks), dẫn đến việc thuật toán luật không gian bị vô hiệu hóa. Tương tự, nếu dây an toàn có màu sắc hòa lẫn với áo khoác của tài xế, mạng nơ-ron rất dễ bỏ sót (FN). Điều thuộc do kích thước quá nhỏ và thường bị ngón tay che khuất cũng là nguyên nhân gây tụt giảm chỉ số Recall.

Biến dạng phối cảnh (Perspective Distortion): Khi góc đặt camera bị xô lệch do xe xóc nảy, hệ tọa độ bị biến dạng làm cho vùng không gian quan tâm (Driver ROI) bị dịch chuyển sai lệch, khiến thuật toán logic hiệu nhầm đồ vật của ghế phụ là của tài xế.

Những trường hợp này là minh chứng cho thấy sự phức tạp của bài toán DMS thực tế so với phân tích ảnh tĩnh. Mặc dù vậy, sự can thiệp của Động cơ luật suy luận (Behavior Rules) kết hợp tư thế người lái đã giúp hệ thống lọc bỏ tương đối thành công hàng loạt các cảnh báo nhiễu (giảm FP) so với phương pháp tiếp cận chỉ dùng bounding box của YOLO thuần túy.

4.5.4. Đánh giá cơ chế lưu trữ và truy vết bằng chứng

Khác biệt cốt lõi giữa một mô hình AI phòng thí nghiệm và một phần mềm doanh nghiệp nằm ở khả năng giải trình. Trong hệ thống này, mỗi cảnh báo được sinh ra đều đi kèm với một tệp bằng chứng kỹ thuật số. Cơ chế này mang lại ba giá trị thực tiễn lớn:

Tính pháp lý và đối soát: Cung cấp tư liệu trực quan để các nhà quản lý đội xe có thể kiểm chứng thủ công (audit) tính đúng đắn của cảnh báo trước khi áp dụng các chế tài xử phạt tài xế.

Vòng lặp phản hồi dữ liệu: Các hình ảnh hệ thống nhận diện sai (cả FP và FN) sẽ được trích xuất tự động và đẩy ngược về kho dữ liệu. Quá trình này giúp làm giàu tập dữ liệu huấn luyện, tập trung vào các trường hợp khó để tái huấn luyện mô hình ở các phiên bản sau.

Cơ sở tinh chỉnh tham số: Thông qua việc phân tích bằng chứng, kỹ sư phát triển có cơ sở thực tế để điều chỉnh lại các ngưỡng tin cậy và giới hạn tỷ lệ của hàm logic hình học sao cho cải thiện độ chính xác và giảm sai lệch trong quá trình nhận diện.

Tóm lại, hệ thống không chỉ giải quyết thành công bài toán phát hiện vi phạm thời gian thực, mà còn thiết lập được một nền tảng truy vết minh bạch, tạo tiền đề cho quá trình tự học và tự cải thiện liên tục trong tương lai

Từ những kết quả thực nghiệm và quá trình đối sánh hiệu năng, có thể nhận thấy kiến trúc phân tán Hybrid Edge-Cloud là một hướng tiếp cận phù hợp cho bài toán giám sát hành vi tài xế, đặc biệt trong các kịch bản cân cân bằng giữa phản hồi thời gian thực tại Edge và quản trị dữ liệu tập trung trên Cloud. Hệ thống đã chứng minh được tính hiệu quả thông qua sự phân vai rõ ràng: thiết bị Edge giải quyết bài toán độ trễ và sự phụ thuộc vào hạ tầng viễn thông, cảnh báo tài xế trong thời gian thực; trong khi Cloud cung cấp không gian lưu trữ lớn, khả năng quản trị tập trung và khả năng triển khai các thuật toán hậu kiểm chuyên sâu.

Hệ thống không hoàn toàn dựa vào mạng nơ-ron để phát hiện đối tượng mà kết hợp chéo với lưới giải phẫu cơ thể (MediaPipe Pose) để khắc phục điểm yếu "mù ngữ cảnh" của YOLO thuần túy. Bằng cách thiết lập các ràng buộc logic giữa vật thể và tư thế người lái, hệ thống đã bóc tách thành công các tác nhân gây nhiễu từ môi trường xung quanh (như điện thoại của hành khách ghế phụ), giúp kéo giảm tỷ lệ cảnh báo sai.

Xét trên phương diện ứng dụng thực tiễn, cơ chế lưu vết bằng chứng kỹ thuật số giúp hệ thống có thể triển khai như một phần mềm quản trị cấp doanh nghiệp. Hệ thống cung cấp khả năng giải trình minh bạch thông qua hình ảnh/video đính kèm giúp người quản trị dễ dàng đối soát, và tạo vòng lặp phản hồi dữ liệu để liên tục cải tiến mô hình trong tương lai.

Dù đạt được một số kết quả khả quan, hệ thống hiện tại vẫn tồn tại một số hạn chế đặc thù. Thứ nhất, độ nhạy của thuật toán MediaPipe Pose suy giảm mạnh trong điều kiện bức xạ ánh sáng yếu hoặc khi cơ thể người lái bị che khuất một phần bởi vô lăng và nội thất. Thứ hai, sự tương đồng hình học tĩnh thỉnh thoảng vẫn đánh lừa mạng YOLO, đặc biệt khi các vật thể hình chữ nhật màu đen lọt vào khung hình mà thông tin Pose không đủ rõ ràng để bác bỏ. Thứ ba, mô hình nhận diện hành động theo thời gian như SlowFast hiện mới dừng ở mức baseline (heuristic mapping từ nhãn Kinetics-400, chưa fine-tune chuyên sâu cho hành vi tài xế) và chỉ được kích hoạt theo yêu cầu tại tầng Cloud, chưa được tích hợp tự động vào luồng xử lý bắt buộc đối với mọi cảnh báo.

Định hướng cho những nghiên cứu và nâng cấp ở giai đoạn tiếp theo là ưu tiên mở rộng và cải thiện chất lượng dữ liệu. Mô hình cần được tái huấn luyện trên một tập dữ liệu cabin thực tế lớn hơn, bao quát nhiều điều kiện thời tiết, dải ánh sáng và đa dạng các dòng xe cơ giới. Để chinh phục bài toán nhận diện ban đêm, việc nâng cấp phần cứng tích hợp Camera hồng ngoại (IR Camera) kết hợp các thuật toán tăng cường ảnh thiếu sáng là yêu cầu bắt buộc. Về mặt tối ưu hóa vi kiến trúc phần cứng, việc lượng tử hóa mô hình xuống định dạng FP16/INT8 và biên dịch qua bộ tăng tốc TensorRT sẽ được triển khai toàn diện trên các bo mạch NVIDIA Jetson nhằm đẩy thông lượng FPS lên giới hạn cao hơn. Cuối cùng, việc hoàn thiện pipeline xác thực đám mây bằng mô hình SlowFast sẽ giúp giải pháp DMS này trở thành một hệ thống giám sát an toàn giao thông có độ tin cậy cao, có thể thương mại hóa và triển khai trên hàng loạt đội xe doanh nghiệp.

CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết quả đạt được

Trong khuôn khổ của khóa luận tốt nghiệp, đề tài đã hoàn thành chu trình nghiên cứu, thiết kế, lập trình và thực nghiệm thành công Hệ thống giám sát hành vi tài xế dựa trên nền tảng kiến trúc phân tán Hybrid Edge–Cloud. Hệ thống được chế tạo với mục tiêu tự động hóa quá trình phát hiện các hành vi vi phạm có rủi ro cao trong buồng lái – bao gồm sử dụng điện thoại di động, hút thuốc và không thắt dây an toàn – đồng thời giải quyết vấn đề cân bằng giữa khả năng phản ứng thời gian thực tại hiện trường và năng lực quản trị, lưu trữ tập trung trên đám mây.

Về phương diện lý luận, khóa luận đã tìm hiểu và trình bày các kiến thức nền tảng cho việc xây dựng hệ thống giám sát hành vi tài xế. Các nội dung chính bao gồm phát hiện đối tượng bằng YOLO, ước lượng tư thế bằng MediaPipe Pose, nhận diện hành động bằng SlowFast và mô hình triển khai Hybrid Edge–Cloud. Trên cơ sở đó, đề tài lựa chọn các công nghệ phù hợp và vận dụng vào quá trình thiết kế, xây dựng hệ thống thử nghiệm.

Về phương diện thiết kế kiến trúc, hệ thống đã được xây dựng theo mô hình Hybrid Edge–Cloud gồm hai phân hệ chính. Phân hệ Edge được triển khai để xử lý luồng hình ảnh trực tiếp từ camera, thực hiện phát hiện đối tượng, ước lượng tư thế và đưa ra cảnh báo tại chỗ khi phát hiện hành vi vi phạm. Phân hệ Cloud tiếp nhận dữ liệu cảnh báo từ Edge thông qua API, lưu trữ thông tin vi phạm, quản lý bằng chứng hình ảnh/video và cung cấp dữ liệu cho giao diện giám sát. Cách tổ chức này giúp hệ thống phản hồi nhanh tại thiết bị biên, đồng thời vẫn có khả năng lưu trữ và quản lý dữ liệu tập trung trên máy chủ.

Trong phân hệ Edge, hệ thống không chỉ sử dụng YOLO để phát hiện các đối tượng như điện thoại, điều thuốc và dây an toàn, mà còn kết hợp thêm thông tin tư thế người lái từ MediaPipe Pose. Dựa trên các điểm mốc cơ thể, hệ thống kiểm tra vị trí tương đối giữa vật thể và người lái, chẳng hạn khoảng cách từ điện thoại đến tay hoặc vùng đầu, cũng như vị trí của vật thể trong vùng quan tâm của tài xế. Cơ chế xác thực ngữ cảnh này được bổ sung nhằm hạn chế một số cảnh báo sai do các vật thể không liên quan trong cabin gây ra.

Để giải quyết bài toán nút thắt cổ chai về tài nguyên phần cứng trên thiết bị nhúng, hệ thống đã được tích hợp một tổ hợp các giải pháp tối ưu hóa phần mềm. Bằng việc phối hợp kỹ thuật thay đổi kích thước ảnh đầu vào, trích mẫu thời gian (Frame

Skipping), và đặc biệt là chiến lược kích hoạt lười chỉ chạy MediaPipe khi YOLO phát hiện vật thể khả nghi, hệ thống đã giải phóng hàng chục triệu phép toán dư thừa mỗi giây. Kết hợp cùng cơ chế đóng băng cảnh báo (Cooldown Mechanism) để tránh cảnh báo liên tục và bộ đệm vòng (Ring Buffer) để lưu vết video trước/sau sự kiện. Những kỹ thuật này giúp giảm bớt khối lượng xử lý không cần thiết và hỗ trợ hệ thống vận hành ổn định hơn trong quá trình thử nghiệm.

Tại tầng Cloud, đề tài đã xây dựng Backend bằng FastAPI, kết hợp cùng hệ quản trị cơ sở dữ liệu SQLite và ORM SQLAlchemy. Các RESTful API được xây dựng nhằm tiếp nhận metadata, hình ảnh/video bằng chứng từ Edge và cung cấp dữ liệu cho giao diện giám sát. Ngoài ra, hệ thống có bổ sung chức năng hậu kiểm bằng SlowFast ở mức thử nghiệm. Ở phiên bản hiện tại, SlowFast sử dụng mô hình đã được huấn luyện sẵn trên bộ dữ liệu Kinetics-400 và ánh xạ một số nhãn hành động gần tương đồng sang nhãn của đề tài. Chức năng này chỉ kích hoạt khi người dùng chủ động gọi API verify cho từng cảnh báo và dịch vụ SlowFast khả dụng. Trong phiên bản hiện tại, SlowFast chưa được fine-tune chuyên sâu cho hành vi tài xế, do đó đây là hướng cần tiếp tục hoàn thiện trong tương lai.

Về phương diện thực nghiệm, nguyên mẫu phần mềm đã được kiểm thử trên một số kịch bản mô phỏng như sử dụng điện thoại, hút thuốc, không thắt dây an toàn và một số trường hợp có vật thể gây nhiễu trong cabin. Kết quả cho thấy hệ thống có thể phát hiện hành vi vi phạm, tạo cảnh báo, lưu lại bằng chứng và đồng bộ dữ liệu lên Cloud. Nhìn chung, đề tài đã hoàn thành các mục tiêu chính ở mức nguyên mẫu thử nghiệm, đồng thời tạo nền tảng để tiếp tục cải thiện độ chính xác, khả năng tối ưu và mức độ ổn định khi triển khai trong môi trường thực tế.

5.2. Hạn chế của hệ thống

Mặc dù nguyên mẫu (prototype) của hệ thống đã chứng minh được tính khả thi và đạt được những kết quả nhất định trong điều kiện thực nghiệm, song đề tài vẫn còn tồn tại một số hạn chế nhất định về phương diện dữ liệu, thuật toán và khả năng thích ứng phân cứng.

Hạn chế lớn nhất là sự nhạy cảm của mô hình với môi trường quang học và chất lượng dữ liệu đầu vào. Trong không gian buồng lái thực tế, sự biến thiên cực đoan của bức xạ ánh sáng (như ngược sáng gắt vào ban ngày hoặc thiếu sáng trầm trọng vào ban đêm) gây ra hiện tượng suy thoái đặc trưng hình ảnh. Điều này làm giảm đáng kể độ tin cậy của mạng YOLO và khiến MediaPipe Pose đánh mất tọa độ các

điểm mốc giải phẫu. Nguyên nhân nằm ở giới hạn về quy mô và tính phân phối đa dạng của tập dữ liệu huấn luyện. Bộ dữ liệu hiện tại chưa đủ độ phủ để bao quát toàn vẹn các tình huống phức tạp trong thực tế, điển hình như: sự sai khác về cấu trúc nội thất của từng dòng xe, sự đa dạng trong vóc dáng và trang phục của tài xế, sự biến thiên của góc đặt camera hay các mức độ che khuất.

Tiếp theo, về logic thuật toán, phương pháp xác định hành vi đang được vận hành chủ yếu dựa trên hệ thống luật tắt định với các ngưỡng không gian được thiết lập thủ công. Dù phương pháp này mang lại ưu điểm về tốc độ tính toán và khả năng giải trình rõ ràng, nó lại thiếu đi tính thích ứng động. Trong kịch bản camera bị xô lệch khỏi góc chuẩn ban đầu do phương tiện xóc nảy, các hệ quy chiếu tọa độ sẽ bị sai lệch, dẫn đến việc các luật không gian mất đi tính chính xác. Hơn nữa, không gian nhận diện hiện tại mới chỉ giới hạn trong ba hành vi (dùng điện thoại, hút thuốc, không thắt dây an toàn). Hệ thống đang chưa xem xét đến các hành vi mở mức nhỏ hơn (Micro-biometric states) như: tần suất chớp mắt thể hiện trạng thái buồn ngủ, ngáp liên tục, hay mất tập trung do quay đầu sang hướng khác quá lâu.

Cuối cùng, xét về kiến trúc tổng thể và triển khai vật lý, hệ thống vẫn còn một số hạn chế. Phân hệ xác thực ngữ cảnh tại Đám mây bằng mạng nơ-ron SlowFast hiện mới dừng ở mức baseline (heuristic mapping từ nhãn Kinetics-400 thông qua keyword matching, chưa fine-tune chuyên sâu cho hành vi tài xế). Hệ thống chỉ suy ra nhãn dự án dựa trên các từ khóa liên quan trong tập 400 lớp hành động của Kinetics, do đó độ chính xác xác thực còn hạn chế đối với các hành vi đặc thù của tài xế. Song song đó, ở phân hệ Edge, các bài đo kiểm thực nghiệm mới chỉ đánh giá hiệu năng phần mềm cục bộ mà chưa trải qua các chu kỳ kiểm thử sức chịu đựng vật lý dài hạn trên các bo mạch nhúng chuyên dụng cho công nghiệp ô tô (như NVIDIA Jetson Orin Nano). Do đó, những tham số vận hành quan trọng như biểu đồ tản nhiệt, giới hạn tiêu thụ điện năng và độ ổn định của bộ nhớ khi hệ thống chạy liên tục 24/7 dưới thời tiết khắc nghiệt vẫn chưa được khảo sát và định lượng đầy đủ.

5.3.Hướng phát triển trong tương lai

Từ các kết quả đạt được và các hạn chế còn tồn tại, một số hướng phát triển của đề tài được đề xuất như sau:

Thứ nhất: thu thập và xây dựng kho dữ liệu đa phương thức, đa miền, bao gồm nhiều dòng phương tiện thương mại khác nhau, các kịch bản chiếu sáng cực đoan và đa dạng hóa đặc điểm nhân chủng học của tài xế. Đồng thời tích hợp phần cứng

Camera hồng ngoại (IR Camera) kết hợp với tập dữ liệu quang phổ hồng ngoại là điều kiện giúp tăng cường khả năng giám sát 24/7.

Thứ hai: Tầng nhận thức hình ảnh có thể được thay thế bằng các kiến trúc State-of-the-Art (SOTA) hiện đại hơn như RT-DETR để bắt kịp xu hướng End-to-End Object Detection, hoặc ứng dụng Vision Transformers (ViT) nhằm tận dụng cơ chế tập trung (Attention Mechanism) giúp mô hình hiểu ngữ cảnh toàn cục tốt hơn. Đặc biệt, đối với tầng phân tích hành vi, hệ thống có thể được mở rộng từ việc chỉ phân tích các ảnh tĩnh/tọa độ tĩnh sang việc phân tích chuỗi động học thời gian (Spatiotemporal Analysis) bằng cách triển khai thực tế các mạng 3D CNN như SlowFast hoặc Video Swin Transformer, giúp xử lý các vi phạm có tính chất chu kỳ [3].

Thứ ba: mở rộng không gian giám sát hành vi sinh trắc học. Các thuật toán chuyên sâu như Gaze Estimation (ước lượng hướng nhìn) để phát hiện sự mất tập trung, Head Pose Estimation (tính toán góc quay đầu Yaw/Pitch/Roll), và đặc biệt là phân tích chỉ số PERCLOS (Percentage of Eye Closure) để đo lường và cảnh báo sớm trạng thái buồn ngủ, ngủ gật sẽ được tích hợp thành các module độc lập chạy song song tại Edge.

Thứ tư: gia tốc phần cứng và tối ưu vi kiến trúc. Hệ thống có thể được triển khai trên các nền tảng SoC công nghiệp mạnh mẽ hơn như NVIDIA Jetson Orin. Tại đây, quá trình lượng tử hóa mô hình xuống chuẩn INT8 kết hợp với trình biên dịch TensorRT sẽ được tối ưu hóa. Đồng thời, kiến trúc mã nguồn sẽ được viết lại bằng C++ kết hợp với các luồng xử lý bất đồng bộ nhằm ép xung thông lượng FPS, giảm thiểu tài nguyên RAM và điện năng tiêu thụ.

Cuối cùng, về hoàn thiện hệ sinh thái Đám mây và triển khai thực địa. Phần hệ Backend sẽ được nâng cấp thành một kiến trúc phần mềm dạng dịch vụ hỗ trợ đa khách hàng. Giao diện quản trị sẽ được phát triển thành một hệ thống Telemetry chuyên nghiệp, cung cấp các biểu đồ thống kê thời gian thực, quản lý phân quyền (RBAC) và tự động thiết lập hồ sơ rủi ro cho từng cá nhân. Đồng thời, hệ thống sẽ bước vào giai đoạn triển khai thử nghiệm thực địa trên các đội xe buýt hoặc xe tải đường dài. Quá trình kiểm thử trong môi trường thực tế với sự tham gia của con người, cung cấp những đánh giá về Chỉ số thời gian giữa các lần hỏng hóc, tỷ lệ chấp nhận của người dùng, từ đó đo lường chính xác hiệu quả giảm thiểu tai nạn giao thông mang lại.

TÀI LIỆU THAM KHẢO

- [1] Bazarevsky, V., Grishchenko, I., Raveendran, K., Zhu, F., Zhang, F., and Grundmann, M. (2020), "BlazePose: On-device Real-time Body Pose tracking," arXiv preprint arXiv:2006.10204.
- [2] Bradski, G. (2000), "The OpenCV Library," Dr. Dobb's Journal of Software Tools, 25(11), pp. 120-125.
- [3] Carion, N., Massa, F., Synnaeve, G., Usunier, N., Kirillov, A., and Zagoruyko, S. (2020), "End-to-End Object Detection with Transformers," In Proceedings of the European Conference on Computer Vision (ECCV), pp. 213-229.
- [4] Fan, H., Xiong, B., Mangalam, K., Li, Y., Yan, Z., Malik, J., and Feichtenhofer, C. (2021), "Multiscale Vision Transformers," In Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), pp. 6824-6835.
- [5] Feichtenhofer, C., Fan, H., Malik, J., and He, K. (2019), "SlowFast Networks for Video Recognition," In Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), pp. 6202-6211.
- [6] He, K., Zhang, X., Ren, S., and Sun, J. (2016), "Deep Residual Learning for Image Recognition," In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 770-778.
- [7] Jacob, B., Kligys, S., Chen, B., Zhu, M., Tang, M., Howard, A., Adam, H., and Kalenichenko, D. (2018), "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference," In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 2704-2713.
- [8] Kingma, D.P. and Ba, J. (2015), "Adam: A Method for Stochastic Optimization," In Proceedings of the International Conference on Learning Representations (ICLR).
- [9] Lin, T.Y., Dollár, P., Girshick, R., He, K., Hariharan, B., and Belongie, S. (2017), "Feature Pyramid Networks for Object Detection," In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 2117-2125.

- [10] Liu, W., Anguelov, D., Erhan, D., Szegedy, C., Reed, S., Fu, C.Y., and Berg, A.C. (2016), "SSD: Single Shot MultiBox Detector," In Proceedings of the European Conference on Computer Vision (ECCV), pp. 21-37.
- [11] Lugaresi, C., Tang, J., Nash, H., McClanahan, C., Uboweja, E., Hays, M., Zhang, F., Chang, C., Yong, M.G., Lee, J., Chang, W., Hua, W., Georg, M., and Grundmann, M. (2019), "MediaPipe: A Framework for Building Perception Pipelines," arXiv preprint arXiv:1906.08172.
- [12] Ramstedt, S. and Pal, C. (2017), "Real-time Traffic Sign Detection, Classification and Post-Processing," arXiv preprint arXiv:1709.07897.
- [13] Redmon, J. and Farhadi, A. (2018), "YOLOv3: An Incremental Improvement," arXiv preprint arXiv:1804.02767.
- [14] Redmon, J., Divvala, S., Girshick, R., and Farhadi, A. (2016), "You Only Look Once: Unified, Real-Time Object Detection," In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 779-788.
- [15] Ren, S., He, K., Girshick, R., and Sun, J. (2015), "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," In Proceedings of the Advances in Neural Information Processing Systems (NeurIPS), pp. 91-99.